OperationalController

Редакция: 2026-10-05. Диаграмм: 38. Документов: 32.

Диаграммы сохранены векторными. Ссылки на включённые схемы и описания открывают соответствующие страницы этого PDF. Оглавление и закладки позволяют перейти к любому разделу.

Размеры страниц схем подобраны по содержимому для сохранения читаемости текста. Внешние сетевые ссылки, если они есть в исходниках, сохранены как URL.

Диаграммы

OperationalController - OperationalController

OperationalController-1 - OperationalController

OperationalController-2 - OperationalController

OperationalController-3 - OperationalController

OperationalController-4 - OperationalController

OperationalController-5 - OperationalController

OperationalController-6 - OperationalController

OperationalController-7 - OperationalController

OperationalController-8 - OperationalController

OperationalController-9 - OperationalController

OperationalController-10 - OperationalController

OperationalController-11 - OperationalController

OperationalController-12 - OperationalController

OperationalController-13 - OperationalController

OperationalController-14 - OperationalController

OperationalController-15 - OperationalController

OperationalController-16 - OperationalController

OperationalController-17 - OperationalController

OperationalController-18 - OperationalController

OperationalController-19 - OperationalController

OperationalController-20 - OperationalController

OperationalController-21 - OperationalController

OperationalController-22 - OperationalController

OperationalController-23 - OperationalController

OperationalController-24 - OperationalController

OperationalController-25 - OperationalController

OperationalController-26 - OperationalController

OperationalController-27 - OperationalController

OperationalController-28 - OperationalController

OperationalController-29 - OperationalController

OperationalController-30 - OperationalController

GetSurfaceFigures_ActDiag - OperationalController

ModelView - OperationalController

ProjectEvents - OperationalController

Sew3DFrom2D - OperationalController

ChangeObjsCollectionsColor_ActDiag - pdfs

CrossSectionMaker - pdfs

GetVolumeFigures_ActDiag - pdfs

Описания и полные сигнатуры

OperationalController - описание

OperationalController-1 - описание

OperationalController-2 - описание

OperationalController-3 - описание

OperationalController-4 - описание

OperationalController-5 - описание

OperationalController-6 - описание

OperationalController-7 - описание

OperationalController-8 - описание

OperationalController-9 - описание

OperationalController-10 - описание

OperationalController-11 - описание

OperationalController-12 - описание

OperationalController-13 - описание

OperationalController-14 - описание

OperationalController-15 - описание

OperationalController-16 - описание

OperationalController-17 - описание

OperationalController-18 - описание

OperationalController-19 - описание

OperationalController-20 - описание

OperationalController-21 - описание

OperationalController-22 - описание

OperationalController-23 - описание

OperationalController-24 - описание

OperationalController-25 - описание

OperationalController-26 - описание

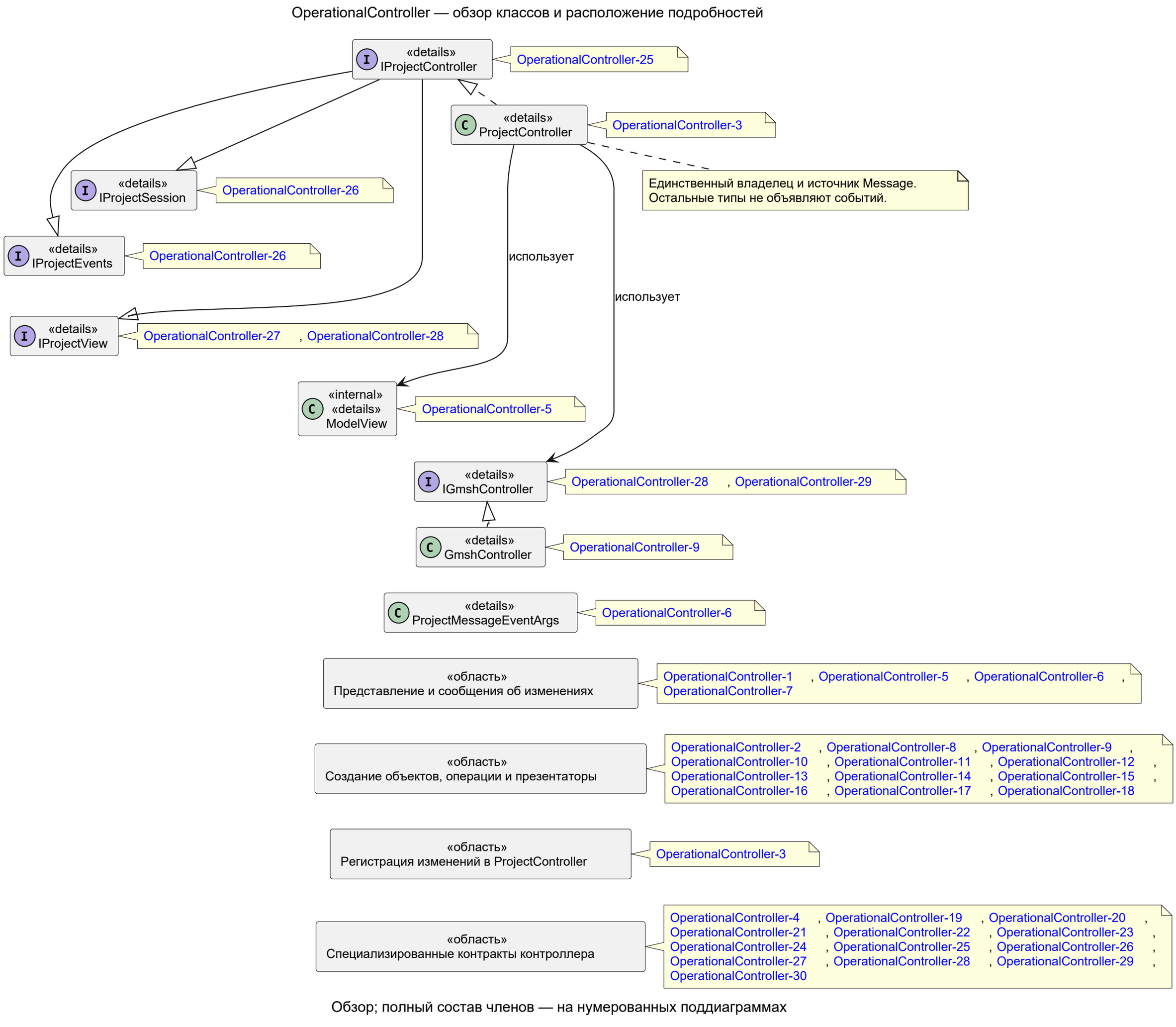
OperationalController-27 - описание

OperationalController-28 - описание

OperationalController-29 - описание

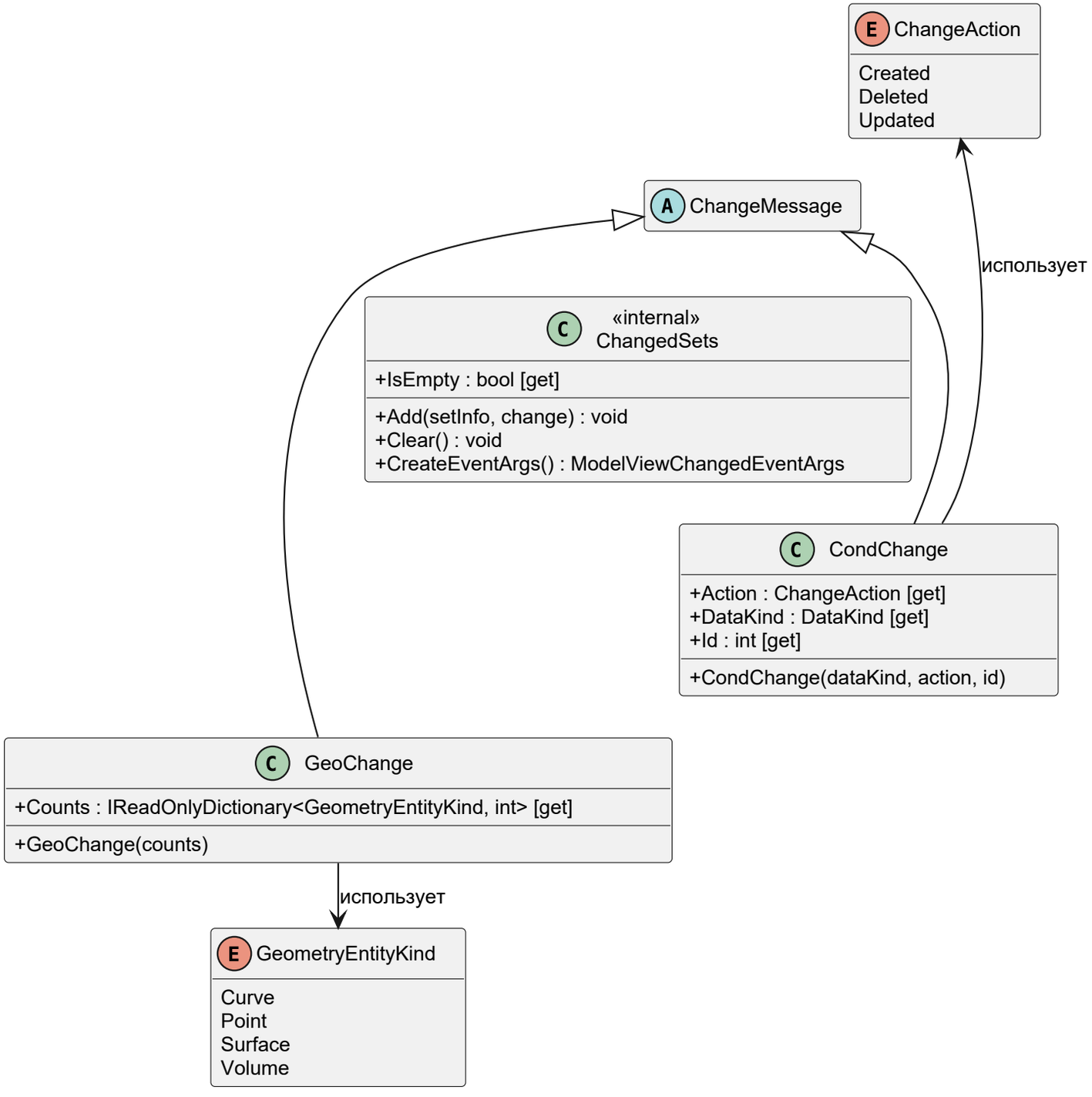
OperationalController-30 - описание

ModelView - описание



Обзор; полный состав членов — на нумерованных поддиаграммах

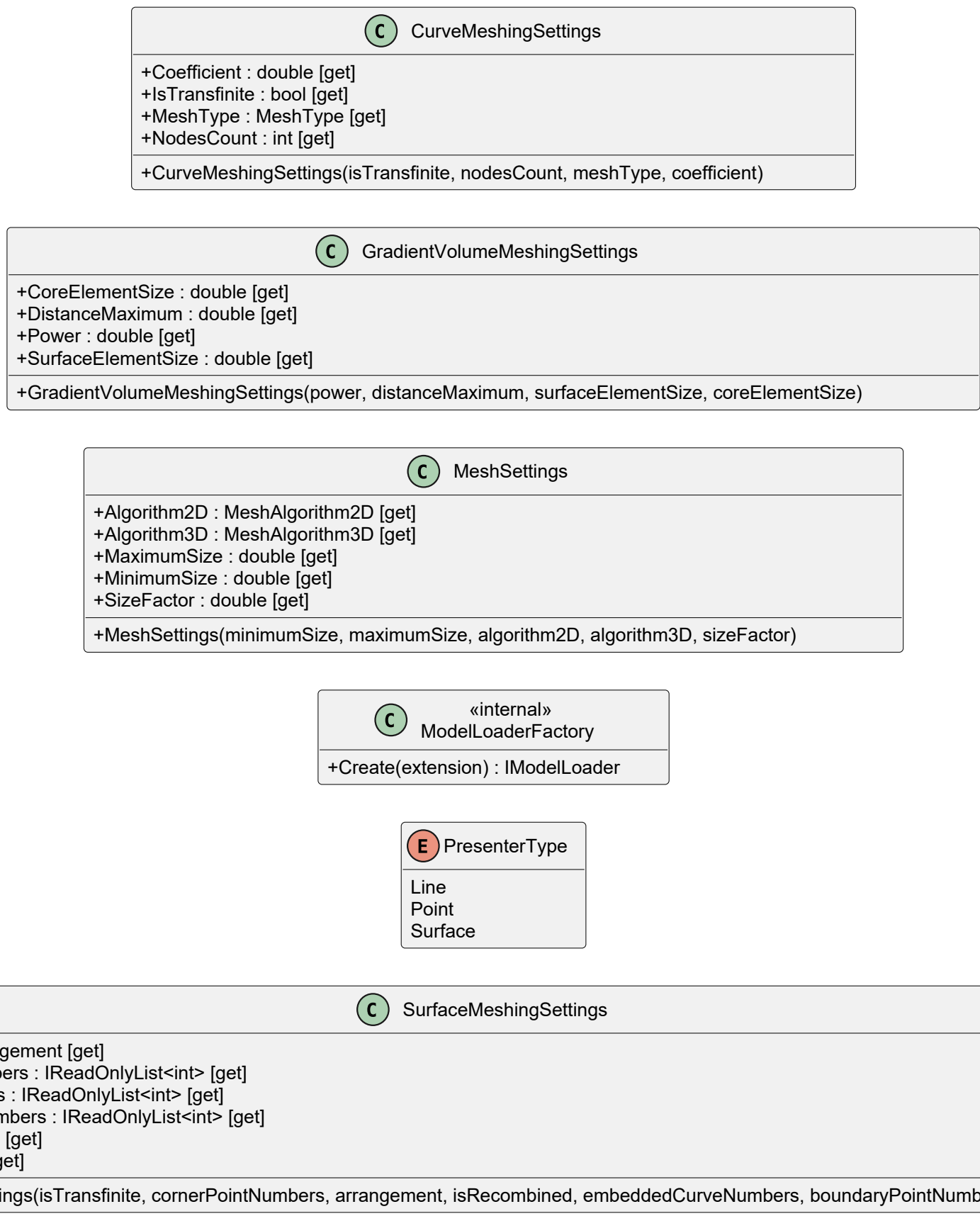
OperationalController-1 — Представление и сообщения об изменениях



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

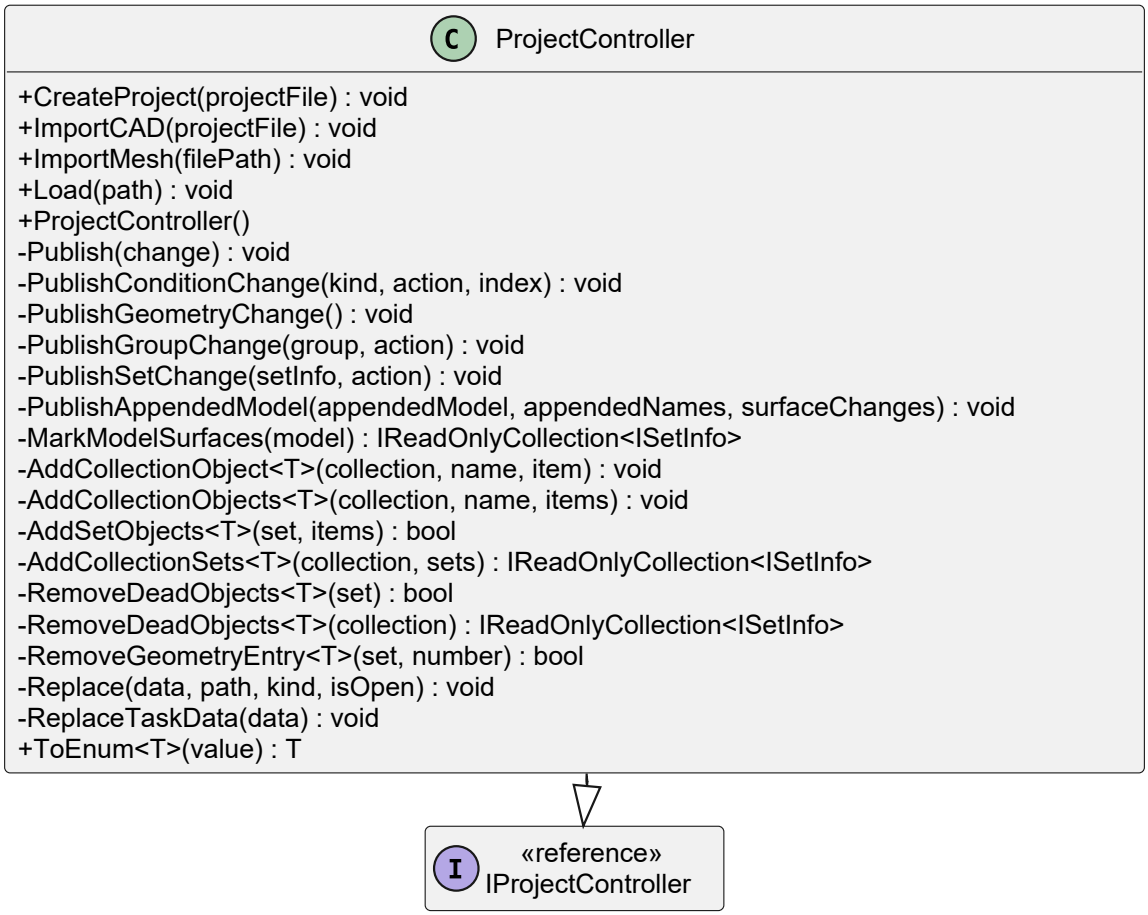
OperationalController-2 — Создание объектов, операции и презентаторы



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

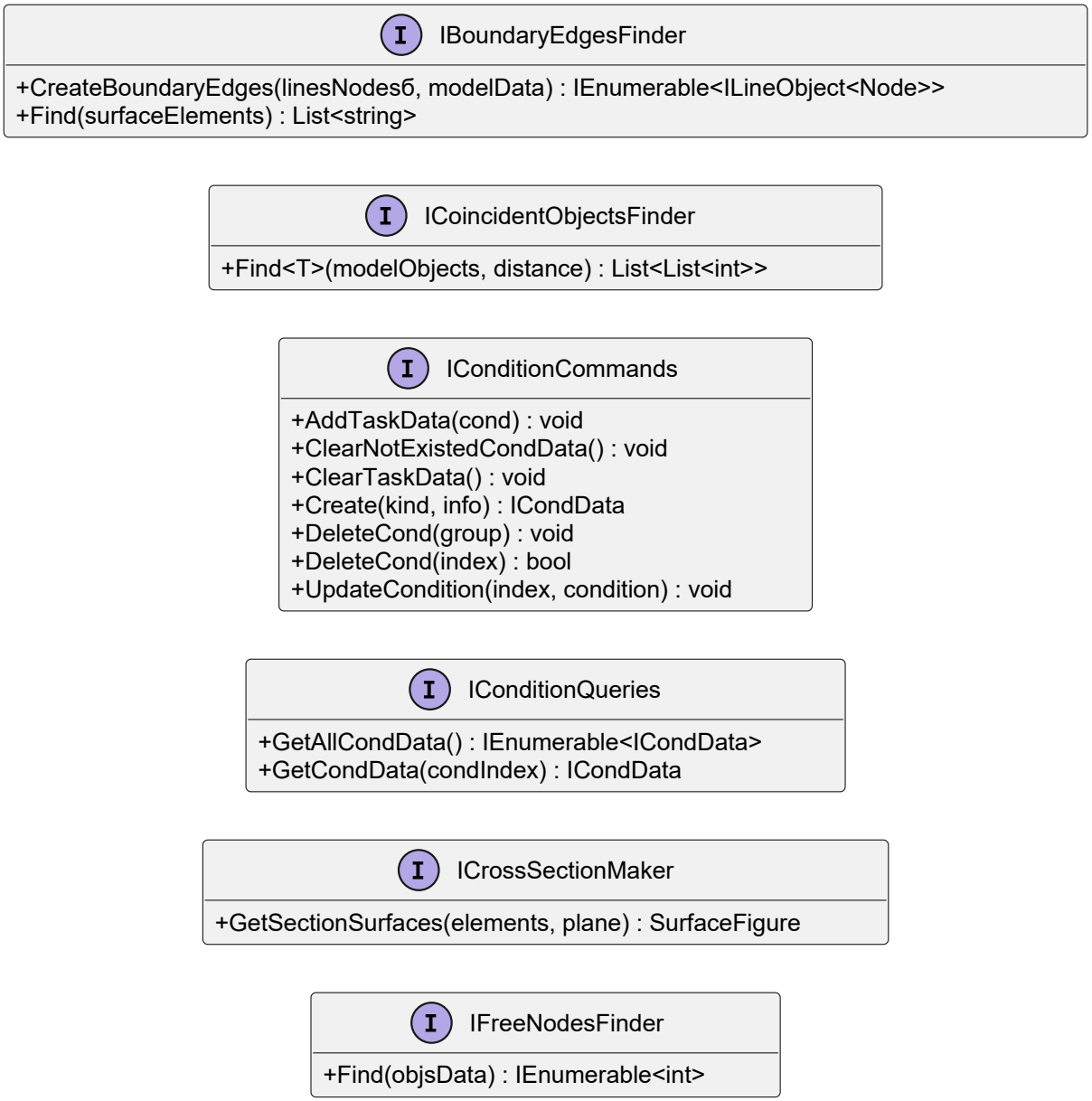
OperationalController-3 — Регистрация изменений в ProjectController



Обзор: OperationalController ; сигнатуры: OperationalController-3.md .
Контракт: IProjectController .
Точки входа в Model: IModelData, IGroup.
Точки входа в Model: ISetInfo, IObjectsSet, IObjectsCollection.
reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

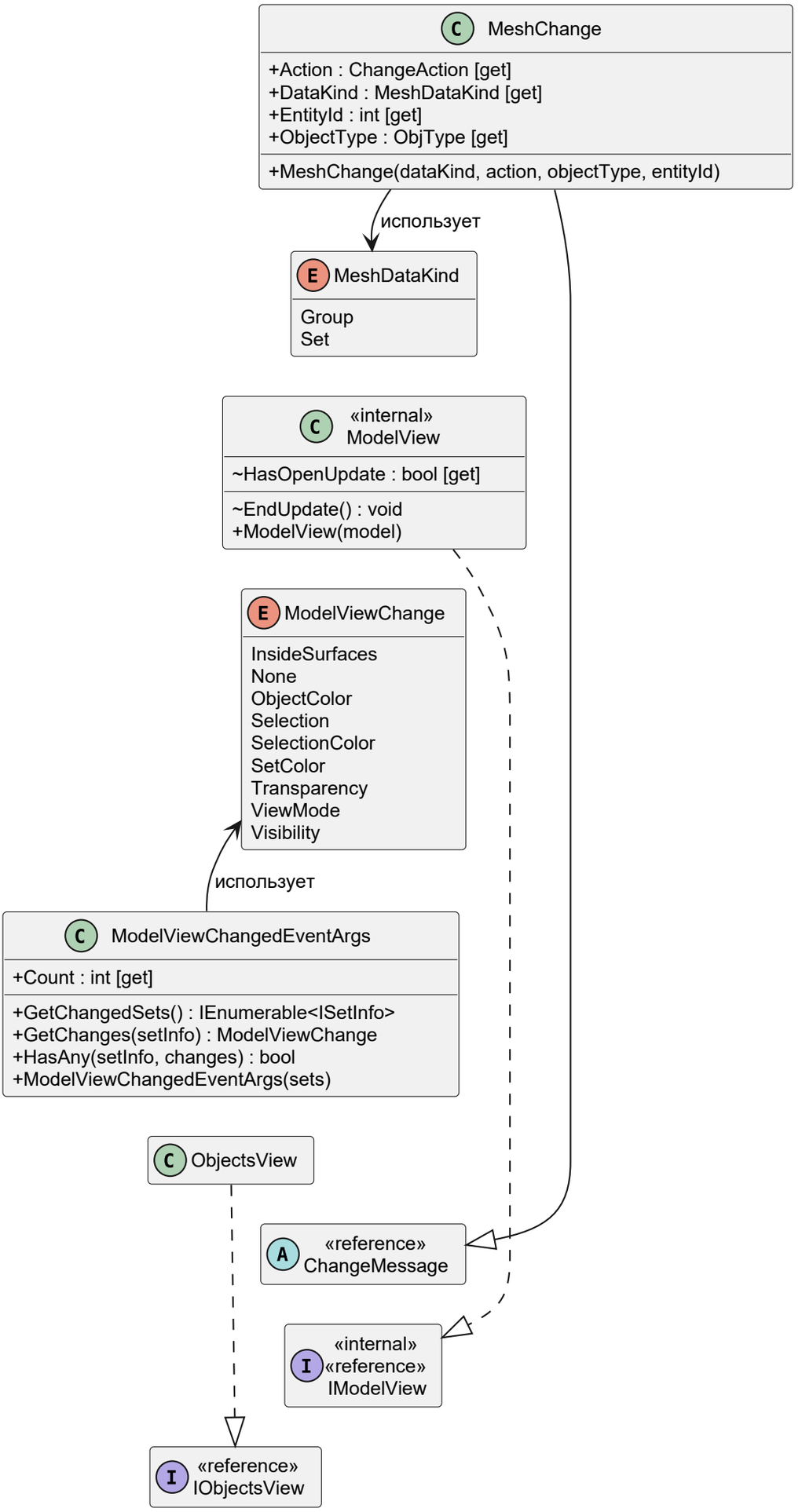
OperationalController-4 — Специализированные контракты контроллера



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

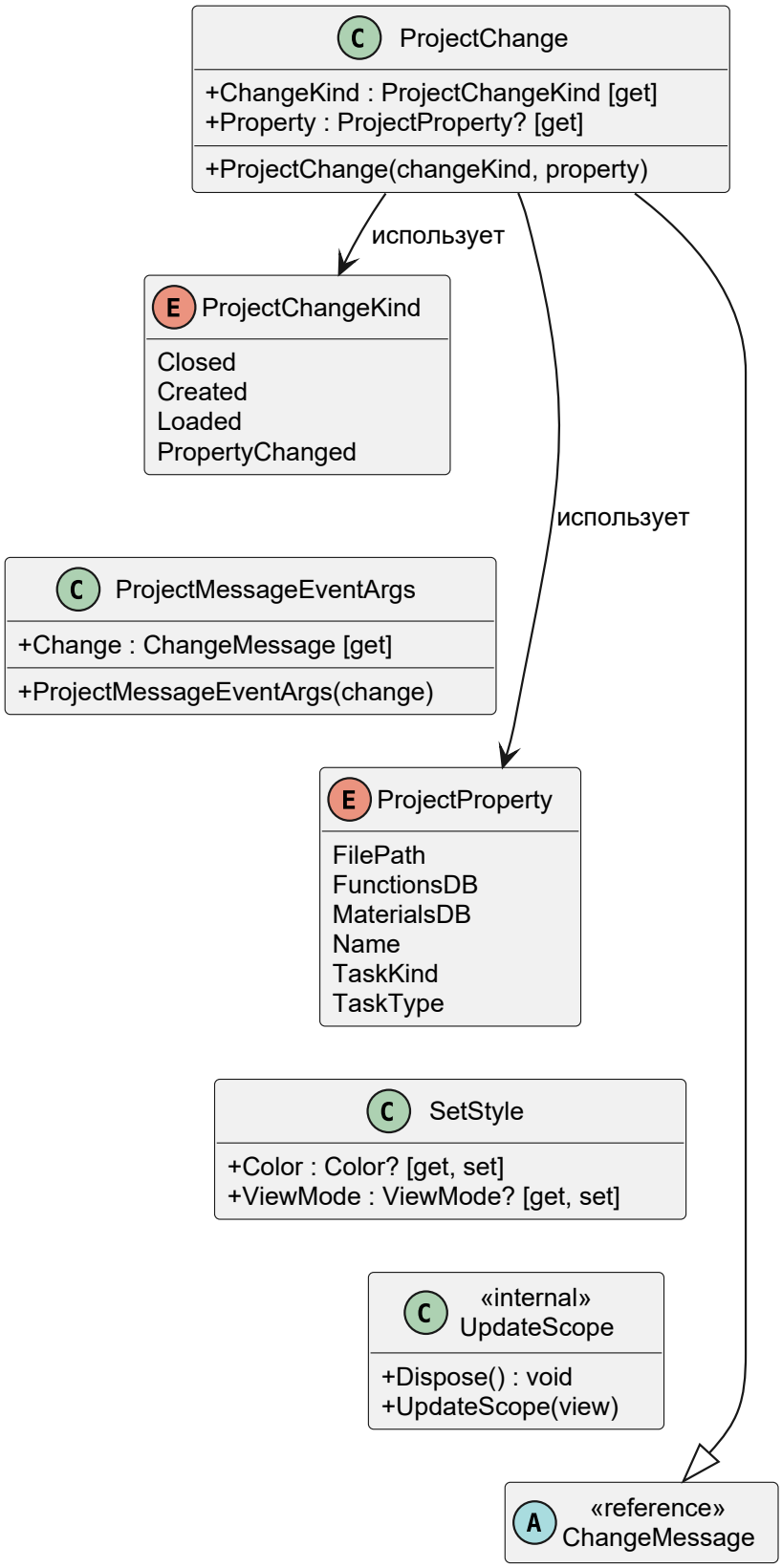
OperationalController-5 — Представление и сообщения об изменениях



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

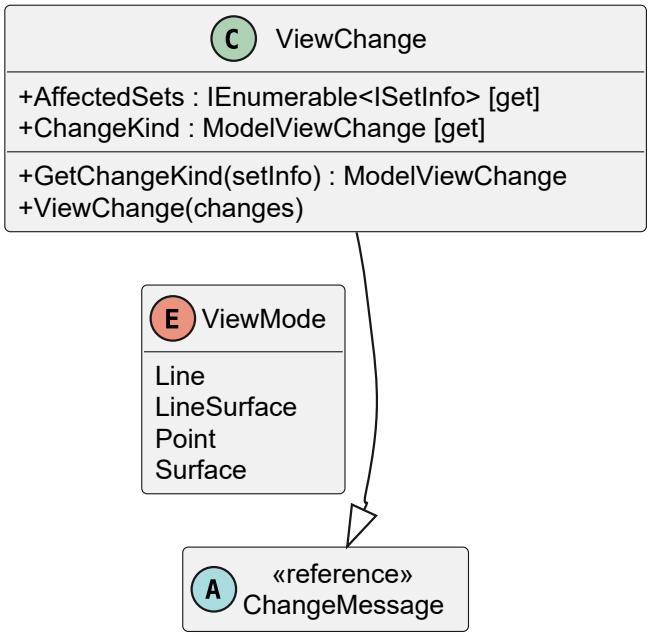
OperationalController-6 — Представление и сообщения об изменениях



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

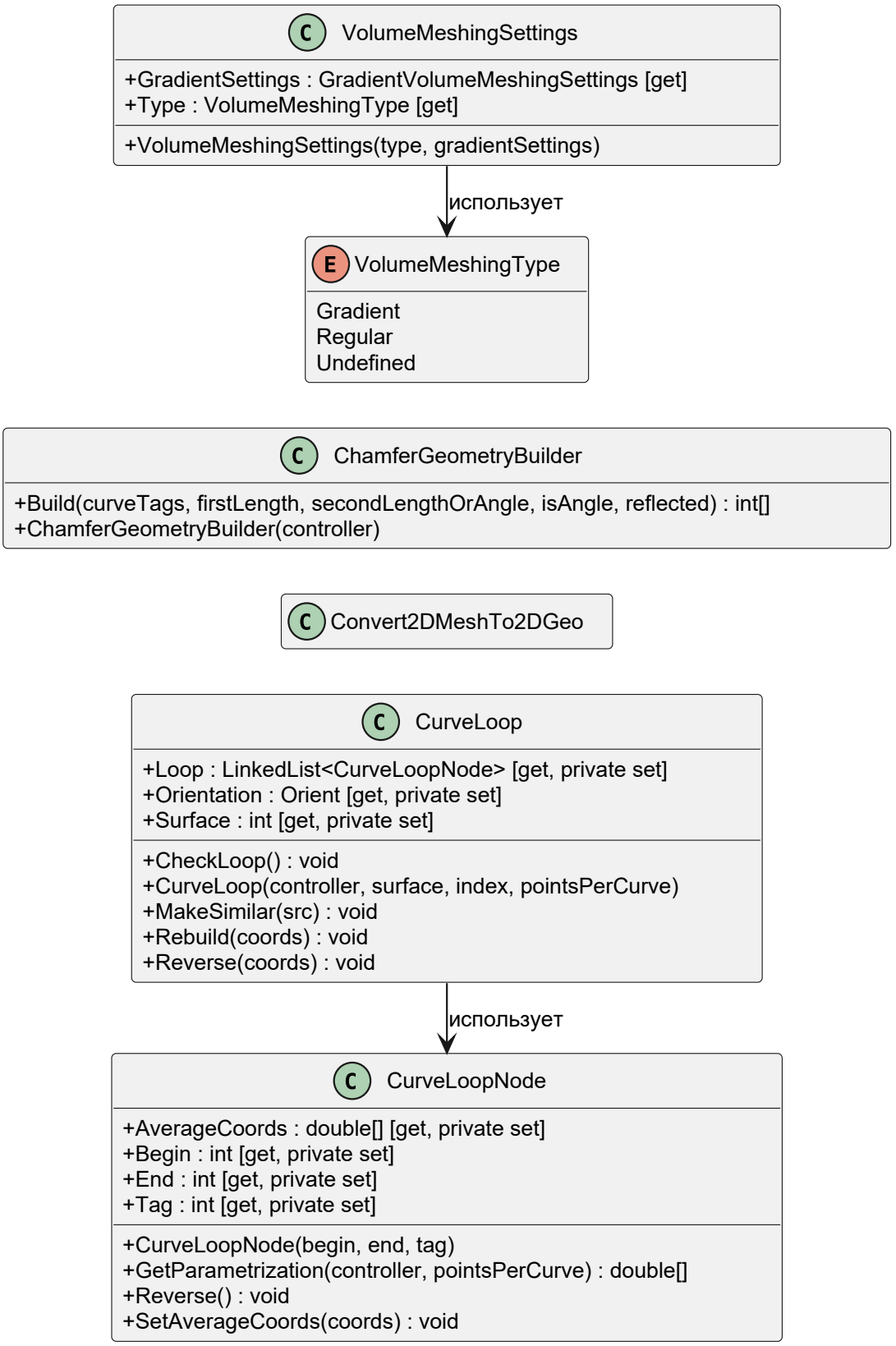
OperationalController-7 — Представление и сообщения об изменениях



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

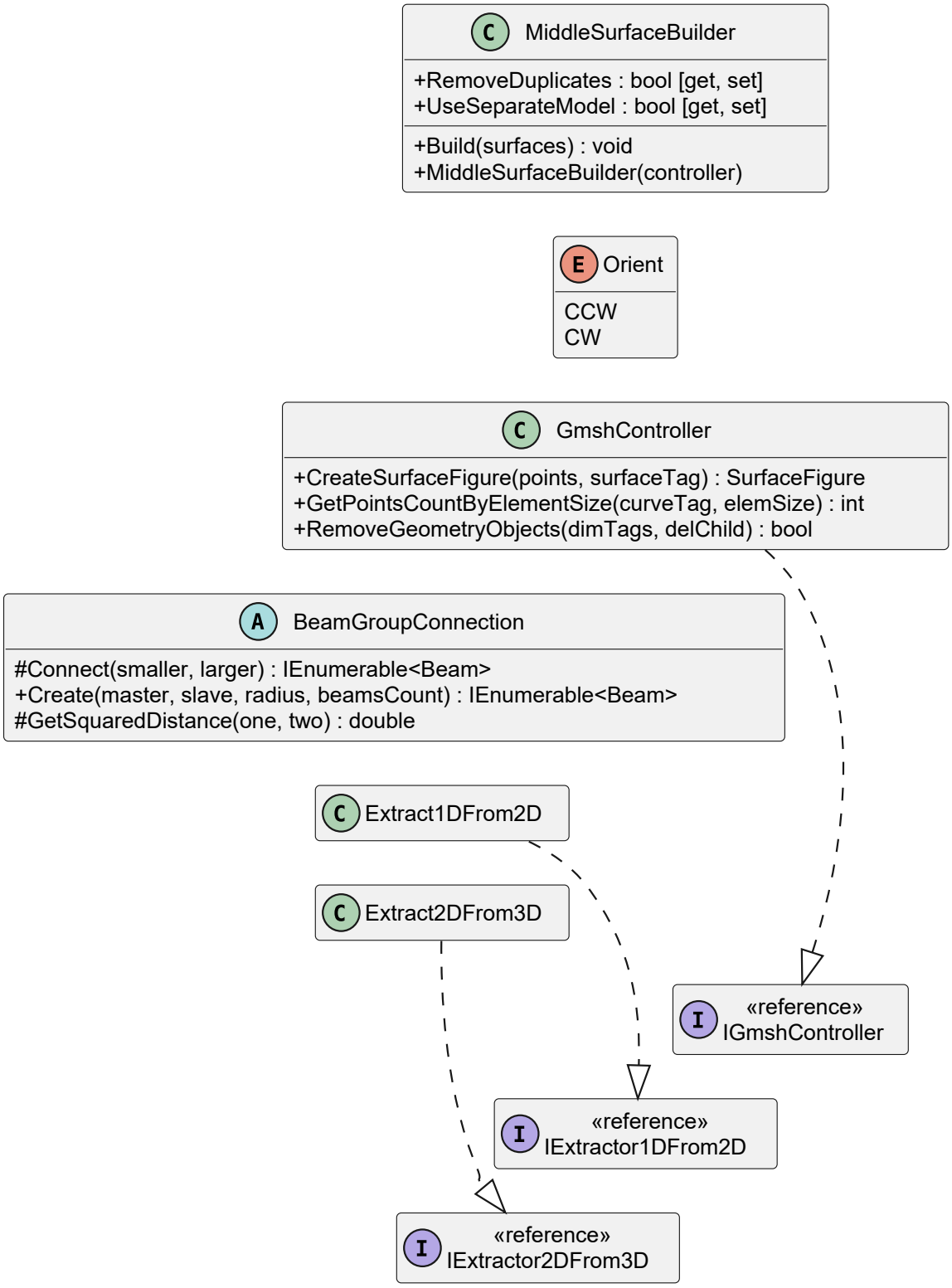
OperationalController-8 — Создание объектов, операции и презентаторы



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

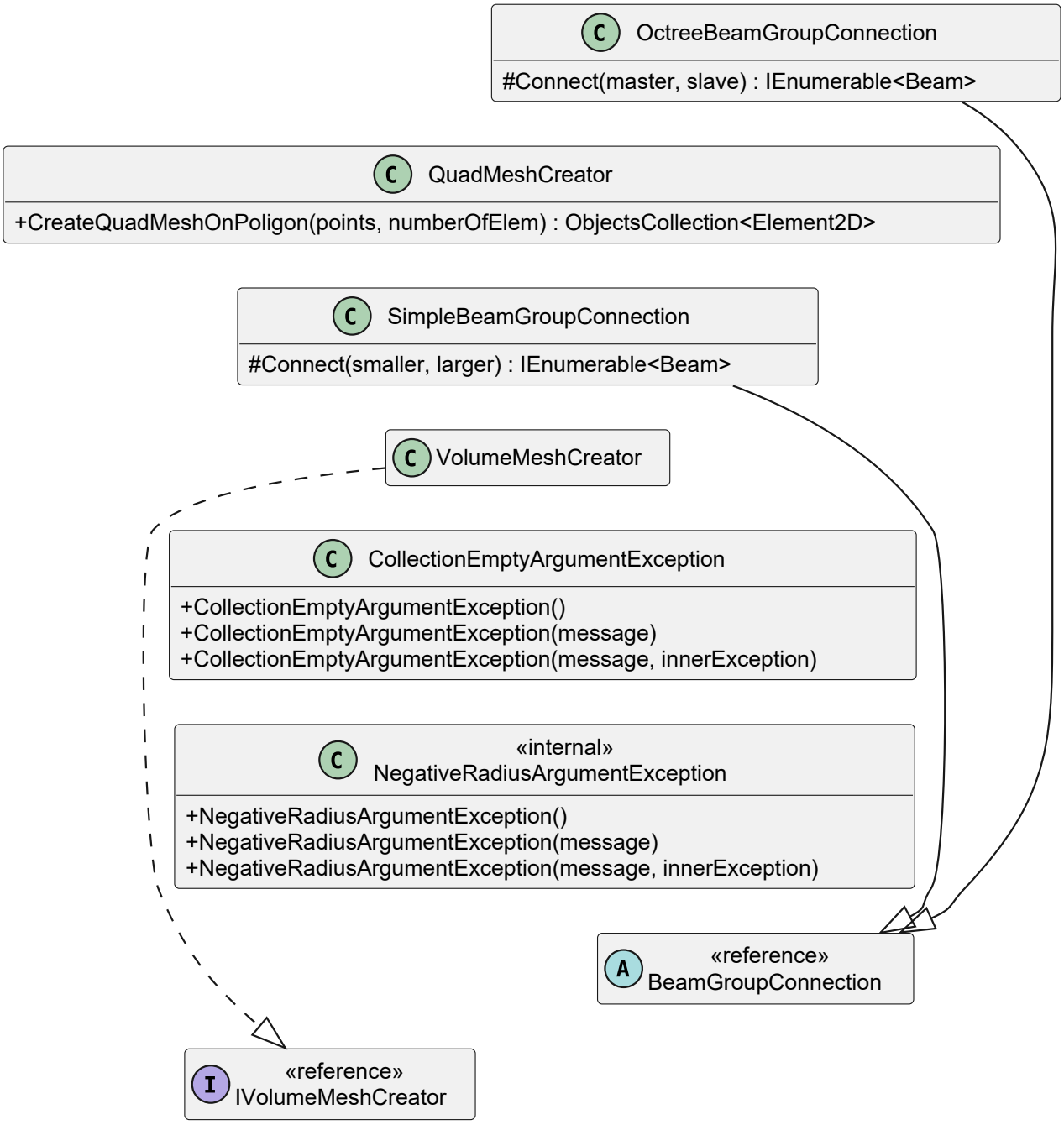
OperationalController-9 — Создание объектов, операции и презентаторы



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

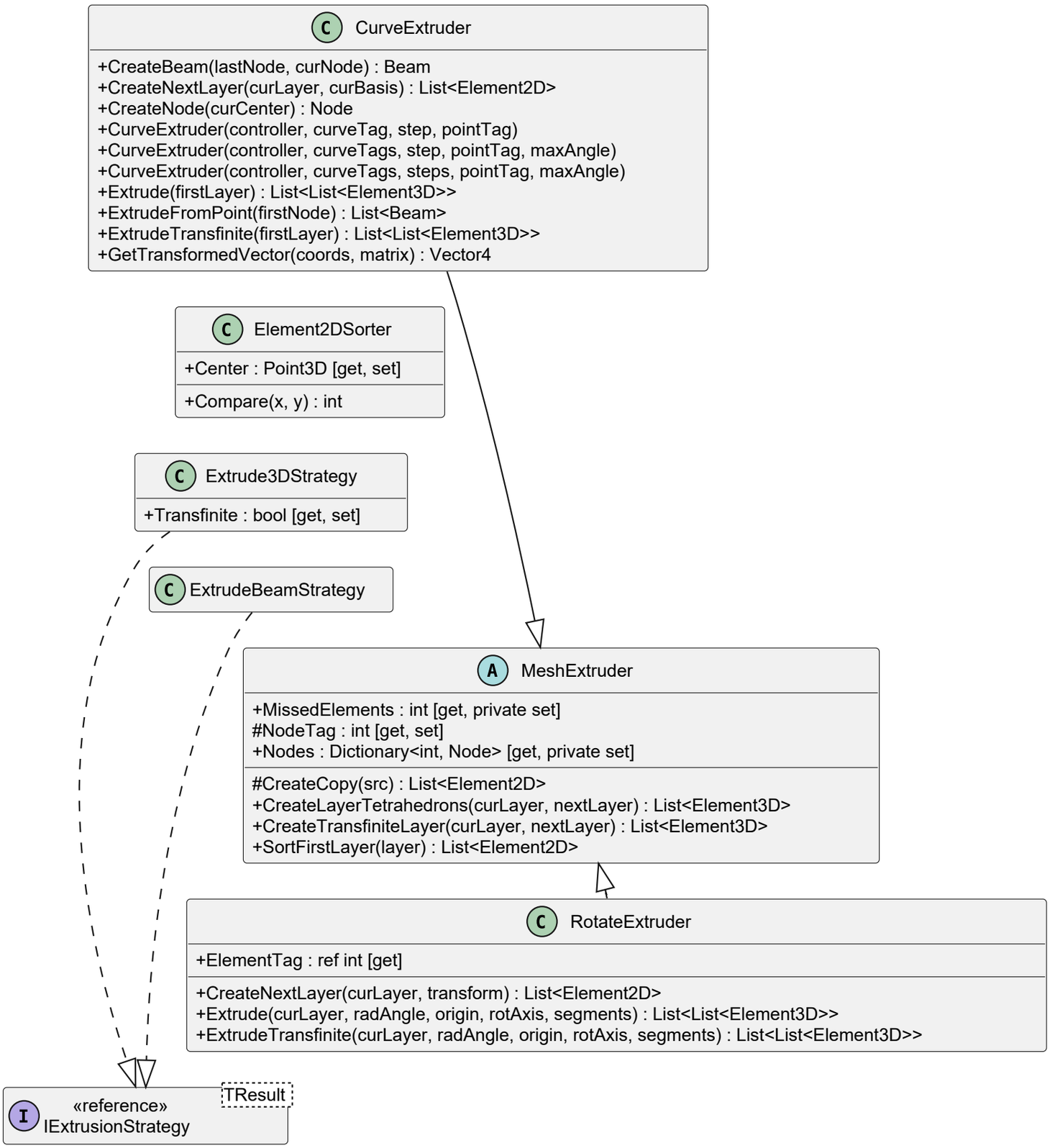
OperationalController-10 — Создание объектов, операции и презентаторы



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

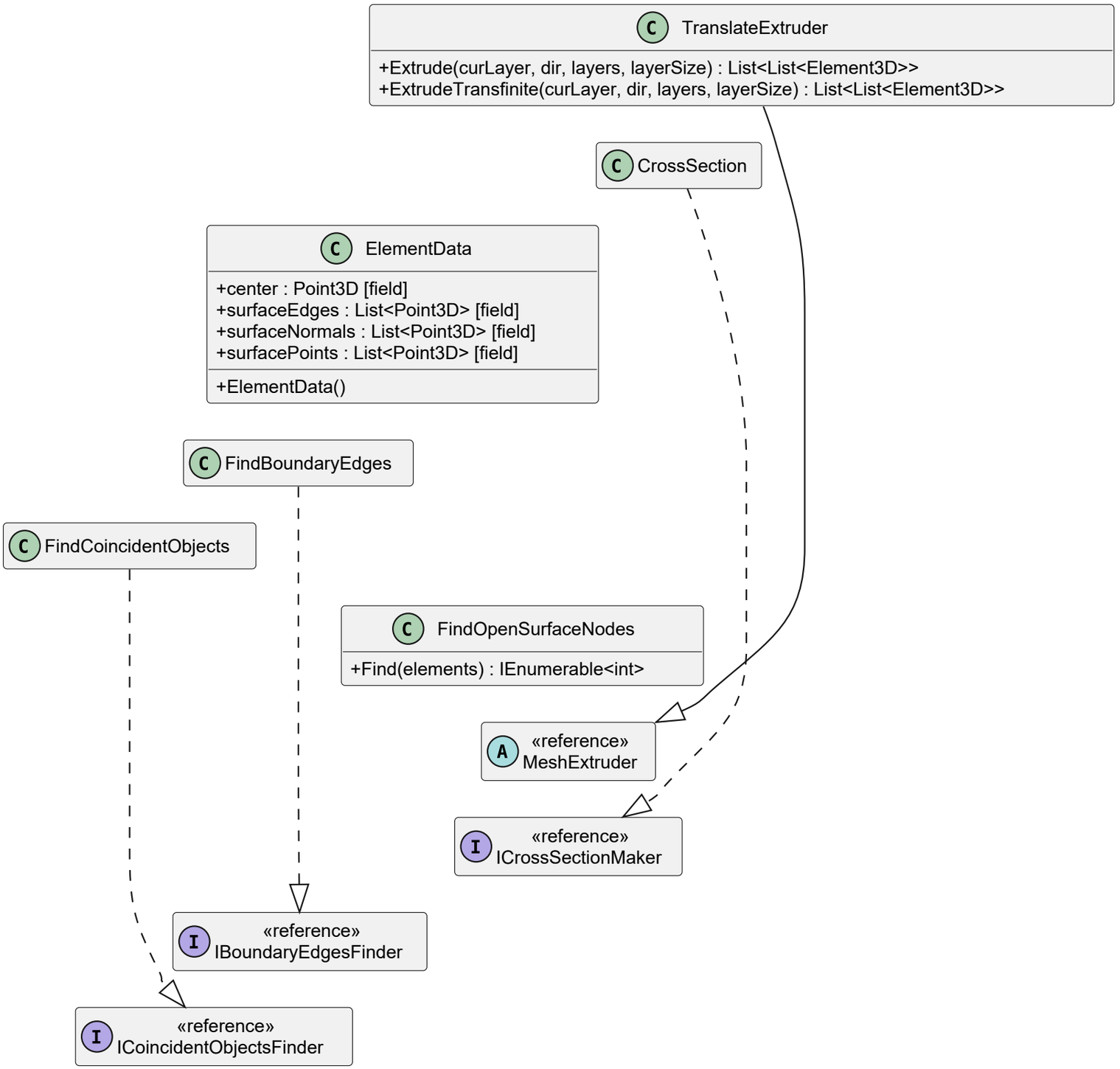
OperationalController-11 — Создание объектов, операции и презентаторы



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

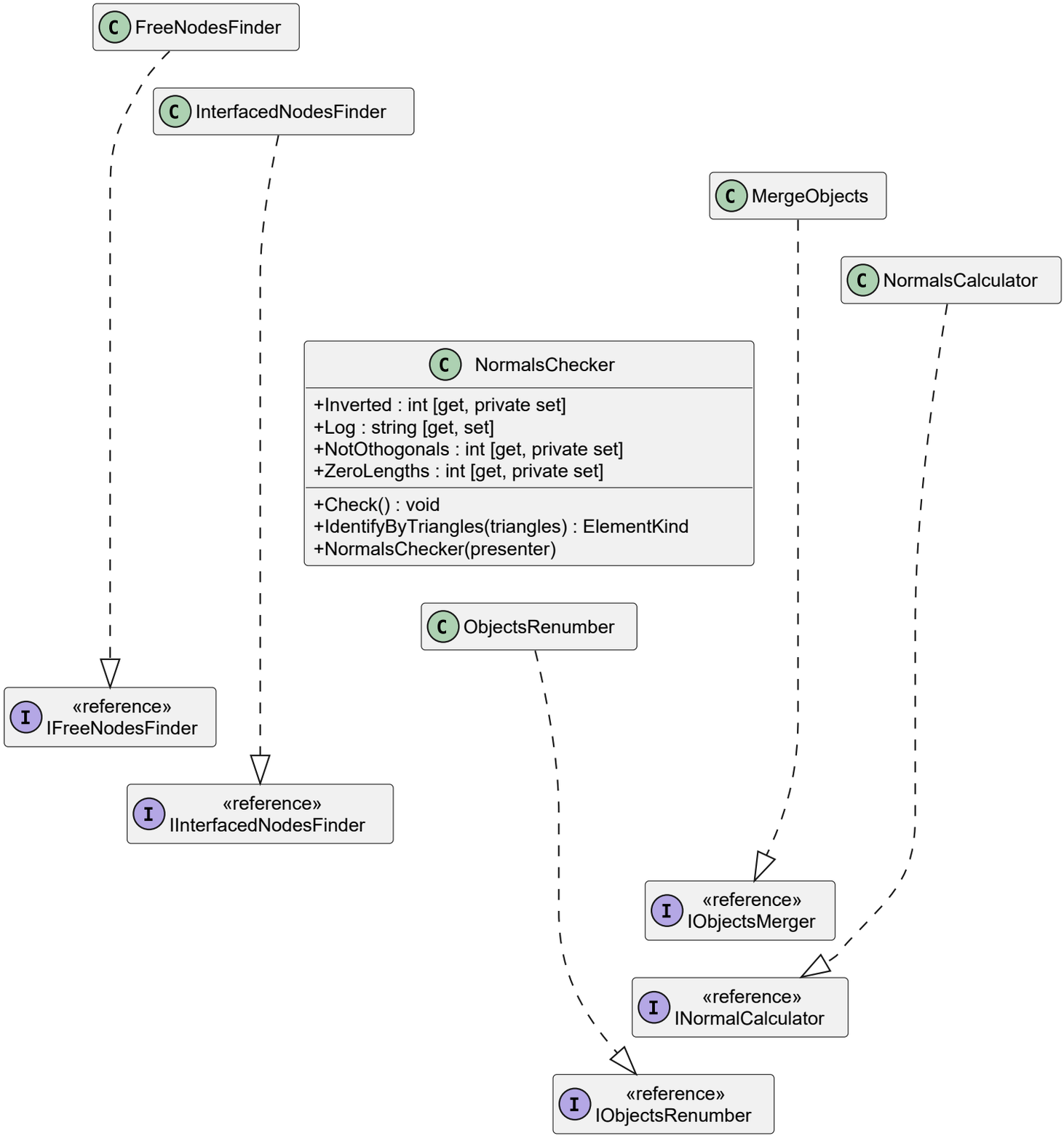
OperationalController-12 — Создание объектов, операции и презентаторы



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

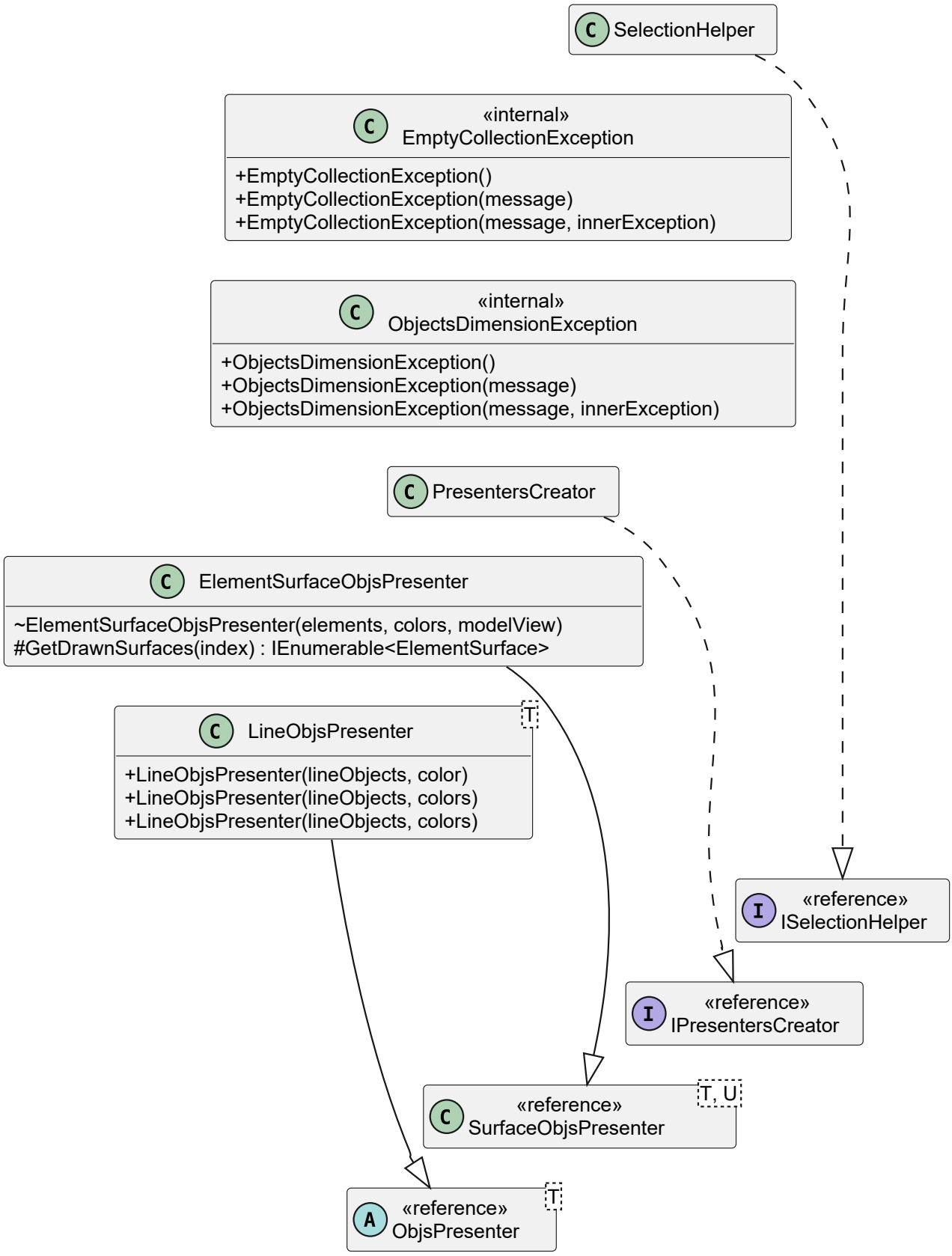
OperationalController-13 — Создание объектов, операции и презентаторы



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

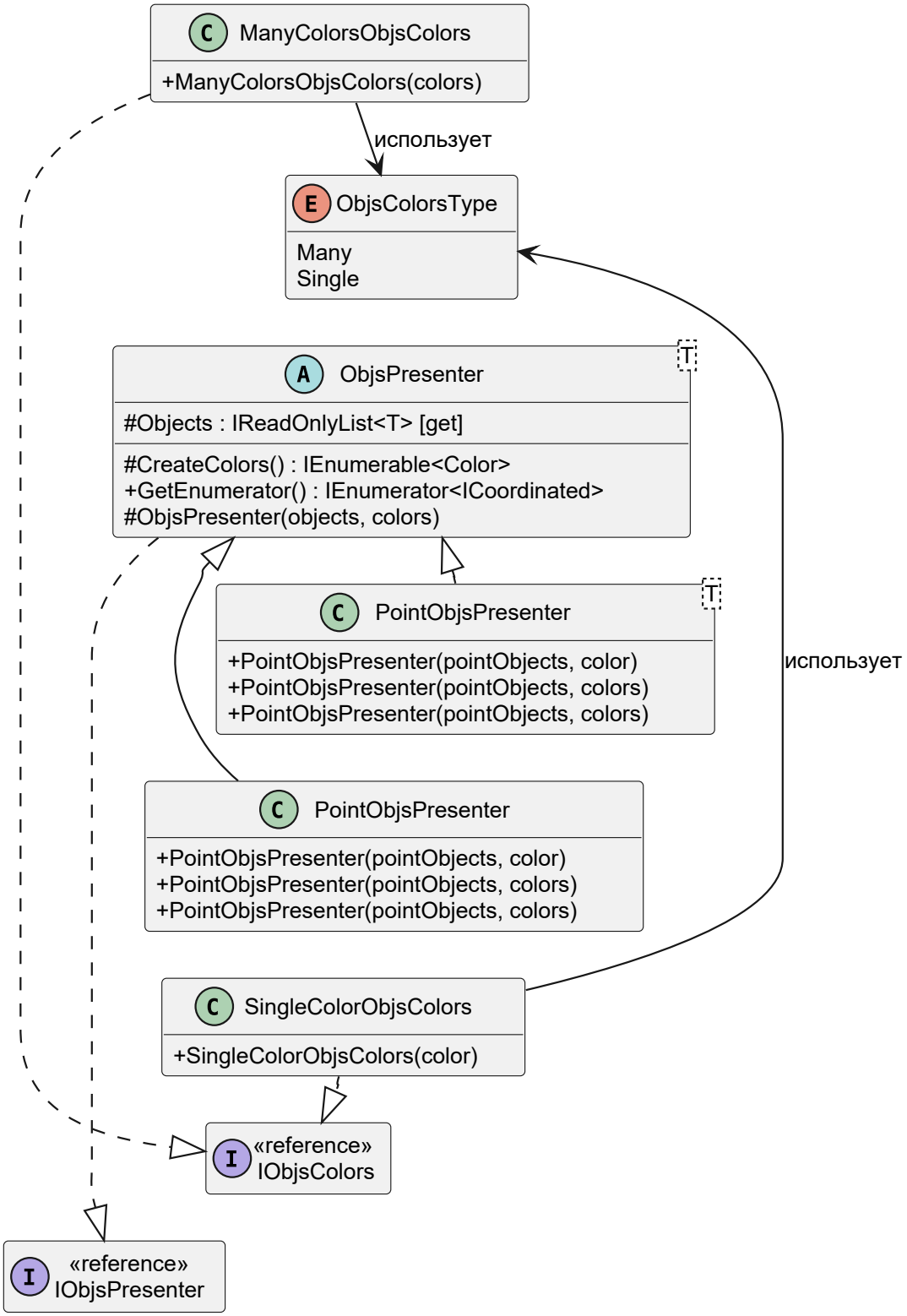
OperationalController-14 — Создание объектов, операции и презентаторы



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

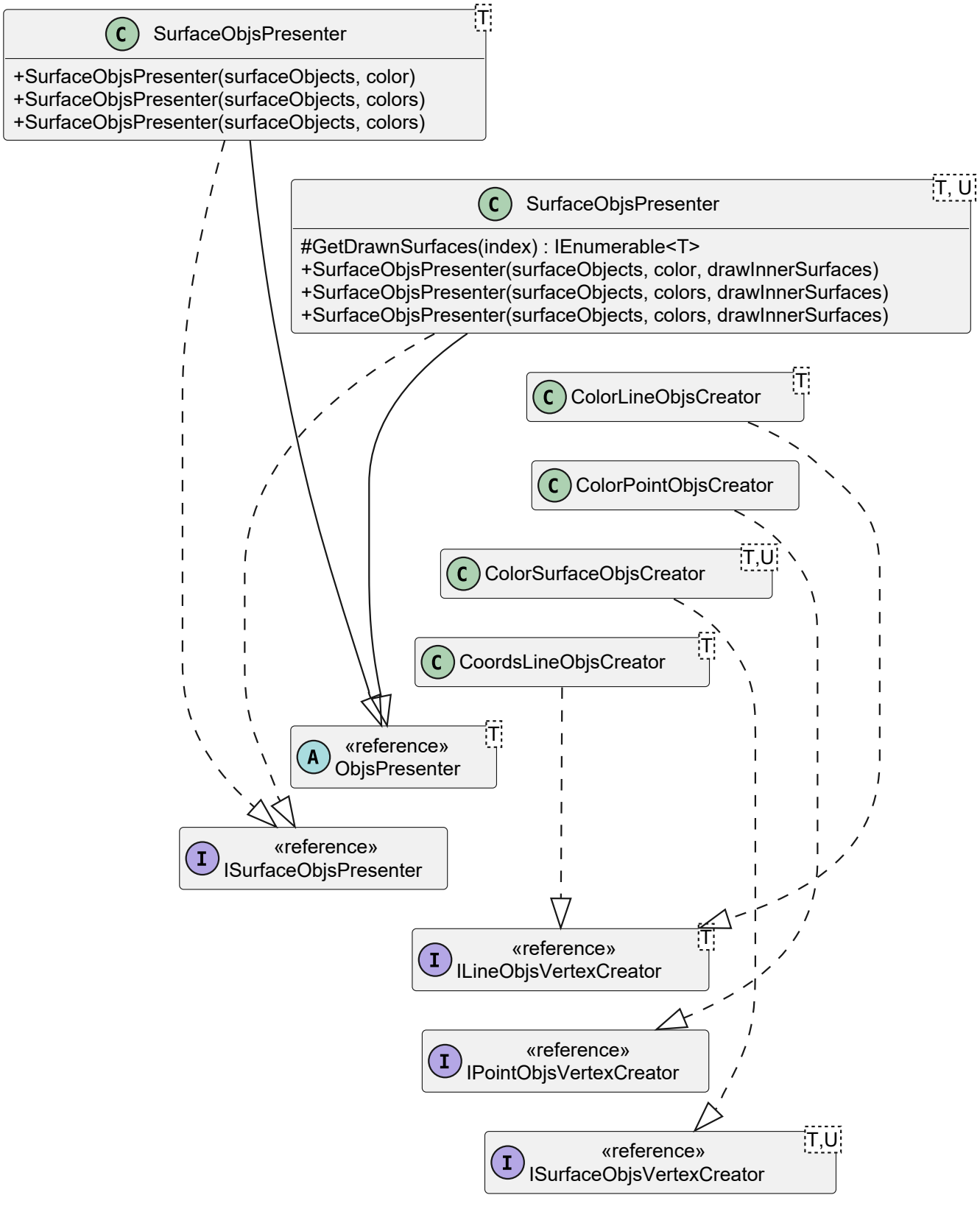
OperationalController-15 — Создание объектов, операции и презентаторы



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

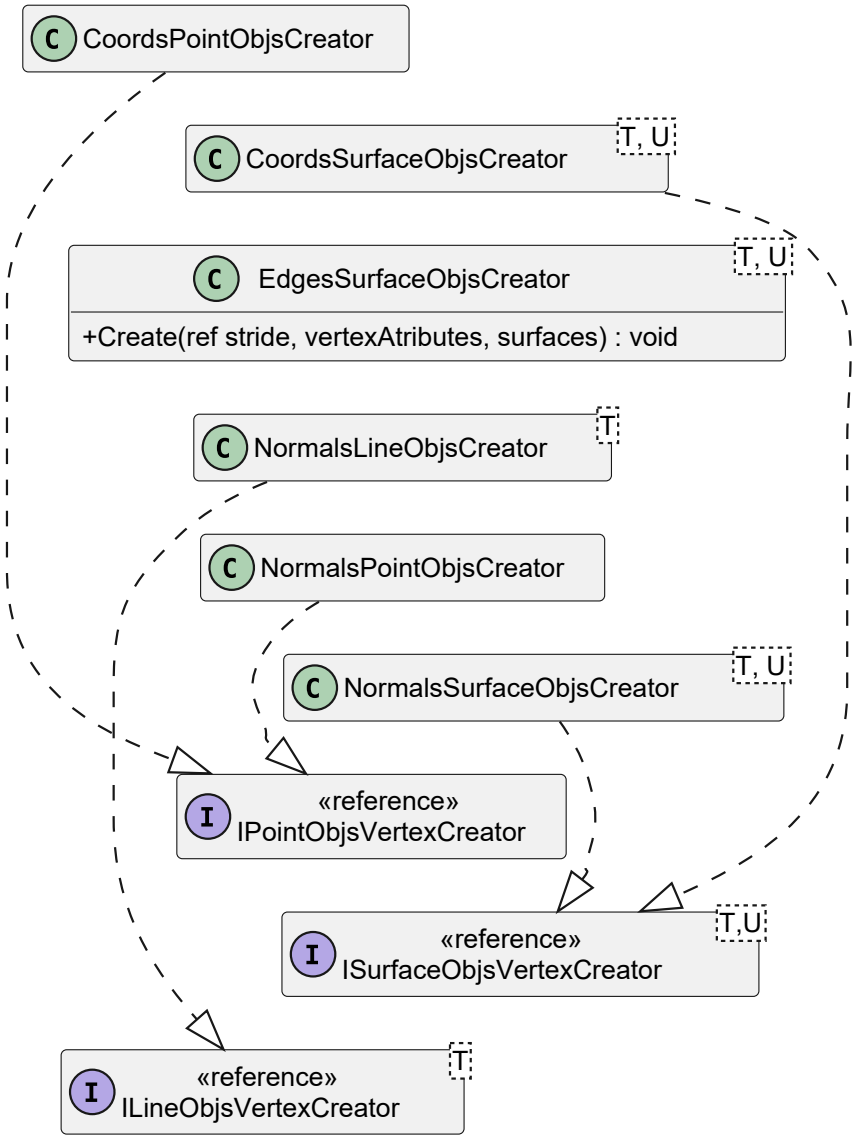
OperationalController-16 — Создание объектов, операции и презентаторы



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

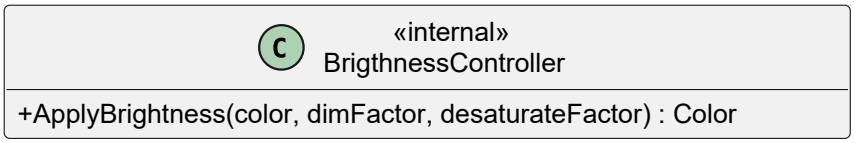
OperationalController-17 — Создание объектов, операции и презентаторы



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

OperationalController-18 — Создание объектов, операции и презентаторы



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

OperationalController-19 — Специализированные контракты контроллера

IGeometryCommands

+CreateChamfer(curveTags, firstLength, secondLengthOrAngle, isAngle, reflected) : int[]
+CreateLine(startPointNumber, endPointNumber) : int
+CreatePoint(x, y, z, meshSize) : int
+CreatePointByVector(basePointTag, directionPointTag, offset) : int
+CreatePointProjectionOntoGeometry(basePointTag, dim, geometryTag) : int
+CreateSurface(lineNumbers) : int
+DeleteGeometryObject(dim, number, recursive) : void
+InitializeGeometry(gmshDllPath) : void
+SetEmbeddedGeometryEntities(hostDimension, hostNumber, embeddedDimension, embeddedNumbers) : void
+UnloadGeometry() : void

IGeometryQueries

+GetAdjacentGeometryObjects(obj, direction) : IEnumerable<IModelObject>
+GetAdjacentGeometryObjects(dim, number) : Tuple<int[], int[]>
+GetBoundary(dimTags, combined, oriented, recursive) : int[]
+GetCenterOfMass(dim, tag) : (double X, double Y, double Z)
+GetCurveLength(curveTag) : double
+GetGeoObjPoints(dim, tag, parametricCoord) : double[]
+GetGeometryFilePath() : string
+GetGeometryLastError() : string
+GetModelCurve(objNumber) : Curve
+GetModelPoint(objNumber) : GeometryPoint
+GetModelSurface(objNumber) : SurfaceFigure
+GetModelVolumes() : IEnumerable<VolumeFigure>
+GetParametrization(dim, tag, coordinates) : double[]
+GetParametrizationBounds(dim, tag) : Tuple<double[], double[]>
+GetSurfaceNormal(surfaceTag, parametricCoordinates) : double[]
+GetType(dim, tag) : string
+SelectByScope(startDim, selectedNumbers, targetDim) : HashSet<int>

IGroupCommands

+ClearGroupData() : void
+ClearNotExistedGroupData() : void
+CreateGroup(objs) : void
+CreateGroup(grName, objs) : void
+CreateGroupByGeoObjs(meshDim, geoDim, tag) : string
+CreateGroupByMeshObjs(dim, setName) : string
+CreateOpenSurfaceNodesGroup(setName) : string
+DeleteModelGroup(grName) : void

IGroupQueries

+ContainsConditions(grName) : bool
+GetAllModelGroups() : IEnumerable<IGroup>
+GetModelGroup(grIndex) : IGroup
+GetModelGroup(grName) : IGroup

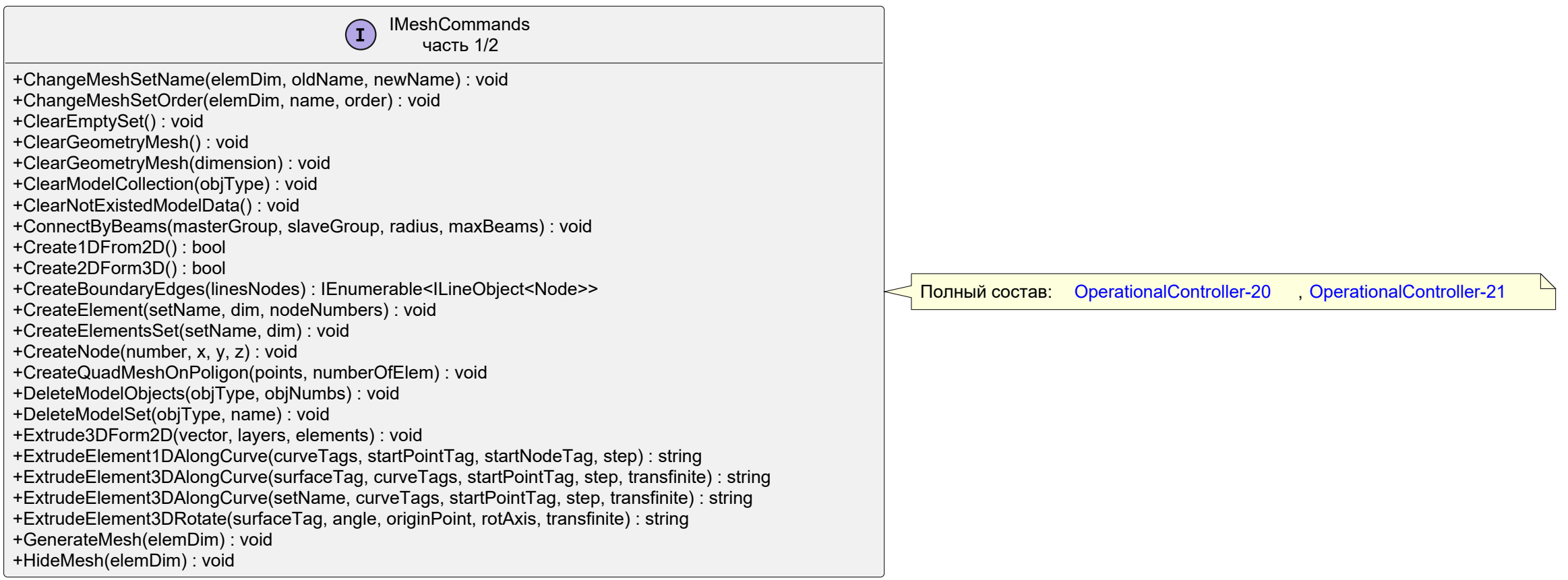
IInterfacedNodesFinder

+Find(elements) : Dictionary<int, List<int>>
+Find(nodes, elements) : Dictionary<int, List<int>>

reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

OperationalController-20 — Специализированные контракты контроллера



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

OperationalController-21 — Специализированные контракты контроллера

IMeshCommands
часть 2/2

+MergeElements(objType, setMaster, setSlave) : void
+MergeNodes(numbers) : void
+MoveMesh(objType, vector) : void
+RecombineMesh() : void
+RefineMesh() : void
+RrenumberElements() : void
+RotateMesh(objType, axis, angle) : void
+SetCurveMeshingSettings(curveNumber, settings) : void
+SetMeshAlgorithm2D(algorithm) : void
+SetMeshAlgorithm3D(algorithm) : void
+SetMeshMaximumSize(size) : void
+SetMeshMinimumSize(size) : void
+SetMeshPointSize(pointNumber, size) : void
+SetMeshSizeFactor(factor) : void
+SetSurfaceMeshingSettings(surfaceNumber, settings) : void
+SetVolumeMeshingSettings(volumeNumber, settings) : void

Полный состав: OperationalController-20 , OperationalController-21

IMeshQueries
часть 1/2

+CalcElemsNormals(elemDim) : IEnumerable<Curve>
+CalcNodeNormals() : Dictionary<int, Point3D>
+ContainsModelObject(objType, objNumber) : bool
+CreateModelObjectsPresentor(setInfo) : IObjsPresenter
+FindBoundaryEdges() : List<string>
+FindCoincidentObjects(objType, distance) : List<List<int>>
+FindFreeNodes() : IEnumerable<int>
+FindInterfacedNodes(nodes, elemDim) : Dictionary<int, List<int>>
+FindInterfacedNodes(elemDim) : Dictionary<int, List<int>>
+GetAllModelElements() : IEnumerable<IElement>
+GetAllModelNodes() : IEnumerable<Node>
+GetAllModelObjects() : IEnumerable<IModelObject>
+GetAllModelSetsInfo() : IEnumerable<ISetInfo>
+GetCurveMeshingSettings(curveNumber) : CurveMeshingSettings
+GetGeometryMeshNodes(dimension, entityNumbers, includeBoundary) : IEnumerable<Node>
+GetMeshPointSize(pointNumber) : double
+GetMeshSettings() : MeshSettings
+GetModelElements(dim, setName) : IEnumerable<IElement>
+GetModelObject(objType, objNumber) : IModelObject
+GetModelObjects(objType) : IEnumerable<IModelObject>
+GetModelSetInfo(objType, objNumber) : ISetInfo
+GetModelSetInfo(objType, setName) : ISetInfo
+GetModelSetsInfo(objType) : IEnumerable<ISetInfo>
+GetModelSurfaceElements(dim) : IEnumerable<ISurfaceElement>

Полный состав: OperationalController-21 , OperationalController-22

reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

OperationalController-22 — Специализированные контракты контроллера

IMeshQueries
часть 2/2

+GetSectionSurfaces(plane) : SurfaceFigure
+GetSurfaceMeshingSettings(surfaceNumber) : SurfaceMeshingSettings
+GetVolumeMeshingSettings(volumeNumber) : VolumeMeshingSettings
+SelectE2DInPlane(angleInGradus, startNumber, color) : List<int>
+SelectNodeInDirection(angleInGradus, numbFirst, numbSecond, color) : List<int>
+SelectNodeInPlane(plane, color) : List<int>

Полный состав: OperationalController-21 , OperationalController-22

«internal»
IModelView
часть 1/2

+HideInsideSurfaces : bool [get, set]
+Model : IModelData [get]
+SelectedCount : int [get]
+SelectedTypes : IEnumerable<ObjType> [get]
+SelectionColor : Color [get, set]
+Transparency : byte [get, set]

+BeginUpdate() : IDisposable
+ClearColor() : void
+ClearColor(objType, numbers) : void
+ClearSelection() : void
+ClearSelection(objType) : void
+Deselect(objType, numbers) : void
+GetColor(setInfo) : Color
+GetColor(setInfo, number) : Color
+GetColor(objType, number) : Color
+GetSelected(objType) : IEnumerable<int>
+GetSelection() : IEnumerable<IModelObject>
+GetViewMode(setInfo) : ViewMode
+GetVisible(objType, number) : bool
+HideSelected() : void
+IsSelected(objType, number) : bool
+Prune() : void
+Reset() : void
+Select(objType, numbers) : void

Полный состав: OperationalController-22 , OperationalController-23

reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

OperationalController-23 — Специализированные контракты контроллера

«internal»
IModelView
часть 2/2

+SetColor(setInfo, color) : void
+SetColor(objType, numbers, color) : void
+SetSelection(objType, numbers) : void
+SetViewMode(setInfo, viewMode) : void
+SetVisible(objType, numbers, visible) : void
+ShowAll() : void
+ToggleSelection(objType, numbers) : void
+TryGetSingleSelectedType(out objType) : bool

Полный состав: OperationalController-22 , OperationalController-23

IObjectsMerger

+Merge(numbers, modelObjects) : void
+Merge(numbers, modelObjects, out changedElements, out nodesRemoved) : void

IObjectsRenumber

+Renumber(objectsData, objType) : void

IObjectsView

+HiddenCount : int [get]
+SelectedCount : int [get]

+ClearObjectColor(numbers) : bool
+ClearObjectColors() : bool
+ClearSelection() : bool
+ClearSelectionAndHidden() : bool
+Deselect(numbers) : bool
+GetSelected() : IEnumerable<int>
+IsHidden(number) : bool
+IsSelected(number) : bool
+Prune(exists) : bool
+PruneSetStyles(existingSets) : bool
+Select(numbers) : bool
+SetHidden(numbers, hidden) : bool
+SetObjectColor(numbers, color) : bool
+SetSelection(numbers) : bool
+SetSetColor(setInfo, color) : bool
+SetViewMode(setInfo, viewMode) : bool
+ShowAll() : bool
+ToggleSelection(numbers) : bool
+TryGetObjectColor(number, out color) : bool
+TryGetSetColor(setInfo, out color) : bool
+TryGetViewMode(setInfo, out viewMode) : bool

IObjsPresenter

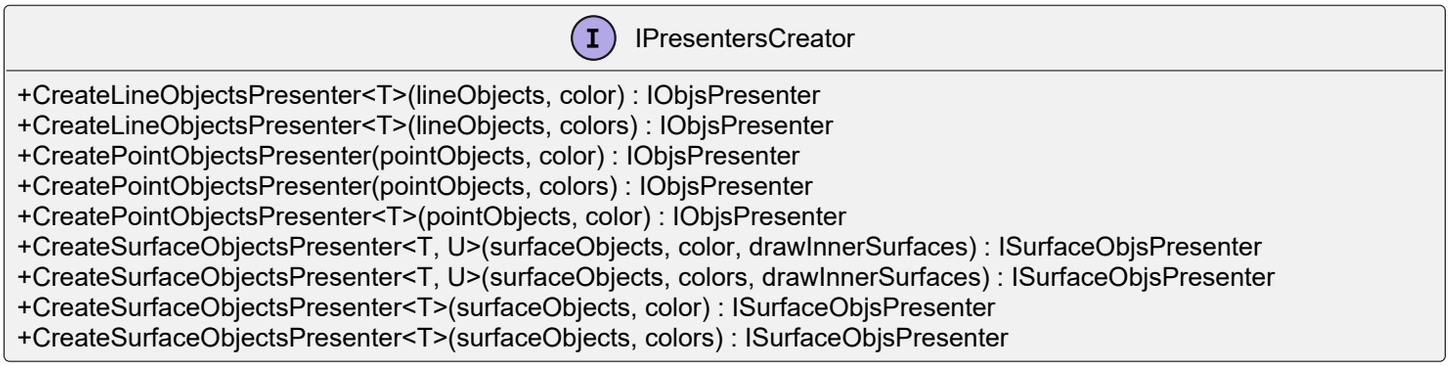
+Name : string [get, set]
+PresenterType : PresenterType [get]
+ViewMode : ViewMode [get, set]

+Count() : int
+CreateEdgeFlags(edgesLength) : bool[]
+CreateIndexes() : Tuple<int, int, int, int>
+CreatePointers(indexLength) : int[]
+CreateVertexes(vertexLength, vertexType) : float[]

reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

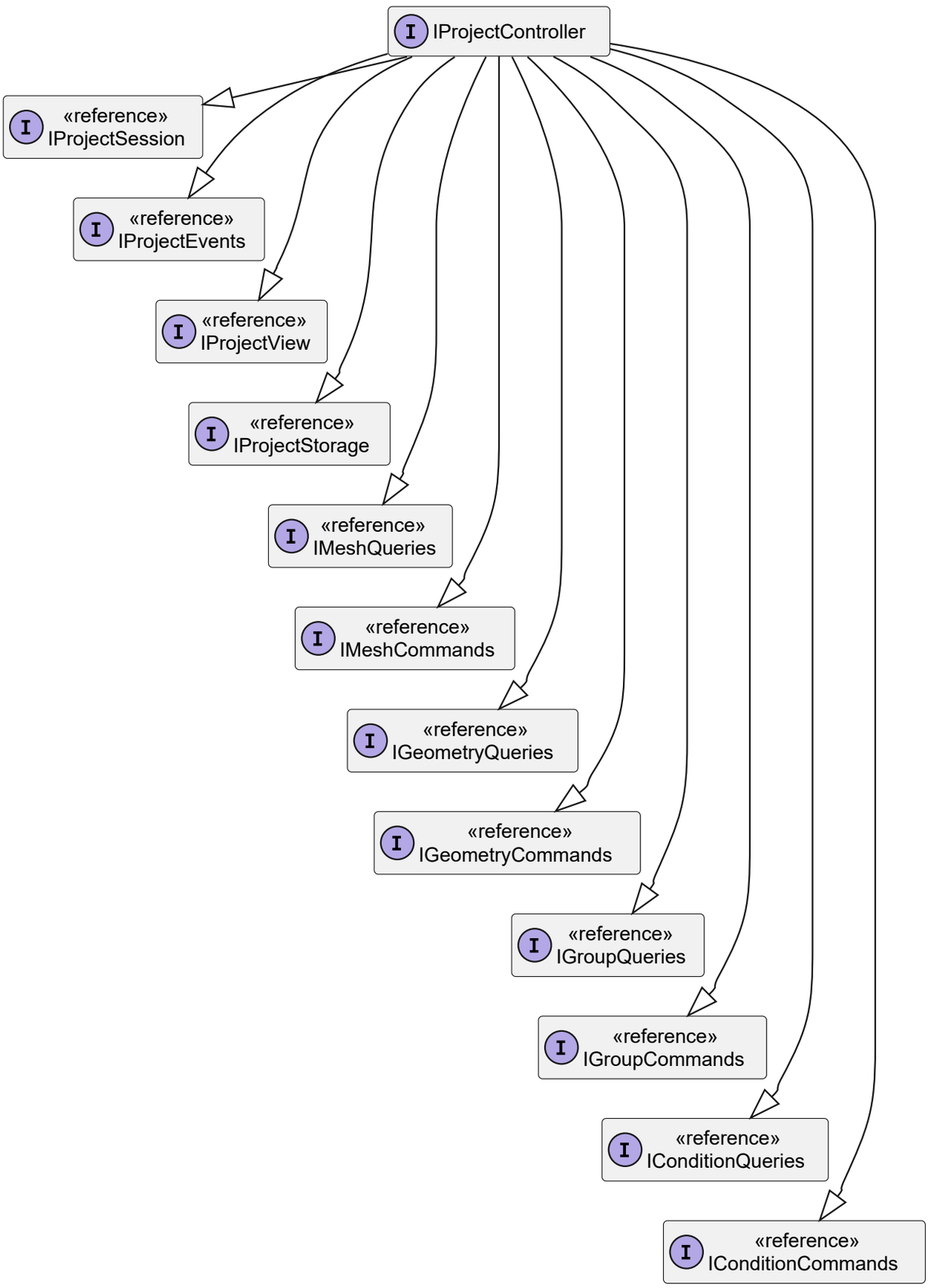
OperationalController-24 — Специализированные контракты контроллера



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

OperationalController-25 — Специализированные контракты контроллера



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

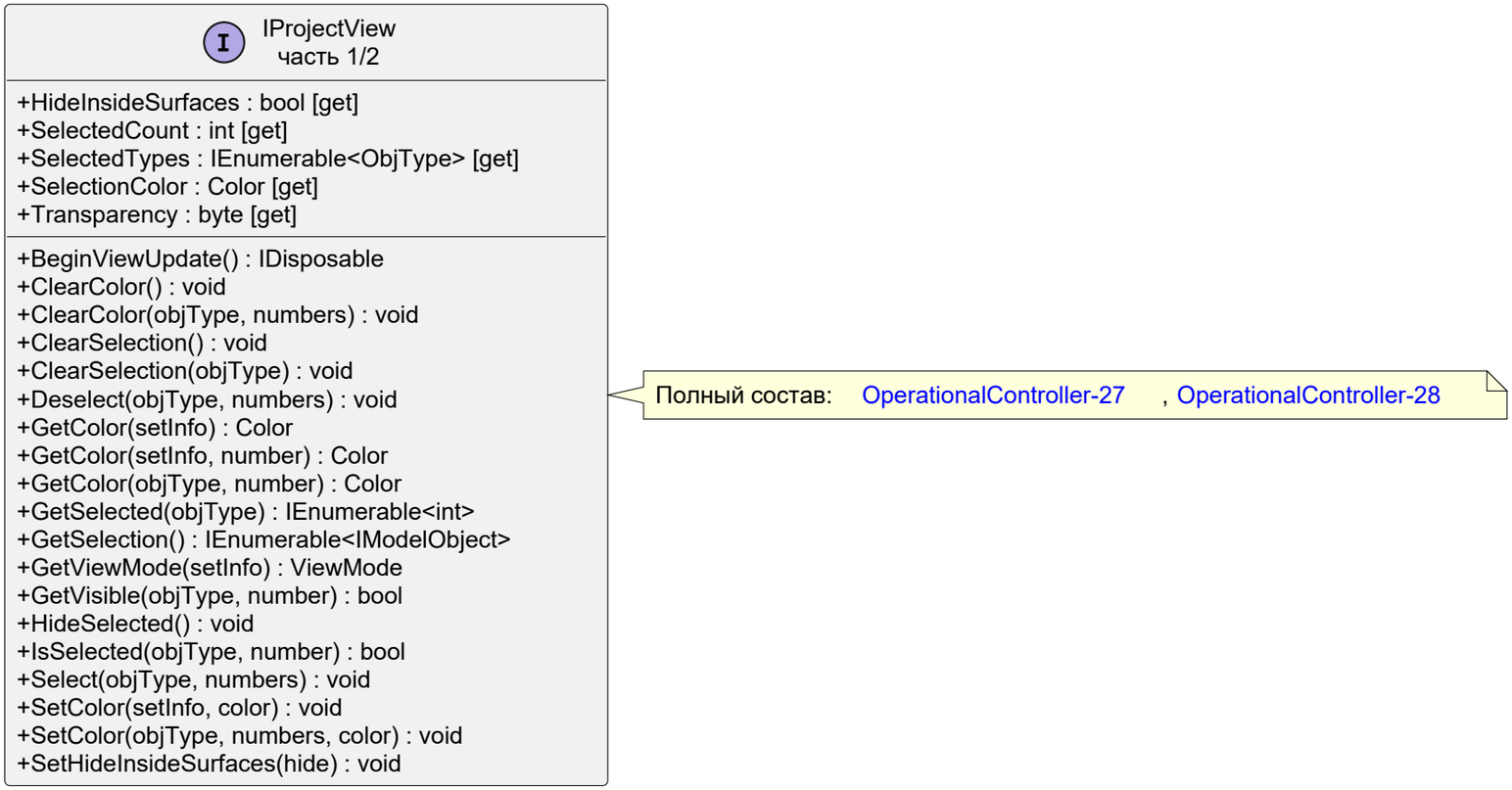
OperationalController-26 — Специализированные контракты контроллера



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

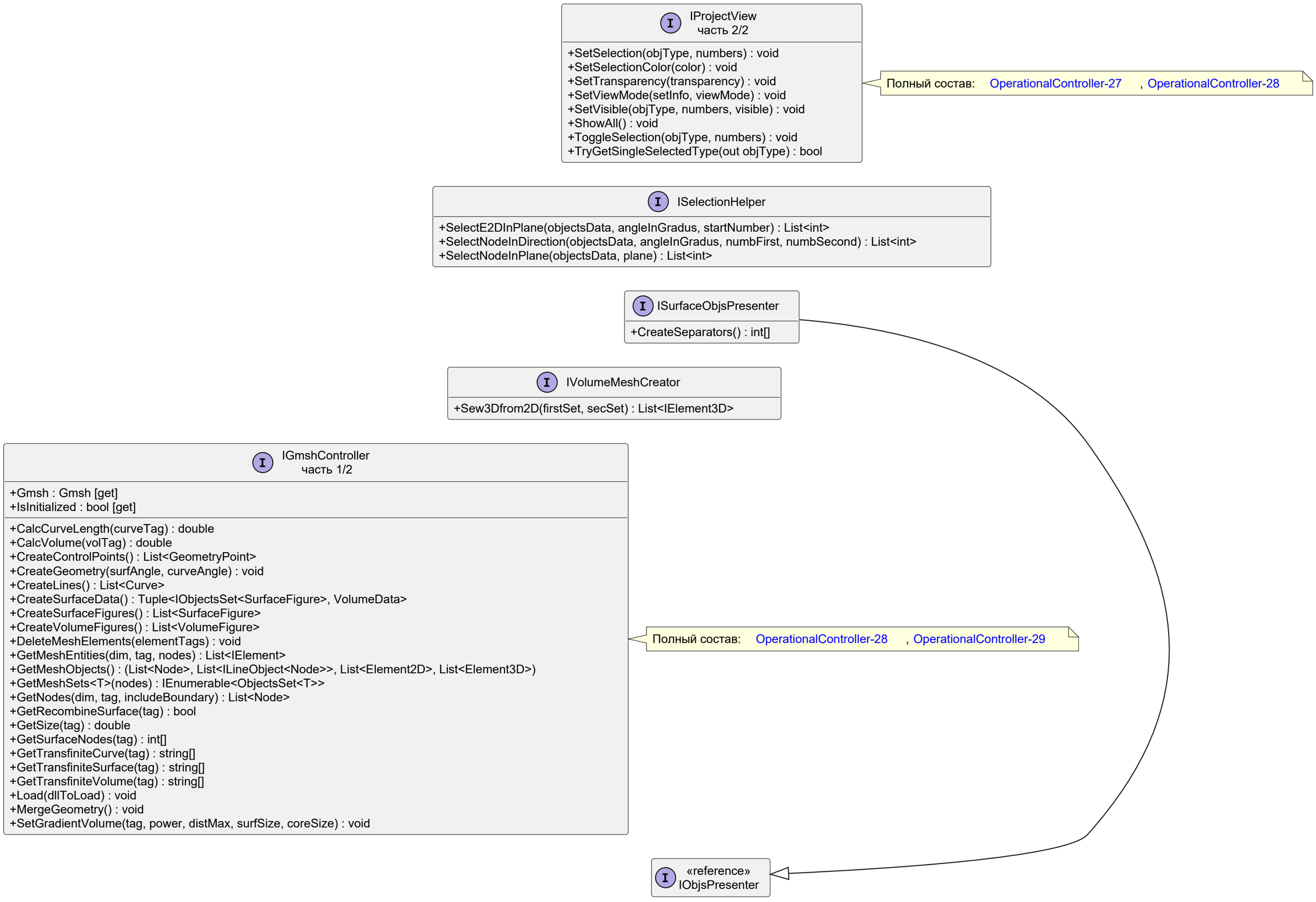
OperationalController-27 — Специализированные контракты контроллера



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

OperationalController-28 — Специализированные контракты контроллера



reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

OperationalController-29 — Специализированные контракты контроллера

IGmshController
часть 2/2

+SetRecombineSurface(tag) : void
+SetSize(tag, meshDize) : void
+SetTransfiniteCurve(tag, numNodes, meshType, coef) : void
+SetTransfiniteSurface(tag, arrangement, cornerTags) : void
+SetTransfiniteVolume(tag, cornerTags) : void
+Unload() : void

Полный состав: OperationalController-28 , OperationalController-29

IExtrusionStrategy <TResult>

+AddElement(extruder, correctMatrix, outIndex, inIndex) : void
+GetResult() : TResult
+Initialize(extruder, extrudeElements) : void
+PostProcess(extruder, rotateStatus, curveIndex, lastCurveIndex, lastCorrectMatrix) : void
+PreProcess(extruder, firstMatrix) : void

INormalCalculator

+CalcElemsNormals(elements) : IEnumerable<Curve>
+CalcNodeNormals(elements) : Dictionary<int, Point3D>

IObjsColors

+Type : ObjsColorsType [get]
+CreateColors(objectsCount) : IEnumerable<Color>

ILineObjsVertexCreator <T>

+Create(ref stride, vertexAtributes, nodeObject, color) : void

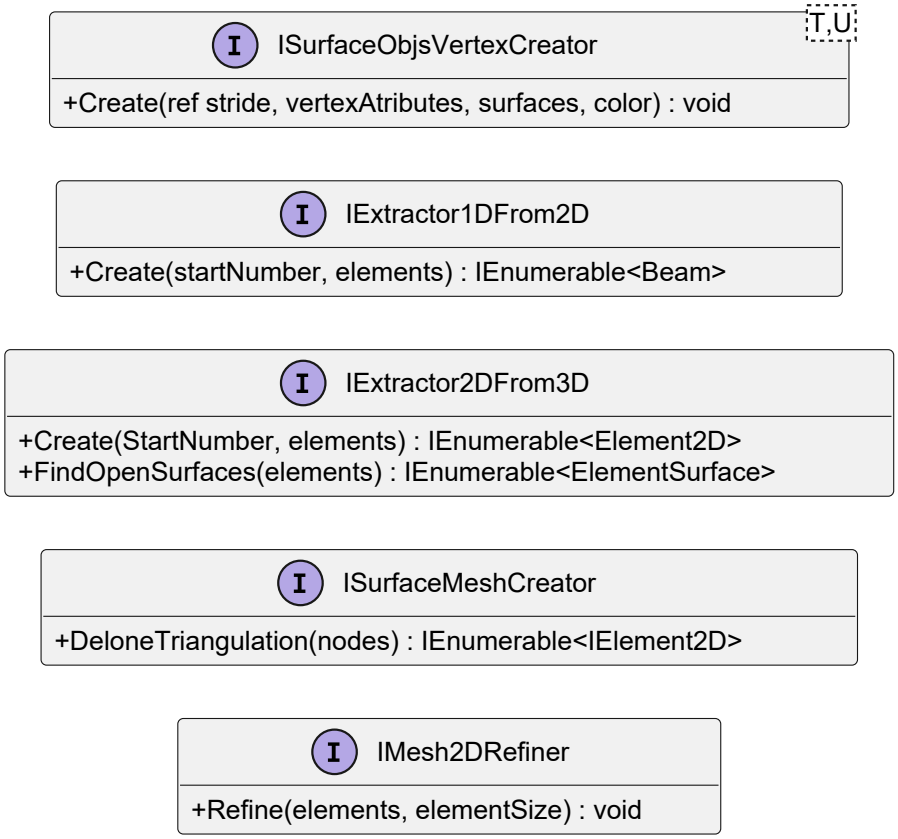
IPointObjsVertexCreator

+Create(ref stride, vertexAtributes, pointObject, color) : void

reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

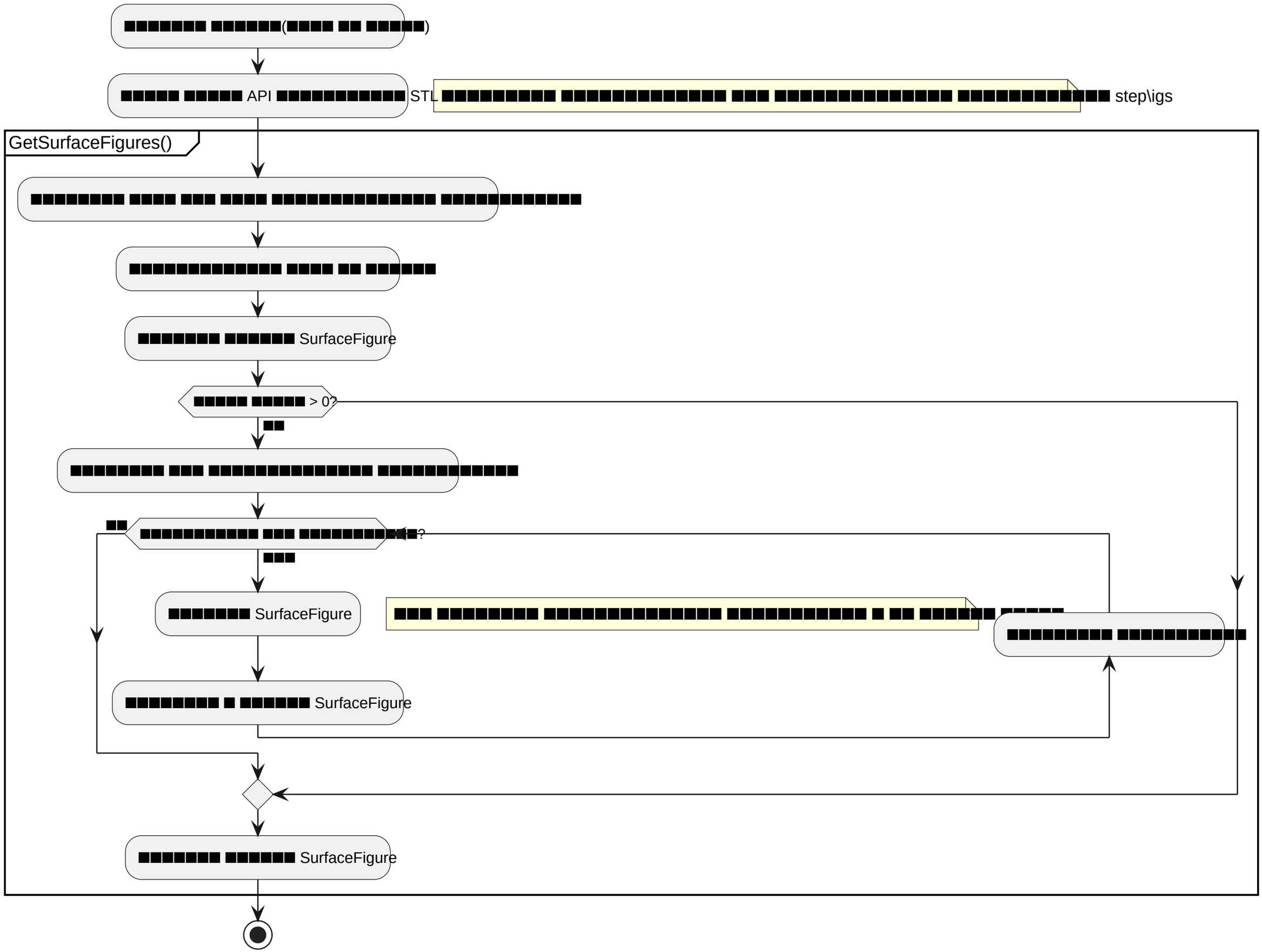
OperationalController-30 — Специализированные контракты контроллера



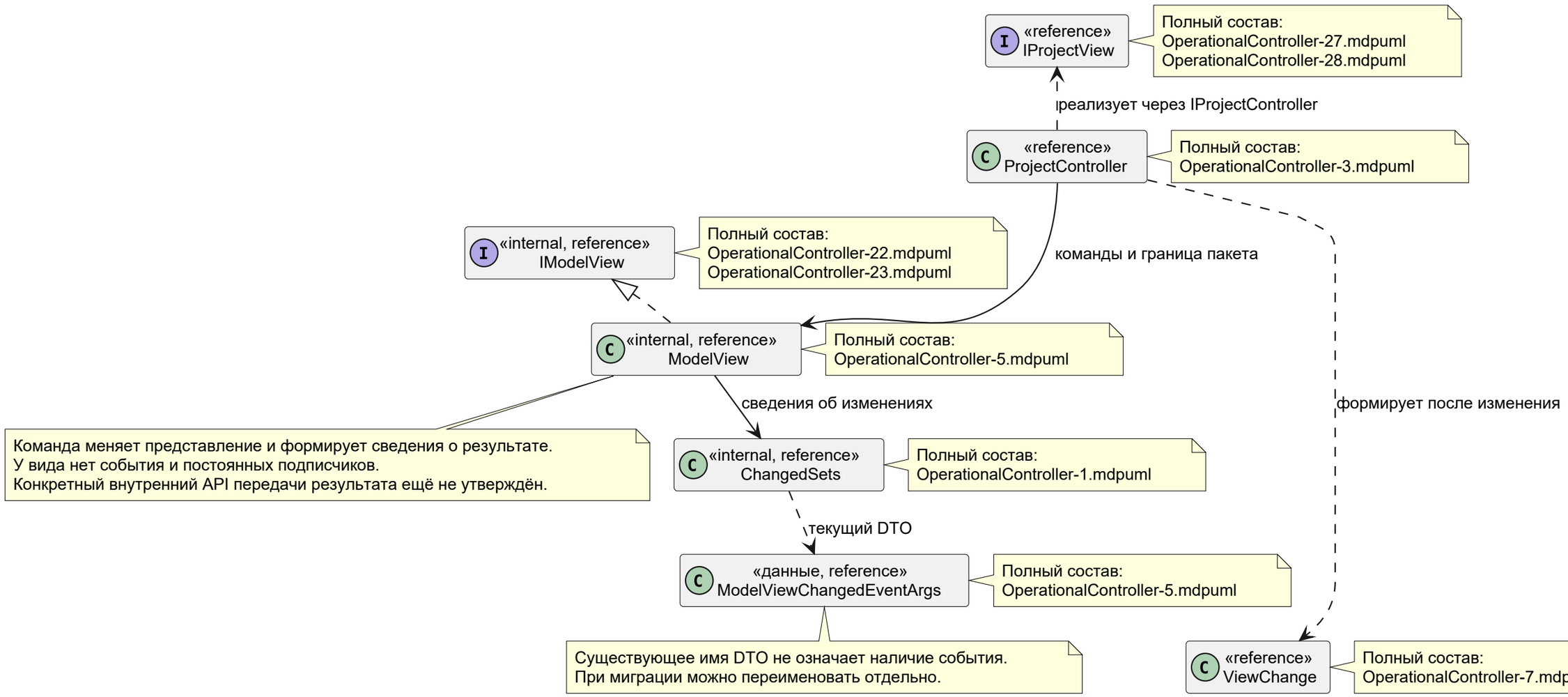
reference — тип раскрыт на другой диаграмме.
Параметры методов: имена; полные сигнатуры — в соседнем .md.
Унаследованные члены интерфейсов в классах не повторяются.

Целевой состав API; расхождения с кодом — ArchitectureDifferences.md

```
This syntax is deprecated, you must add <<#green>> at the end of the line, after the ';'
You should use a bracket ({) when defining your container 'group' GetSurfaceFigures()
You should use a bracket (}) instead of 'end group'
```

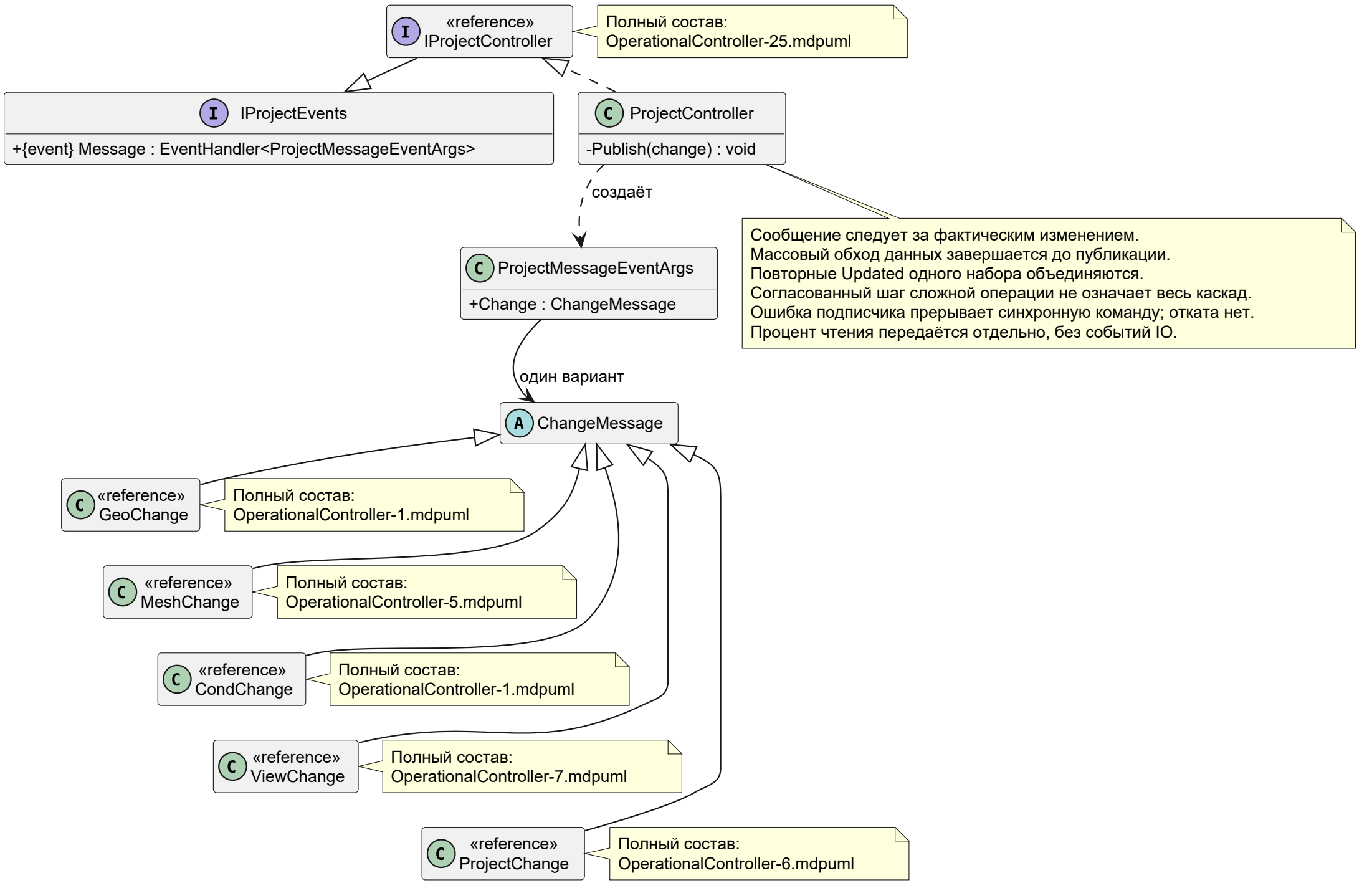


ModelView — целевое представление без событий

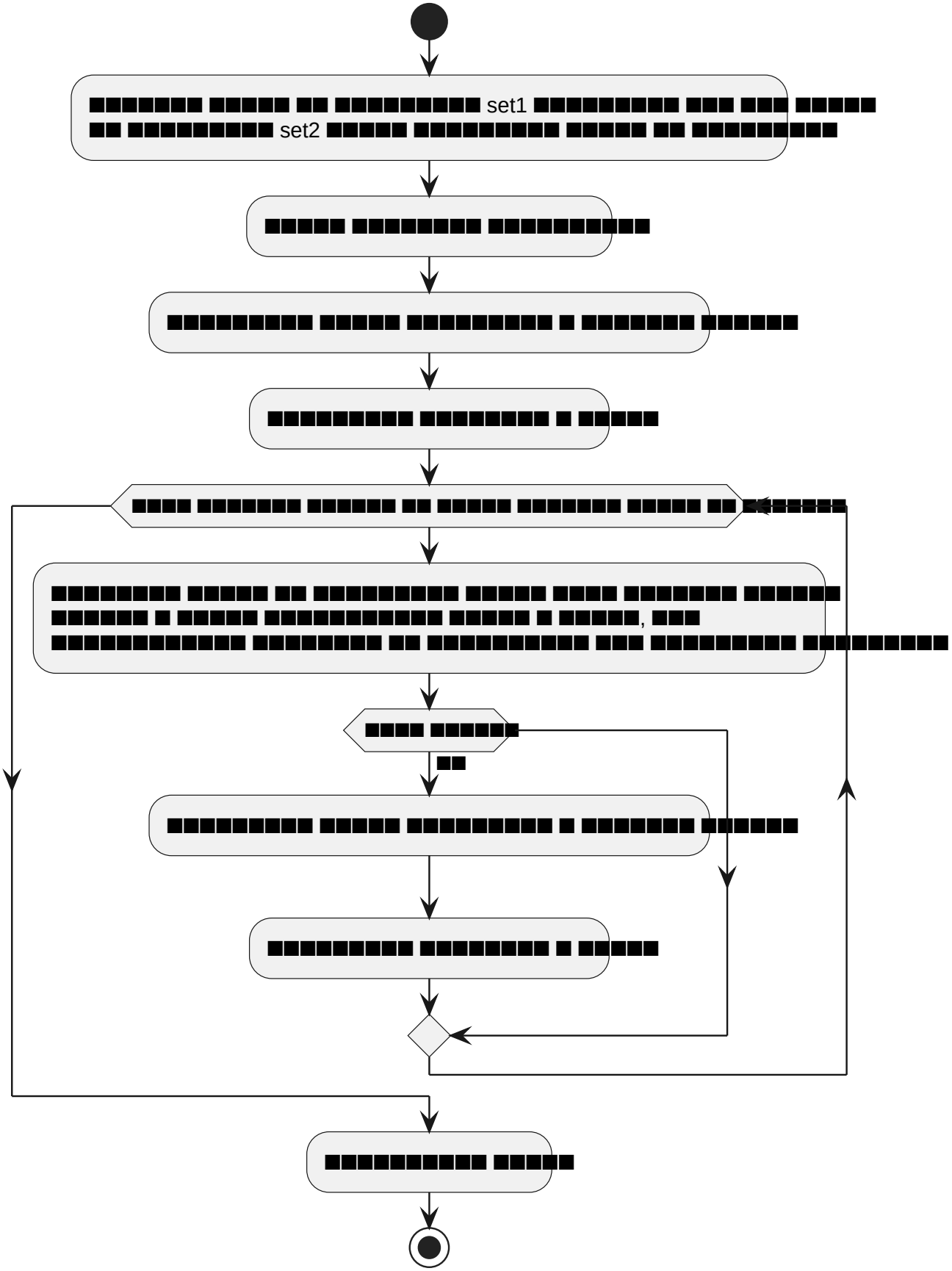


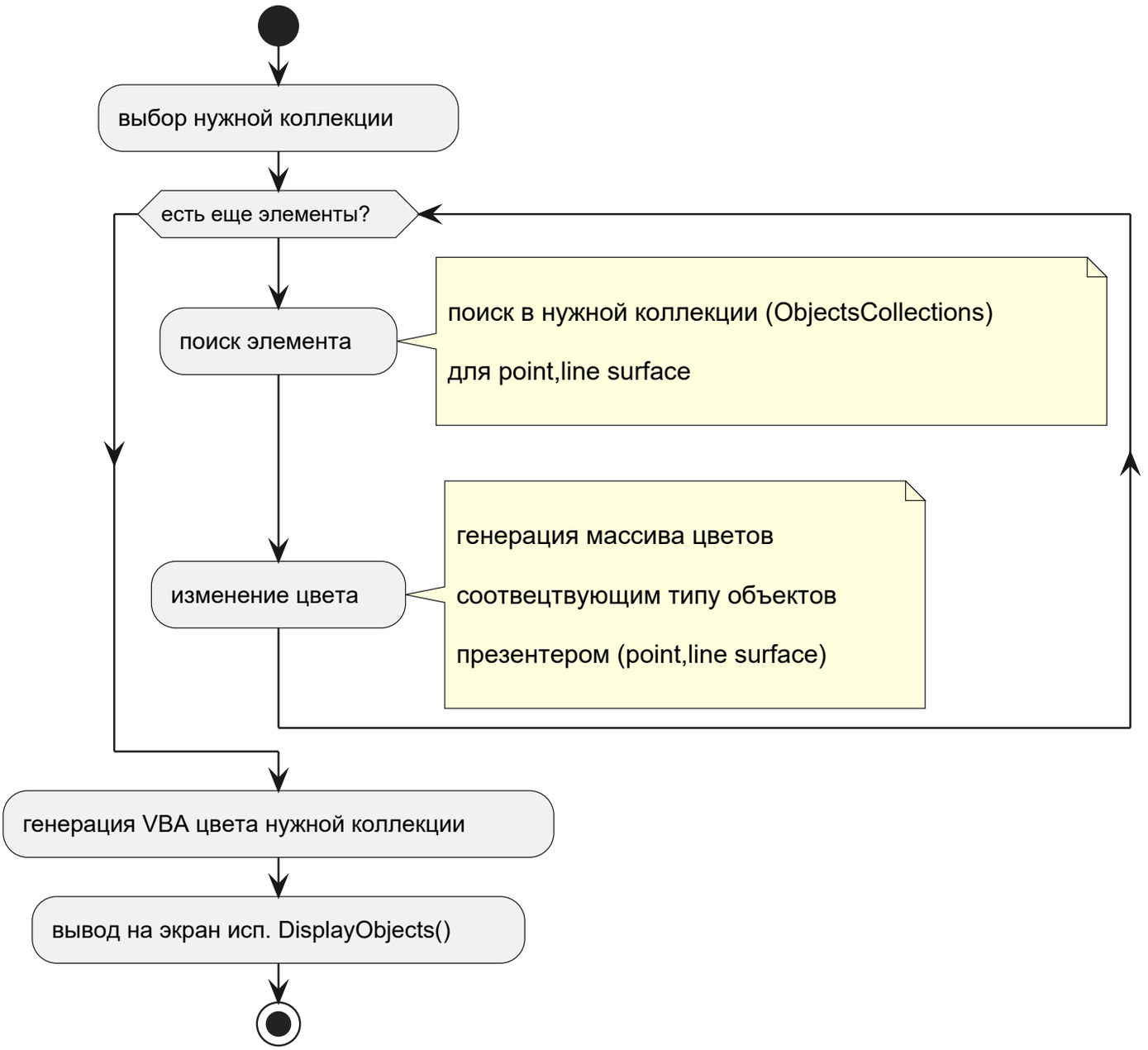
Публичное событие вызывает ProjectController; Changed подлежит удалению

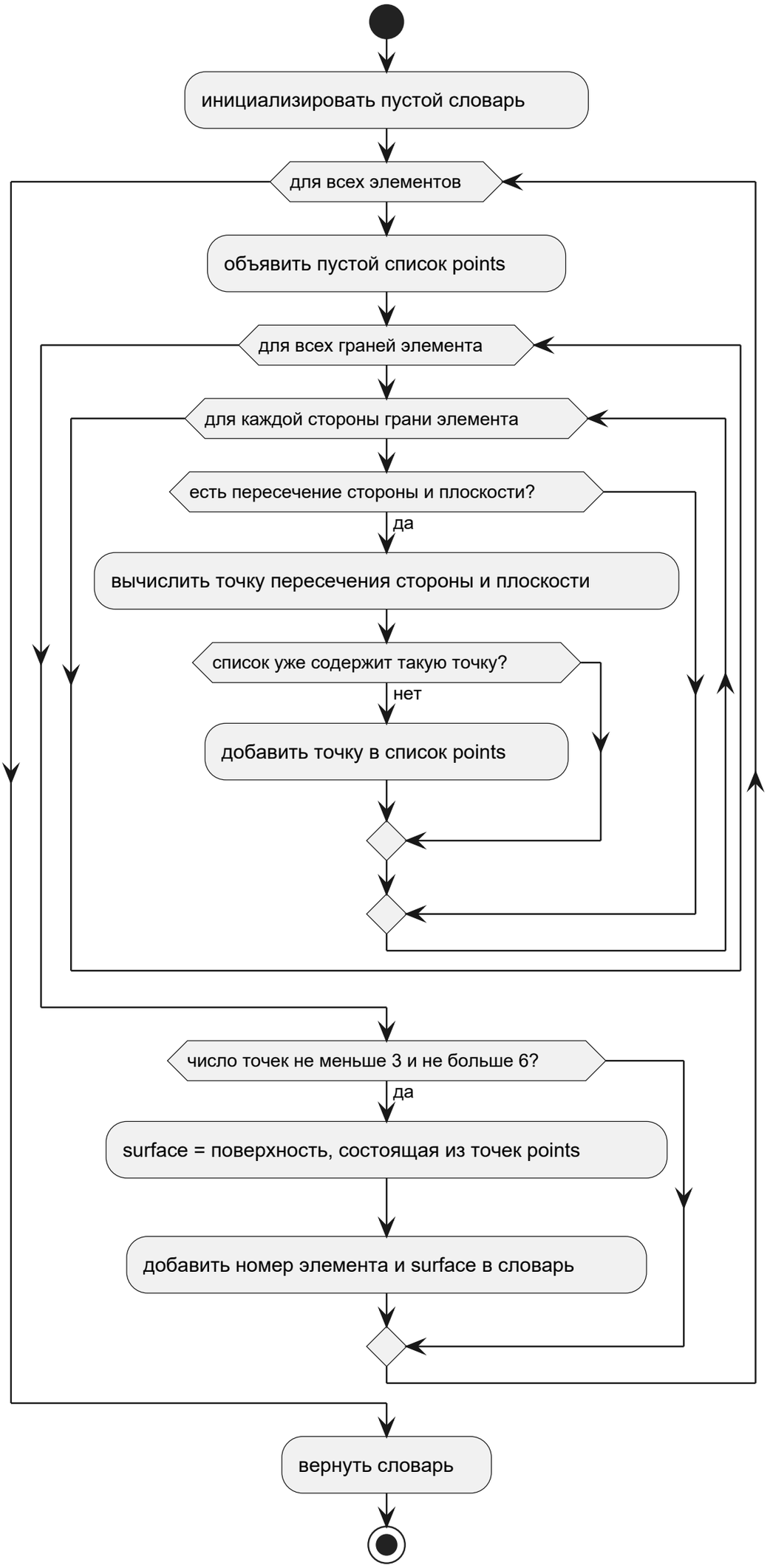
ProjectEvents — сообщения единственного события контроллера

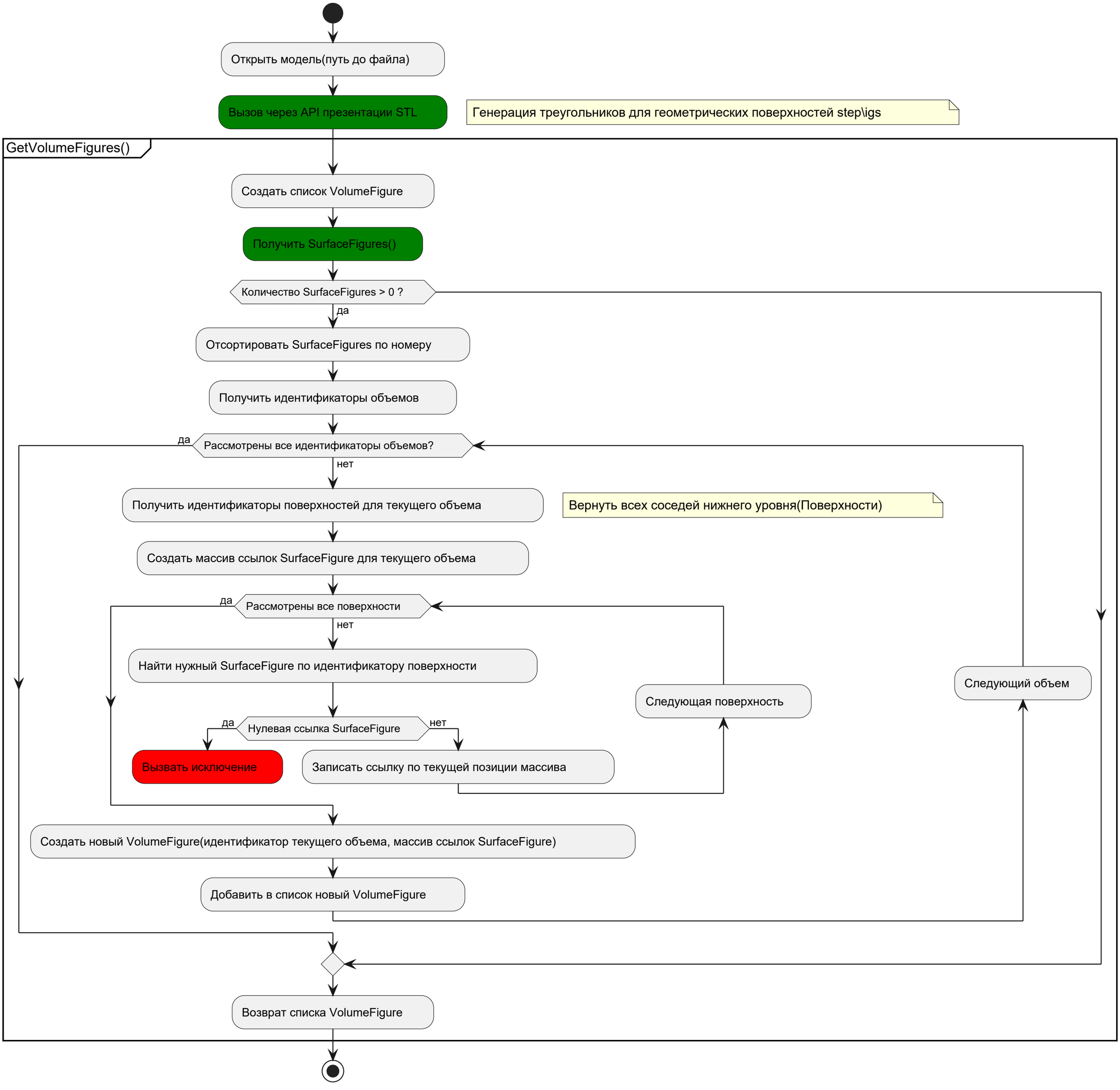


Целевой протокол; события ModelView и библиотек отсутствуют


set1
set2


выбор нужной коллекции
есть еще элементы?
поиск элемента
поиск в нужной коллекции (ObjectsCollections)
для point,line surface
изменение цвета
генерация массива цветов
соотвецтвующим типу объектов
презентером (point,line surface)
генерация VBA цвета нужной коллекции
вывод на экран исп. DisplayObjects()


инициализировать пустой словарь
для всех элементов
объявить пустой список points
для всех граней элемента
для каждой стороны грани элемента
есть пересечение стороны и плоскости?
да
вычислить точку пересечения стороны и плоскости
список уже содержит такую точку?
нет
добавить точку в список points
число точек не меньше 3 и не больше 6?
да
surface = поверхность, состоящая из точек points
добавить номер элемента и surface в словарь
вернуть словарь


Открыть модель(путь до файла)
Вызов через API презентации STL
Генерация треугольников для геометрических поверхностей step\igs
GetVolumeFigures()
Создать список VolumeFigure
Получить SurfaceFigures()
Количество SurfaceFigures > 0 ?
да
Отсортировать SurfaceFigures по номеру
Получить идентификаторы объемов
да
Рассмотрены все идентификаторы объемов?
нет
Получить идентификаторы поверхностей для текущего объема
Вернуть всех соседей нижнего уровня(Поверхности)
Создать массив ссылок SurfaceFigure для текущего объема
да
Рассмотрены все поверхности
нет
Найти нужный SurfaceFigure по идентификатору поверхности
Следующий объем
Следующая поверхность
да
Нулевая ссылка SurfaceFigure
нет
Вызвать исключение
Записать ссылку по текущей позиции массива
Создать новый VolumeFigure(идентификатор текущего объема, массив ссылок SurfaceFigure)
Добавить в список новый VolumeFigure
Возврат списка VolumeFigure

OperationalController

`ProjectController` реализует `IProjectController` и координирует изменения проекта, геометрии, сетки и представления. Только этот класс вызывает событие `IProjectEvents.Message`; зависимые библиотеки и внутренний `ModelView` событий в целевой архитектуре не содержат.

Обзорная диаграмма. Архитектура решения. Расхождения с кодом.

Поддиаграммы

Основная схема показывает связи и адреса подробностей. На поддиаграммах раскрыты методы, свойства и события интерфейсов; классы содержат собственные методы без повторения интерфейсного контракта. Методы в диаграммах показывают имена параметров; полные сигнатуры и ссылки на код находятся в соседних Markdown-файлах.

Диаграмма	Область
OperationalController-1	Представление и сообщения об изменениях; ChangeAction, ChangeMessage, ChangedSets, CondChange, GeoChange, GeometryEntityKind
OperationalController-2	Создание объектов, операции и презентаторы; CurveMeshingSettings, GradientVolumeMeshingSettings, MeshSettings, ModelLoaderFactory, PresenterType, SurfaceMeshingSettings
OperationalController-3	Регистрация изменений в ProjectController; ProjectController
OperationalController-4	Специализированные контракты контроллера; IBoundaryEdgesFinder, ICoincidentObjectsFinder, IConditionCommands, IConditionQueries, ICrossSectionMaker, IFreeNodesFinder
OperationalController-5	Представление и сообщения об изменениях; MeshChange, MeshDataKind, ModelView, ModelViewChange, ModelViewChangedEventArgs, ObjectsView
OperationalController-6	Представление и сообщения об изменениях; ProjectChange, ProjectChangeKind, ProjectMessageEventArgs, ProjectProperty, SetStyle, UpdateScope
OperationalController-7	Представление и сообщения об изменениях; ViewChange, ViewMode
OperationalController-8	Создание объектов, операции и презентаторы; VolumeMeshingSettings, VolumeMeshingType, ChamferGeometryBuilder, Convert2DMeshTo2DGeo, CurveLoop, CurveLoopNode
OperationalController-9	Создание объектов, операции и презентаторы; MiddleSurfaceBuilder, Orient, GmshController, BeamGroupConnection, Extract1DFrom2D, Extract2DFrom3D

Диаграмма	Область
OperationalController-10	Создание объектов, операции и презентаторы; OctreeBeamGroupConnection, QuadMeshCreator, SimpleBeamGroupConnection, VolumeMeshCreator, CollectionEmptyArgumentException, NegativeRadiusArgumentException
OperationalController-11	Создание объектов, операции и презентаторы; CurveExtruder, Element2DSorter, Extrude3DStrategy, ExtrudeBeamStrategy, MeshExtruder, RotateExtruder
OperationalController-12	Создание объектов, операции и презентаторы; TranslateExtruder, CrossSection, ElementData, FindBoundaryEdges, FindCoincidentObjects, FindOpenSurfaceNodes
OperationalController-13	Создание объектов, операции и презентаторы; FreeNodesFinder, InterfacedNodesFinder, MergeObjects, NormalsCalculator, NormalsChecker, ObjectsRenumber
OperationalController-14	Создание объектов, операции и презентаторы; SelectionHelper, EmptyCollectionException, ObjectsDimensionException, PresentersCreator, ElementSurfaceObjsPresenter, LineObjsPresenter
OperationalController-15	Создание объектов, операции и презентаторы; ManyColorsObjsColors, ObjsColorsType, ObjsPresenter, PointObjsPresenter, SingleColorObjsColors
OperationalController-16	Создание объектов, операции и презентаторы; SurfaceObjsPresenter, ColorLineObjsCreator, ColorPointObjsCreator, ColorSurfaceObjsCreator, CoordsLineObjsCreator
OperationalController-17	Создание объектов, операции и презентаторы; CoordsPointObjsCreator, CoordsSurfaceObjsCreator, EdgesSurfaceObjsCreator, NormalsLineObjsCreator, NormalsPointObjsCreator, NormalsSurfaceObjsCreator
OperationalController-18	Создание объектов, операции и презентаторы; BrigthnessController
OperationalController-19	Специализированные контракты контроллера; IGeometryCommands, IGeometryQueries, IGroupCommands, IGroupQueries, IInterfacedNodesFinder
OperationalController-20	Специализированные контракты контроллера; IMeshCommands
OperationalController-21	Специализированные контракты контроллера; IMeshCommands, IMeshQueries
OperationalController-22	Специализированные контракты контроллера; IMeshQueries, IModelView

Диаграмма	Область
OperationalController-23	Специализированные контракты контроллера; IModelView, IObjectsMerger, IObjectsRenumber, IObjectsView, IObjsPresenter
OperationalController-24	Специализированные контракты контроллера; IPresentersCreator
OperationalController-25	Специализированные контракты контроллера; IProjectController
OperationalController-26	Специализированные контракты контроллера; IProjectEvents, IProjectSession, IProjectStorage
OperationalController-27	Специализированные контракты контроллера; IProjectView
OperationalController-28	Специализированные контракты контроллера; IProjectView, ISelectionHelper, ISurfaceObjsPresenter, IVolumeMeshCreator, IGmshController
OperationalController-29	Специализированные контракты контроллера; IGmshController, IExtrusionStrategy, INormalCalculator, IObjsColors, ILineObjsVertexCreator, IPointObjsVertexCreator
OperationalController-30	Специализированные контракты контроллера; ISurfaceObjsVertexCreator, IExtractor1DFrom2D, IExtractor2DFrom3D, ISurfaceMeshCreator, IMesh2DRefiner

Указатель типов

Тип	Диаграммы	Исходник
`OperationalController.ChangeAction`	OperationalController-1	ChangeMessage.cs
`OperationalController.ChangeMessage`	OperationalController-1	ChangeMessage.cs
`OperationalController.ChangedSets`	OperationalController-1	ChangedSets.cs
`OperationalController.CondChange`	OperationalController-1	ChangeMessage.cs
`OperationalController.CurveMeshingSettings`	OperationalController-2	CurveMeshingSettings.cs
`OperationalController.GeoChange`	OperationalController-1	ChangeMessage.cs
`OperationalController.GeometryEntityKind`	OperationalController-1	ChangeMessage.cs
`OperationalController.GradientVolumeMeshingSettings`	OperationalController-2	VolumeMeshingSettings.cs

Тип	Диаграммы	Исходник
`OperationalController.IBoundaryEdgesFinder`	OperationalController-4	IBoundaryEdgesFinder.cs
`OperationalController.ICoincidentObjectsFinder`	OperationalController-4	ICoincidentObjectsFinder.cs
`OperationalController.IConditionCommands`	OperationalController-4	IConditionCommands.cs
`OperationalController.IConditionQueries`	OperationalController-4	IConditionQueries.cs
`OperationalController.ICrossSectionMaker`	OperationalController-4	ICrossSectionMaker.cs
`OperationalController.IFreeNodesFinder`	OperationalController-4	IFreeNodesFinder.cs
`OperationalController.IGeometryCommands`	OperationalController-19	IGeometryCommands.cs
`OperationalController.IGeometryQueries`	OperationalController-19	IGeometryQueries.cs
`OperationalController.IGroupCommands`	OperationalController-19	IGroupCommands.cs
`OperationalController.IGroupQueries`	OperationalController-19	IGroupQueries.cs
`OperationalController.IInterfacedNodesFinder`	OperationalController-19	IInterfacedNodesFinder.cs
`OperationalController.IMeshCommands`	OperationalController-20, OperationalController-21	IMeshCommands.cs
`OperationalController.IMeshQueries`	OperationalController-21, OperationalController-22	IMeshQueries.cs
`OperationalController.IModelView`	OperationalController-22, OperationalController-23	IModelView.cs
`OperationalController.IObjectsMerger`	OperationalController-23	IObjectsMerger.cs
`OperationalController.IObjectsRenumber`	OperationalController-23	IObjectsRenumber.cs
`OperationalController.IObjectsView`	OperationalController-23	IObjectsView.cs
`OperationalController.IObjsPresenter`	OperationalController-23	IObjsPresenter.cs
`OperationalController.IPresentersCreator`	OperationalController-24	IPresentersCreator.cs

Тип	Диаграммы	Исходник
`OperationalController.IProjectController`	OperationalController-25	IProjectController.cs
`OperationalController.IProjectEvents`	OperationalController-26	IProjectEvents.cs
`OperationalController.IProjectSession`	OperationalController-26	IProjectSession.cs
`OperationalController.IProjectStorage`	OperationalController-26	IProjectStorage.cs
`OperationalController.IProjectView`	OperationalController-27, OperationalController-28	IProjectView.cs
`OperationalController.ISelectionHelper`	OperationalController-28	ISelectionHelper.cs
`OperationalController.ISurfaceObjsPresenter`	OperationalController-28	ISurfaceObjsPresenter.cs
`OperationalController.IVolumeMeshCreator`	OperationalController-28	IVolumeMeshCreator.cs
`OperationalController.MeshChange`	OperationalController-5	ChangeMessage.cs
`OperationalController.MeshDataKind`	OperationalController-5	ChangeMessage.cs
`OperationalController.MeshSettings`	OperationalController-2	GetMeshSettings.cs
`OperationalController.ModelLoaderFactory`	OperationalController-2	ModelLoaderFactory.cs
`OperationalController.ModelView`	OperationalController-5	ModelView.cs
`OperationalController.ModelViewChange`	OperationalController-5	ModelViewChange.cs
`OperationalController.ModelViewChangedEventArgs`	OperationalController-5	ModelViewChangedEventArgs.cs
`OperationalController.ObjectsView`	OperationalController-5	ObjectsView.cs
`OperationalController.PresenterType`	OperationalController-2	IObjsPresenter.cs
`OperationalController.ProjectChange`	OperationalController-6	ChangeMessage.cs
`OperationalController.ProjectChangeKind`	OperationalController-6	ChangeMessage.cs

Тип	Диаграммы	Исходник
`OperationalController.ProjectController`	OperationalController-3	ProjectController.cs
`OperationalController.ProjectMessageEventArgs`	OperationalController-6	ProjectMessageEventArgs.cs
`OperationalController.ProjectProperty`	OperationalController-6	ChangeMessage.cs
`OperationalController.SetStyle`	OperationalController-6	SetStyle.cs
`OperationalController.SurfaceMeshingSettings`	OperationalController-2	SurfaceMeshingSettings.cs
`OperationalController.UpdateScope`	OperationalController-6	UpdateScope.cs
`OperationalController.ViewChange`	OperationalController-7	ChangeMessage.cs
`OperationalController.ViewMode`	OperationalController-7	ViewMode.cs
`OperationalController.VolumeMeshingSettings`	OperationalController-8	VolumeMeshingSettings.cs
`OperationalController.VolumeMeshingType`	OperationalController-8	VolumeMeshingSettings.cs
`OperationalController.GeomObjsCreator.ChamferGeometryBuilder`	OperationalController-8	ChamferGeometryBuilder.cs
`OperationalController.GeomObjsCreator.Convert2DMeshTo2DGeo`	OperationalController-8	Convert2DMeshTo2DGeo.cs
`OperationalController.GeomObjsCreator.CurveLoop`	OperationalController-8	CurveLoop.cs
`OperationalController.GeomObjsCreator.CurveLoop.CurveLoopNode`	OperationalController-8	CurveLoop.cs
`OperationalController.GeomObjsCreator.MiddleSurfaceBuilder`	OperationalController-9	MiddleSurfaceBuilder.cs
`OperationalController.GeomObjsCreator.CurveLoop.Orient`	OperationalController-9	CurveLoop.cs

OperationalController

Тип	Диаграммы	Исходник
`OperationalController.GmshController.GmshController`	OperationalController-9	GmshController.cs
`OperationalController.GmshController.IGmshController`	OperationalController-28, OperationalController-29	IGmshController.cs
`OperationalController.MeshObjsCreator.BeamGroupConnection`	OperationalController-9	BeamGroupConnection.cs
`OperationalController.MeshObjsCreator.Extract1DFrom2D`	OperationalController-9	Extract1DFrom2D.cs
`OperationalController.MeshObjsCreator.Extract2DFrom3D`	OperationalController-9	Extract2DFrom3D.cs
`OperationalController.MeshObjsCreator.OctreeBeamGroupConnection`	OperationalController-10	OctreeBeamGroupConnection.cs
`OperationalController.MeshObjsCreator.QuadMeshCreator`	OperationalController-10	QuadMeshCreator.cs
`OperationalController.MeshObjsCreator.SimpleBeamGroupConnection`	OperationalController-10	SimpleBeamGroupConnection.cs
`OperationalController.MeshObjsCreator.VolumeMeshCreator`	OperationalController-10	VolumeMeshCreator.cs
`OperationalController.MeshObjsCreator.Exceptions.CollectionEmptyArgumentException`	OperationalController-10	CollectionArgumentException.cs
`OperationalController.MeshObjsCreator.Exceptions.NegativeRadiusArgumentException`	OperationalController-10	RadiusArgumentException.cs
`OperationalController.MeshObjsCreator.Extrude.CurveExtruder`	OperationalController-11	CurveExtruder.cs
`OperationalController.MeshObjsCreator.Extrude.MeshExtruder.Element2DSorter`	OperationalController-11	MeshExtruder.cs

Тип	Диаграммы	Исходник
`OperationalController.MeshObjsCreator.Extrude.Extrude3DStrategy`	OperationalController-11	Extrude3DStrategy.cs
`OperationalController.MeshObjsCreator.Extrude.ExtrudeBeamStrategy`	OperationalController-11	ExtrudeBeamStrategy.cs
`OperationalController.MeshObjsCreator.Extrude.IExtrusionStrategy<TResult>`	OperationalController-29	IExtrusionStrategy.cs
`OperationalController.MeshObjsCreator.Extrude.MeshExtruder`	OperationalController-11	MeshExtruder.cs
`OperationalController.MeshObjsCreator.Extrude.RotateExtruder`	OperationalController-11	RotateExtruder.cs
`OperationalController.MeshObjsCreator.Extrude.TranslateExtruder`	OperationalController-12	TranslateExtruder.cs
`OperationalController.MeshObjsUtility.CrossSection`	OperationalController-12	CrossSection.cs
`OperationalController.MeshObjsUtility.NormalsChecker.ElementData`	OperationalController-12	NormalsChecker.cs
`OperationalController.MeshObjsUtility.FindBoundaryEdges`	OperationalController-12	FindBoundaryEdges.cs
`OperationalController.MeshObjsUtility.FindCoincidentObjects`	OperationalController-12	FindCoincidentObjects.cs
`OperationalController.MeshObjsUtility.FindOpenSurfaceNodes`	OperationalController-12	FindOpenSurfaceNodes.cs
`OperationalController.MeshObjsUtility.FreeNodesFinder`	OperationalController-13	FreeNodesFinder.cs
`OperationalController.MeshObjsUtility.INormalCalculator`	OperationalController-29	INormalCalculator.cs
`OperationalController.MeshObjsUtility.InterfacedNodesFinder`	OperationalController-13	InterfacedNodesFinder.cs

Тип	Диаграммы	Исходник
OperationalController.MeshObjsUtility.MergeObjects	OperationalController-13	MergeObjects.cs
OperationalController.MeshObjsUtility.NormalsCalculator	OperationalController-13	NormalsCalculator.cs
OperationalController.MeshObjsUtility.NormalsChecker	OperationalController-13	NormalsChecker.cs
OperationalController.MeshObjsUtility.ObjectsRenumber	OperationalController-13	ObjectsRenumber.cs
OperationalController.MeshObjsUtility.SelectionHelper	OperationalController-14	SelectionHelper.cs
OperationalController.Methods.Exceptions.EmptyCollectionException	OperationalController-14	EmptyCollectionException.cs
OperationalController.Methods.Exceptions.ObjectsDimensionException	OperationalController-14	ObjectsDimensionException.cs
OperationalController.ModelScenePresentator.PresentersCreator	OperationalController-14	PresentersCreator.cs
OperationalController.ModelScenePresentator.GlObjsPresenters.ElementSurfaceObjsPresenter	OperationalController-14	ElementSurfaceObjsPresenter.cs
OperationalController.ModelScenePresentator.GlObjsPresenters.IObjsColors	OperationalController-29	IObjsColors.cs
OperationalController.ModelScenePresentator.GlObjsPresenters.LineObjsPresenter<T>	OperationalController-14	LineObjsPresenter.cs
OperationalController.ModelScenePresentator.GlObjsPresenters.ManyColorsObjsColors	OperationalController-15	IObjsColors.cs
OperationalController.ModelScenePresentator.GlObjsPresenters.ObjsColorsType	OperationalController-15	IObjsColors.cs

Тип	Диаграммы	Исходник
`OperationalController.ModelScenePresentator.GlObjsPresenters.ObjsPresenter<T>`	OperationalController-15	ObjsPresenter.cs
`OperationalController.ModelScenePresentator.GlObjsPresenters.PointObjsPresenter<T>`	OperationalController-15	PointObjsPresenter.Coordinated.cs
`OperationalController.ModelScenePresentator.GlObjsPresenters.PointObjsPresenter`	OperationalController-15	PointObjsPresenter.cs
`OperationalController.ModelScenePresentator.GlObjsPresenters.SingleColorObjsColors`	OperationalController-15	IObjsColors.cs
`OperationalController.ModelScenePresentator.GlObjsPresenters.SurfaceObjsPresenter<T>`	OperationalController-16	SurfaceObjsPresenter.Coordinated.cs
`OperationalController.ModelScenePresentator.GlObjsPresenters.SurfaceObjsPresenter<T, U>`	OperationalController-16	SurfaceObjsPresenter.cs
`OperationalController.ModelScenePresentator.GlObjsVertexCreators.ColorLineObjsCreator<T>`	OperationalController-16	ColorLineObjsCreator.cs
`OperationalController.ModelScenePresentator.GlObjsVertexCreators.ColorPointObjsCreator`	OperationalController-16	ColorPointObjsCreator.cs
`OperationalController.ModelScenePresentator.GlObjsVertexCreators.ColorSurfaceObjsCreator<T,U>`	OperationalController-16	ColorSurfaceObjsCreator.cs
`OperationalController.ModelScenePresentator.GlObjsVertexCreators.CoordsLineObjsCreator<T>`	OperationalController-16	CoordsLineObjsCreator.cs

Тип	Диаграммы	Исходник
`OperationalController.ModelScenePresentator.GlObjsVertexCreators.CoordsPointObjsCreator`	OperationalController-17	CoordsPointObjsCreator.cs
`OperationalController.ModelScenePresentator.GlObjsVertexCreators.CoordsSurfaceObjsCreator<T, U>`	OperationalController-17	CoordsSurfaceObjsCreator.cs
`OperationalController.ModelScenePresentator.GlObjsVertexCreators.EdgesSurfaceObjsCreator<T, U>`	OperationalController-17	EdgesSurfaceObjsCreator.cs
`OperationalController.ModelScenePresentator.GlObjsVertexCreators.NormalsLineObjsCreator<T>`	OperationalController-17	NormalsLineObjsCreator.cs
`OperationalController.ModelScenePresentator.GlObjsVertexCreators.NormalsPointObjsCreator`	OperationalController-17	NormalsPointObjsCreator.cs
`OperationalController.ModelScenePresentator.GlObjsVertexCreators.NormalsSurfaceObjsCreator<T, U>`	OperationalController-17	NormalsSurfaceObjsCreator.cs
`OperationalController.ModelScenePresentator.Interfaces.ILineObjsVertexCreator<T>`	OperationalController-29	ILineObjsVertexCreator.cs
`OperationalController.ModelScenePresentator.Interfaces.IPointObjsVertexCreator`	OperationalController-29	IPointObjsVertexCreator.cs
`OperationalController.ModelScenePresentator.Interfaces.ISurfaceObjsVertexCreator<T,U>`	OperationalController-30	ISurfaceObjsVertexCreator.cs
`OperationalController.ObjsCreator.IExtractor1DFrom2D`	OperationalController-30	IExtractor1DFrom2D.cs

Тип	Диаграммы	Исходник
`OperationalController.ObjsCreator.IExtractor2DFrom3D`	OperationalController-30	IExtractor2DFrom3D.cs
`OperationalController.SceneBrightnessController.BrigthnessController`	OperationalController-18	BrigthnessController.cs
`OperationalController.SurfaceMeshCreator.ISurfaceMeshCreator`	OperationalController-30	ISurfaceMeshCreator.cs
`OperationalController.Utilities.IMesh2DRefiner`	OperationalController-30	IMesh2DRefiner.cs

Уведомления

Публичный контракт и правила публикации: ProjectController.md. Сообщения: ProjectEvents.mdpuml. Целевой протокол вида: ModelView.md. Существующий `ModelView.Changed` и события IO перечислены в реестре расхождений; отсутствие этих событий на целевых диаграммах не означает изменения исходников.

Источники и статус

Состав типов и сигнатуры сверены с текущими `.cs` и исключениями `<Compile Remove>` в проекте. Диаграммы описывают целевое размещение событий. Старые `Classes.mdpuml`, схемы отдельных алгоритмов и их изображения имеют статус исторических материалов и не определяют актуальный контракт.

OperationalController-1 — Представление и сообщения об изменениях

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

ChangeAction

Пространство имён: `OperationalController`.

Исходники: ChangeMessage.cs.

```
Created
Deleted
Updated
```

ChangeMessage

Пространство имён: `OperationalController`.

Исходники: ChangeMessage.cs.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

ChangedSets

Пространство имён: `OperationalController`.

Исходники: ChangedSets.cs.

```
IsEmpty : bool [get]
Add(ISetInfo setInfo, ModelViewChange change) : void
Clear() : void
CreateEventArgs() : ModelViewChangedEventArgs
```

CondChange

Пространство имён: `OperationalController`.

Исходники: ChangeMessage.cs.

Базовые типы: `ChangeMessage`.

```
Action : ChangeAction [get]
DataKind : DataKind [get]
Id : int [get]
CondChange(DataKind dataKind, ChangeAction action, int id)
```

GeoChange

Пространство имён: `OperationalController`.

Исходники: ChangeMessage.cs.

Базовые типы: ChangeMessage.

```
Counts : IReadOnlyDictionary<GeometryEntityKind, int> [get]
GeoChange(IReadOnlyDictionary<GeometryEntityKind, int> counts)
```

GeometryEntityKind

Пространство имён: OperationalController.

Исходники: ChangeMessage.cs.

```
Curve
Point
Surface
Volume
```

OperationalController-2 — Создание объектов, операции и презентаторы

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

CurveMeshingSettings

Пространство имён: `OperationalController`.

Исходники: CurveMeshingSettings.cs.

```
Coefficient : double [get]
IsTransfinite : bool [get]
MeshType : MeshType [get]
NodesCount : int [get]
CurveMeshingSettings(bool isTransfinite, int nodesCount, MeshType meshType, double coefficient)
```

GradientVolumeMeshingSettings

Пространство имён: `OperationalController`.

Исходники: VolumeMeshingSettings.cs.

```
CoreElementSize : double [get]
DistanceMaximum : double [get]
Power : double [get]
SurfaceElementSize : double [get]
GradientVolumeMeshingSettings(double power, double distanceMaximum, double surfaceElementSize, double
coreElementSize)
```

MeshSettings

Пространство имён: `OperationalController`.

Исходники: GetMeshSettings.cs.

```
Algorithm2D : MeshAlgorithm2D [get]
Algorithm3D : MeshAlgorithm3D [get]
MaximumSize : double [get]
MinimumSize : double [get]
SizeFactor : double [get]
MeshSettings(double minimumSize, double maximumSize, MeshAlgorithm2D algorithm2D, MeshAlgorithm3D algorithm3D,
double sizeFactor)
```

ModelLoaderFactory

Пространство имён: `OperationalController`.

Исходники: ModelLoaderFactory.cs.

```
Create(string extension) : IModelLoader
```

PresenterType

Пространство имён: `OperationalController`.

Исходники: IObjsPresenter.cs.

```
Line
Point
Surface
```

SurfaceMeshingSettings

Пространство имён: `OperationalController`.

Исходники: SurfaceMeshingSettings.cs.

```
Arrangement : Arrangement [get]
BoundaryPointNumbers : IReadOnlyList<int> [get]
CornerPointNumbers : IReadOnlyList<int> [get]
EmbeddedCurveNumbers : IReadOnlyList<int> [get]
IsRecombined : bool [get]
IsTransfinite : bool [get]
SurfaceMeshingSettings(bool isTransfinite, IEnumerable<int> cornerPointNumbers, Arrangement arrangement, bool
isRecombined, IEnumerable<int> embeddedCurveNumbers, IEnumerable<int> boundaryPointNumbers)
```

OperationalController-3 — Регистрация изменений в ProjectController

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

`ProjectController`

Пространство имён: `OperationalController`.

Исходники: ProjectController.cs, ChangeMeshSetName.cs, ChangeMeshSetOrder.cs, ChangeTaskKind.cs, ChangeTaskType.cs, ConnectByBeams.cs, ContainsData.cs, CopyFile.cs, CreateBoundaryEdges.cs, CreateBoundaryMesh.cs, CreateData.cs, CreateGroup.cs, CreateModelObjectsPresentor.cs, CreateProject.cs, CreateTask.cs, DelData .cs, DeleteGeometry.cs, ExportMesh.cs, FindInterfacedNodes.cs, GenerateMesh.cs, GeometryMeshingSettings.cs, GetData.cs, GetGeometryFilePath.cs, GetGeometryMeshNodes.cs, GetMeshSettings.cs, HideMeshByGeo.cs, ImportCAD.cs, ImportMesh.cs, IO.cs, MarkModelSurfaces.cs, Merge.cs, ProjectSession.cs, ProjectView.cs, PublishChanges.cs, RecombineMesh.cs, RefineMesh.cs, RenumberElements.cs, SynchronizeGeometryMesh.cs, TransformMesh.cs, UpdateCondition.cs.

Базовые типы: `IProjectController`.

```
CreateProject(string projectFile) : void
ImportCAD(string projectFile) : void
ImportMesh(string filePath) : void
Load(string path) : void
ProjectController()
Publish(ChangeMessage change) : void
PublishConditionChange(DataKind kind, ChangeAction action, int index) : void
PublishGeometryChange() : void
PublishGroupChange(IGroup group, ChangeAction action) : void
PublishSetChange(ISetInfo setInfo, ChangeAction action) : void
PublishAppendedModel(IModelData appendedModel, IReadOnlyList<string> appendedNames,
IReadOnlyCollection<ISetInfo> surfaceChanges) : void
MarkModelSurfaces(IModelData model) : IReadOnlyCollection<ISetInfo>
AddCollectionObject<T>(IObjectsCollection<T> collection, string name, T item) : void where T : IModelObject
AddCollectionObjects<T>(IObjectsCollection<T> collection, string name, IEnumerable<T> items) : void where T :
IModelObject
AddSetObjects<T>(IObjectsSet<T> set, IEnumerable<T> items) : bool where T : IModelObject
AddCollectionSets<T>(IObjectsCollection<T> collection, IEnumerable<Model.ObjectsCollections.ObjectsSet<T>>
sets) : IReadOnlyCollection<ISetInfo> where T : IModelObject
RemoveDeadObjects<T>(IObjectsSet<T> set) : bool where T : IModelObject
RemoveDeadObjects<T>(IObjectsCollection<T> collection) : IReadOnlyCollection<ISetInfo> where T : IModelObject
RemoveGeometryEntry<T>(IObjectsSet<T> set, int number) : bool where T : IModelObject
Replace(IProjectData data, string path, ProjectChangeKind kind, bool isOpen) : void
ReplaceTaskData(ITaskData data) : void
ToEnum<T>(string value) : T where T : struct, Enum
```

OperationalController-4 — Специализированные контракты контроллера

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

IBoundaryEdgesFinder

Пространство имён: `OperationalController`.

Исходники: IBoundaryEdgesFinder.cs.

```
CreateBoundaryEdges(List<string> linesNodes6, IModelData modelData) : IEnumerable<ILineObject<Node>>
Find(IEnumerable<ISurfaceElement> surfaceElements) : List<string>
```

ICoincidentObjectsFinder

Пространство имён: `OperationalController`.

Исходники: ICoincidentObjectsFinder.cs.

```
Find<T>(List<T> modelObjects, float distance) : List<List<int>> where T : IModelObject
```

IConditionCommands

Пространство имён: `OperationalController`.

Исходники: IConditionCommands.cs.

```
AddTaskData(ICondData cond) : void
ClearNotExistedCondData() : void
ClearTaskData() : void
Create(DataKind kind, string info) : ICondData
DeleteCond(IGroup group) : void
DeleteCond(int index) : bool
UpdateCondition(int index, ICondData condition) : void
```

IConditionQueries

Пространство имён: `OperationalController`.

Исходники: IConditionQueries.cs.

```
GetAllCondData() : IEnumerable<ICondData>
GetCondData(int condIndex) : ICondData
```

ICrossSectionMaker

Пространство имён: `OperationalController`.

Исходники: ICrossSectionMaker.cs.

```
GetSectionSurfaces(IEnumerable<IElement3D> elements, Plane plane) : SurfaceFigure
```

IFreeNodesFinder

Пространство имён: `OperationalController`.

Исходники: IFreeNodesFinder.cs.

```
Find(IObjectsData objsData) : IEnumerable<int>
```

OperationalController-5 — Представление и сообщения об изменениях

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

MeshChange

Пространство имён: `OperationalController`.

Исходники: ChangeMessage.cs.

Базовые типы: `ChangeMessage`.

```
Action : ChangeAction [get]
DataKind : MeshDataKind [get]
EntityId : int [get]
ObjectType : ObjType [get]
MeshChange(MeshDataKind dataKind, ChangeAction action, ObjType objectType, int entityId)
```

MeshDataKind

Пространство имён: `OperationalController`.

Исходники: ChangeMessage.cs.

```
Group
Set
```

ModelView

Пространство имён: `OperationalController`.

Исходники: ModelView.cs.

Базовые типы: `IModelView`.

```
HasOpenUpdate : bool [get]
EndUpdate() : void
ModelView(IModelData model)
```

ModelViewChange

Пространство имён: `OperationalController`.

Исходники: ModelViewChange.cs.

```
InsideSurfaces
None
ObjectColor
Selection
SelectionColor
SetColor
Transparency
ViewMode
```

```
Visibility
```

ModelViewChangedEventArgs

Пространство имён: `OperationalController`.

Исходники: ModelViewChangedEventArgs.cs.

Базовые типы: `EventArgs`.

```
Count : int [get]
GetChangedSets() : IEnumerable<ISetInfo>
GetChanges(ISetInfo setInfo) : ModelViewChange
HasAny(ISetInfo setInfo, ModelViewChange changes) : bool
ModelViewChangedEventArgs(IReadOnlyDictionary<ISetInfo, ModelViewChange> sets)
```

ObjectsView

Пространство имён: `OperationalController`.

Исходники: ObjectsView.cs.

Базовые типы: `IObjectsView`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

OperationalController-6 — Представление и сообщения об изменениях

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

ProjectChange

Пространство имён: `OperationalController`.

Исходники: ChangeMessage.cs.

Базовые типы: `ChangeMessage`.

```
ChangeKind : ProjectChangeKind [get]
Property : ProjectProperty? [get]
ProjectChange(ProjectChangeKind changeKind, ProjectProperty? property= null)
```

ProjectChangeKind

Пространство имён: `OperationalController`.

Исходники: ChangeMessage.cs.

```
Closed
Created
Loaded
PropertyChanged
```

ProjectMessageEventArgs

Пространство имён: `OperationalController`.

Исходники: ProjectMessageEventArgs.cs.

Базовые типы: `EventArgs`.

```
Change : ChangeMessage [get]
ProjectMessageEventArgs(ChangeMessage change)
```

ProjectProperty

Пространство имён: `OperationalController`.

Исходники: ChangeMessage.cs.

```
FilePath
FunctionsDB
MaterialsDB
Name
TaskKind
TaskType
```

SetStyle

Пространство имён: `OperationalController`.

Исходники: SetStyle.cs.

```
Color : Color? [get, set]
ViewMode : ViewMode? [get, set]
```

UpdateScope

Пространство имён: `OperationalController`.

Исходники: UpdateScope.cs.

Базовые типы: `IDisposable`.

```
Dispose() : void
UpdateScope(ModelView view)
```

OperationalController-7 — Представление и сообщения об изменениях

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

ViewChange

Пространство имён: `OperationalController`.

Исходники: ChangeMessage.cs.

Базовые типы: `ChangeMessage`.

```
AffectedSets : IEnumerable<ISetInfo> [get]
ChangeKind : ModelViewChange [get]
GetChangeKind(ISetInfo setInfo) : ModelViewChange
ViewChange(IReadOnlyDictionary<ISetInfo, ModelViewChange> changes)
```

ViewMode

Пространство имён: `OperationalController`.

Исходники: ViewMode.cs.

```
Line
LineSurface
Point
Surface
```

OperationalController-8 — Создание объектов, операции и презентаторы

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

VolumeMeshingSettings

Пространство имён: `OperationalController`.

Исходники: VolumeMeshingSettings.cs.

```
GradientSettings : GradientVolumeMeshingSettings [get]
Type : VolumeMeshingType [get]
VolumeMeshingSettings(VolumeMeshingType type, GradientVolumeMeshingSettings gradientSettings= null)
```

VolumeMeshingType

Пространство имён: `OperationalController`.

Исходники: VolumeMeshingSettings.cs.

```
Gradient
Regular
Undefined
```

ChamferGeometryBuilder

Пространство имён: `OperationalController.GeomObjsCreator`.

Исходники: ChamferGeometryBuilder.cs.

```
Build(IEnumerable<int> curveTags, double firstLength, double secondLengthOrAngle, bool isAngle, bool
reflected= false) : int[]
ChamferGeometryBuilder(IGmshController controller)
```

Convert2DMeshTo2DGeo

Пространство имён: `OperationalController.GeomObjsCreator`.

Исходники: Convert2DMeshTo2DGeo.cs.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

CurveLoop

Пространство имён: `OperationalController.GeomObjsCreator`.

Исходники: CurveLoop.cs.

```
Loop : LinkedList<CurveLoopNode> [get, private set]
Orientation : Orient [get, private set]
Surface : int [get, private set]
CheckLoop() : void
CurveLoop(IGmshController controller, int surface, int index, int pointsPerCurve= 16)
```

```
MakeSimilar(CurveLoop src) : void
Rebuild(double[] coords) : void
Reverse(double[] coords= null) : void
```

CurveLoopNode

Пространство имён: `OperationalController.GeomObjsCreator`.

Исходники: CurveLoop.cs.

```
AverageCoords : double[] [get, private set]
Begin : int [get, private set]
End : int [get, private set]
Tag : int [get, private set]
CurveLoopNode(int begin, int end, int tag)
GetParametrization(IGmshController controller, int pointsPerCurve= 16) : double[]
Reverse() : void
SetAverageCoords(double[] coords) : void
```

OperationalController-9 — Создание объектов, операции и презентаторы

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

MiddleSurfaceBuilder

Пространство имён: `OperationalController.GeomObjsCreator`.

Исходники: MiddleSurfaceBuilder.cs.

```
RemoveDuplicates : bool [get, set]
UseSeparateModel : bool [get, set]
Build(IEnumerable<int> surfaces) : void
MiddleSurfaceBuilder(IGmshController controller)
```

Orient

Пространство имён: `OperationalController.GeomObjsCreator`.

Исходники: CurveLoop.cs.

```
CCW
CW
```

GmshController

Пространство имён: `OperationalController.GmshController`.

Исходники: GmshController.cs.

Базовые типы: `IGmshController`.

```
CreateSurfaceFigure(List<GeometryPoint> points, int surfaceTag) : SurfaceFigure
GetPointsCountByElementSize(int curveTag, float elemSize) : int
RemoveGeometryObjects(int[] dimTags, bool delChild= false) : bool
```

BeamGroupConnection

Пространство имён: `OperationalController.MeshObjsCreator`.

Исходники: BeamGroupConnection.cs.

```
Connect(IEnumerable<Node> smaller, IEnumerable<Node> larger) : IEnumerable<Beam>
Create(IEnumerable<Node> master, IEnumerable<Node> slave, double radius, int beamsCount= 0) : IEnumerable<Beam>
GetSquaredDistance(Node one, Node two) : double
```

Extract1DFrom2D

Пространство имён: `OperationalController.MeshObjsCreator`.

Исходники: Extract1DFrom2D.cs.

Базовые типы: `IExtractor1DFrom2D`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

`Extract2DFrom3D`

Пространство имён: `OperationalController.MeshObjsCreator`.

Исходники: Extract2DFrom3D.cs.

Базовые типы: `IExtractor2DFrom3D`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

OperationalController-10 — Создание объектов, операции и презентаторы

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

OctreeBeamGroupConnection

Пространство имён: `OperationalController.MeshObjsCreator`.

Исходники: OctreeBeamGroupConnection.cs.

Базовые типы: `BeamGroupConnection`.

```
Connect(IEnumerable<Node> master, IEnumerable<Node> slave) : IEnumerable<Beam>
```

QuadMeshCreator

Пространство имён: `OperationalController.MeshObjsCreator`.

Исходники: QuadMeshCreator.cs.

```
CreateQuadMeshOnPoligon(List<Point2D> points, int numberOfElem) : ObjectsCollection<Element2D>
```

SimpleBeamGroupConnection

Пространство имён: `OperationalController.MeshObjsCreator`.

Исходники: SimpleBeamGroupConnection.cs.

Базовые типы: `BeamGroupConnection`.

```
Connect(IEnumerable<Node> smaller, IEnumerable<Node> larger) : IEnumerable<Beam>
```

VolumeMeshCreator

Пространство имён: `OperationalController.MeshObjsCreator`.

Исходники: VolumeMeshCreator.cs.

Базовые типы: `IVolumeMeshCreator`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

CollectionEmptyArgumentException

Пространство имён: `OperationalController.MeshObjsCreator.Exceptions`.

Исходники: CollectionArgumentException.cs.

Базовые типы: `Exception`.

```
CollectionEmptyArgumentException()
CollectionEmptyArgumentException(string message)
CollectionEmptyArgumentException(string message, Exception innerException)
```

NegativeRadiusArgumentException

Пространство имён: `OperationalController.MeshObjsCreator.Exceptions`.

Исходники: RadiusArgumentException.cs.

Базовые типы: `Exception`.

```
NegativeRadiusArgumentException()
NegativeRadiusArgumentException(string message)
NegativeRadiusArgumentException(string message, Exception innerException)
```

OperationalController-11 — Создание объектов, операции и презентаторы

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

CurveExtruder

Пространство имён: `OperationalController.MeshObjsCreator.Extrude`.

Исходники: CurveExtruder.cs.

Базовые типы: `MeshExtruder`.

```
CreateBeam(Node lastNode, Node curNode) : Beam
CreateNextLayer(IEnumerable<Element2D> curLayer, Matrix4x4 curBasis) : List<Element2D>
CreateNode(Vector4 curCenter) : Node
CurveExtruder(IGmshController controller, int curveTag, double step, int pointTag)
CurveExtruder(IGmshController controller, int[] curveTags, double step, int pointTag, float maxAngle= MathF.PI
/ 2)
CurveExtruder(IGmshController controller, int[] curveTags, double[] steps, int pointTag, float maxAngle=
MathF.PI / 2)
Extrude(IEnumerable<Element2D> firstLayer) : List<List<Element3D>>
ExtrudeFromPoint(Node firstNode) : List<Beam>
ExtrudeTransfinite(IEnumerable<Element2D> firstLayer) : List<List<Element3D>>
GetTransformedVector(Vector4 coords, Matrix4x4 matrix) : Vector4
```

Element2DSorter

Пространство имён: `OperationalController.MeshObjsCreator.Extrude`.

Исходники: MeshExtruder.cs.

Базовые типы: `IComparer<Element2D>`.

```
Center : Point3D [get, set]
Compare(Element2D x, Element2D y) : int
```

Extrude3DStrategy

Пространство имён: `OperationalController.MeshObjsCreator.Extrude`.

Исходники: Extrude3DStrategy.cs.

Базовые типы: `IExtrusionStrategy<List<List<Element3D>>>`.

```
Transfinite : bool [get, set]
```

ExtrudeBeamStrategy

Пространство имён: `OperationalController.MeshObjsCreator.Extrude`.

Исходники: ExtrudeBeamStrategy.cs.

Базовые типы: `IExtrusionStrategy<List<Beam>>`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

MeshExtruder

Пространство имён: `OperationalController.MeshObjsCreator.Extrude`.

Исходники: MeshExtruder.cs.

```
MissedElements : int [get, private set]
NodeTag : int [get, set]
Nodes : Dictionary<int, Node> [get, private set]
CreateCopy(IEnumerable<Element2D> src) : List<Element2D>
CreateLayerTetrahedrons(IEnumerable<Element2D> curLayer, List<Element2D> nextLayer) : List<Element3D>
CreateTransfiniteLayer(IEnumerable<Element2D> curLayer, List<Element2D> nextLayer) : List<Element3D>
SortFirstLayer(IEnumerable<Element2D> layer) : List<Element2D>
```

RotateExtruder

Пространство имён: `OperationalController.MeshObjsCreator.Extrude`.

Исходники: RotateExtruder.cs.

Базовые типы: `MeshExtruder`.

```
ElementTag : ref int [get]
CreateNextLayer(IEnumerable<Element2D> curLayer, Matrix4x4 transform) : List<Element2D>
Extrude(IEnumerable<Element2D> curLayer, float radAngle, Vector3 origin, Vector3 rotAxis, int segments= 8) :
List<List<Element3D>>
ExtrudeTransfinite(IEnumerable<Element2D> curLayer, float radAngle, Vector3 origin, Vector3 rotAxis, int
segments= 8) : List<List<Element3D>>
```

OperationalController-12 — Создание объектов, операции и презентаторы

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

TranslateExtruder

Пространство имён: `OperationalController.MeshObjsCreator.Extrude`.

Исходники: TranslateExtruder.cs.

Базовые типы: `MeshExtruder`.

```
Extrude(IEnumerable<Element2D> curLayer, Point3D dir, int layers= 1, float layerSize= 1) :
List<List<Element3D>>
ExtrudeTransfinite(IEnumerable<Element2D> curLayer, Point3D dir, int layers= 1, float layerSize= 1) :
List<List<Element3D>>
```

CrossSection

Пространство имён: `OperationalController.MeshObjsUtility`.

Исходники: CrossSection.cs.

Базовые типы: `ICrossSectionMaker`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

ElementData

Пространство имён: `OperationalController.MeshObjsUtility`.

Исходники: NormalsChecker.cs.

```
center : Point3D [field]
surfaceEdges : List<Point3D> [field]
surfaceNormals : List<Point3D> [field]
surfacePoints : List<Point3D> [field]
ElementData()
```

FindBoundaryEdges

Пространство имён: `OperationalController.MeshObjsUtility`.

Исходники: FindBoundaryEdges.cs.

Базовые типы: `IBoundaryEdgesFinder`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

FindCoincidentObjects

Пространство имён: `OperationalController.MeshObjsUtility`.

Исходники: FindCoincidentObjects.cs.

Базовые типы: `ICoincidentObjectsFinder`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

FindOpenSurfaceNodes

Пространство имён: `OperationalController.MeshObjsUtility`.

Исходники: FindOpenSurfaceNodes.cs.

```
Find(IEnumerable<IElement3D> elements) : IEnumerable<int>
```

OperationalController-13 — Создание объектов, операции и презентаторы

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

`FreeNodesFinder`

Пространство имён: `OperationalController.MeshObjsUtility`.

Исходники: FreeNodesFinder.cs.

Базовые типы: `IFreeNodesFinder`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

`InterfacedNodesFinder`

Пространство имён: `OperationalController.MeshObjsUtility`.

Исходники: InterfacedNodesFinder.cs.

Базовые типы: `IInterfacedNodesFinder`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

`MergeObjects`

Пространство имён: `OperationalController.MeshObjsUtility`.

Исходники: MergeObjects.cs.

Базовые типы: `IObjectsMerger`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

`NormalsCalculator`

Пространство имён: `OperationalController.MeshObjsUtility`.

Исходники: NormalsCalculator.cs.

Базовые типы: `INormalCalculator`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

`NormalsChecker`

Пространство имён: `OperationalController.MeshObjsUtility`.

Исходники: NormalsChecker.cs.

```
Inverted : int [get, private set]
Log : string [get, set]
NotOthogonals : int [get, private set]
```

```
ZeroLengths : int [get, private set]
Check() : void
IdentifyByTriangles(int triangles) : ElementKind
NormalsChecker(SurfaceObjsPresenter<ElementSurface, Node> presenter)
```

ObjectsRenumber

Пространство имён: `OperationalController.MeshObjsUtility`.

Исходники: ObjectsRenumber.cs.

Базовые типы: `IObjectsRenumber`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

OperationalController-14 — Создание объектов, операции и презентаторы

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

SelectionHelper

Пространство имён: `OperationalController.MeshObjsUtility`.

Исходники: SelectionHelper.cs.

Базовые типы: `ISelectionHelper`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

EmptyCollectionException

Пространство имён: `OperationalController.Methods.Exceptions`.

Исходники: EmptyCollectionException.cs.

Базовые типы: `Exception`.

```
EmptyCollectionException()
EmptyCollectionException(string message)
EmptyCollectionException(string message, Exception innerException)
```

ObjectsDimensionException

Пространство имён: `OperationalController.Methods.Exceptions`.

Исходники: ObjectsDimensionException.cs.

Базовые типы: `Exception`.

```
ObjectsDimensionException()
ObjectsDimensionException(string message)
ObjectsDimensionException(string message, Exception innerException)
```

PresentersCreator

Пространство имён: `OperationalController.ModelScenePresentator`.

Исходники: PresentersCreator.cs.

Базовые типы: `IPresentersCreator`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

ElementSurfaceObjsPresenter

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsPresenters`.

Исходники: ElementSurfaceObjsPresenter.cs.

Базовые типы: `SurfaceObjsPresenter<ElementSurface, Node>`.

```
ElementSurfaceObjsPresenter(IReadOnlyList<ISurfaceObject<ElementSurface, Node>> elements, IReadOnlyList<Color>
colors, IModelView modelView)
GetDrawnSurfaces(int index) : IEnumerable<ElementSurface>
```

LineObjsPresenter<T>

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsPresenters`.

Исходники: LineObjsPresenter.cs.

Базовые типы: `ObjsPresenter<T>`.

```
LineObjsPresenter(IReadOnlyList<T> lineObjects, Color color)
LineObjsPresenter(IReadOnlyList<T> lineObjects, IObjsColors colors)
LineObjsPresenter(IReadOnlyList<T> lineObjects, IReadOnlyList<Color> colors)
```

OperationalController-15 — Создание объектов, операции и презентаторы

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

ManyColorsObjsColors

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsPresenters`.

Исходники: IObjsColors.cs.

Базовые типы: `IObjsColors`.

```
ManyColorsObjsColors(IReadOnlyList<Color> colors)
```

ObjsColorsType

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsPresenters`.

Исходники: IObjsColors.cs.

```
Many
Single
```

ObjsPresenter<T>

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsPresenters`.

Исходники: ObjsPresenter.cs.

Базовые типы: `IObjsPresenter`.

```
Objects : IReadOnlyList<T> [get]
CreateColors() : IEnumerable<Color>
GetEnumerator() : IEnumerator<ICoordinated>
ObjsPresenter(IReadOnlyList<T> objects, IObjsColors colors)
```

PointObjsPresenter<T>

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsPresenters`.

Исходники: PointObjsPresenter.Coordinated.cs.

Базовые типы: `ObjsPresenter<T>`.

```
PointObjsPresenter(IReadOnlyList<T> pointObjects, Color color)
PointObjsPresenter(IReadOnlyList<T> pointObjects, IObjsColors colors)
PointObjsPresenter(IReadOnlyList<T> pointObjects, IReadOnlyList<Color> colors)
```

PointObjsPresenter

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsPresenters`.

Исходники: PointObjsPresenter.cs.

Базовые типы: `ObjsPresenter<IModelObject>`.

```
PointObjsPresenter(IReadOnlyList<IModelObject> pointObjects, Color color)
PointObjsPresenter(IReadOnlyList<IModelObject> pointObjects, IObjsColors colors)
PointObjsPresenter(IReadOnlyList<IModelObject> pointObjects, IReadOnlyList<Color> colors)
```

SingleColorObjsColors

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsPresenters`.

Исходники: IObjsColors.cs.

Базовые типы: `IObjsColors`.

```
SingleColorObjsColors(Color color)
```

OperationalController-16 — Создание объектов, операции и презентаторы

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

SurfaceObjsPresenter<T>

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsPresenters`.

Исходники: SurfaceObjsPresenter.Coordinated.cs.

Базовые типы: `ObjsPresenter<T>`, `ISurfaceObjsPresenter`.

```
SurfaceObjsPresenter(IReadOnlyList<T> surfaceObjects, Color color)
SurfaceObjsPresenter(IReadOnlyList<T> surfaceObjects, IObjsColors colors)
SurfaceObjsPresenter(IReadOnlyList<T> surfaceObjects, IReadOnlyList<Color> colors)
```

SurfaceObjsPresenter<T, U>

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsPresenters`.

Исходники: SurfaceObjsPresenter.cs.

Базовые типы: `ObjsPresenter<ISurfaceObject<T, U>>`, `ISurfaceObjsPresenter`.

```
GetDrawnSurfaces(int index) : IEnumerable<T>
SurfaceObjsPresenter(IReadOnlyList<ISurfaceObject<T, U>> surfaceObjects, Color color, bool drawInnerSurfaces=
true)
SurfaceObjsPresenter(IReadOnlyList<ISurfaceObject<T, U>> surfaceObjects, IObjsColors colors, bool
drawInnerSurfaces= true)
SurfaceObjsPresenter(IReadOnlyList<ISurfaceObject<T, U>> surfaceObjects, IReadOnlyList<Color> colors, bool
drawInnerSurfaces= true)
```

ColorLineObjsCreator<T>

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsVertexCreators`.

Исходники: ColorLineObjsCreator.cs.

Базовые типы: `ILineObjsVertexCreator<T>`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

ColorPointObjsCreator

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsVertexCreators`.

Исходники: ColorPointObjsCreator.cs.

Базовые типы: `IPointObjsVertexCreator`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

ColorSurfaceObjsCreator<T,U>

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsVertexCreators`.

Исходники: ColorSurfaceObjsCreator.cs.

Базовые типы: `ISurfaceObjsVertexCreator<T,U>`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

CoordsLineObjsCreator<T>

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsVertexCreators`.

Исходники: CoordsLineObjsCreator.cs.

Базовые типы: `ILineObjsVertexCreator<T>`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

OperationalController-17 — Создание объектов, операции и презентаторы

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

CoordsPointObjsCreator

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsVertexCreators`.

Исходники: CoordsPointObjsCreator.cs.

Базовые типы: `IPointObjsVertexCreator`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

CoordsSurfaceObjsCreator<T, U>

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsVertexCreators`.

Исходники: CoordsSurfaceObjsCreator.cs.

Базовые типы: `ISurfaceObjsVertexCreator<T, U>`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

EdgesSurfaceObjsCreator<T, U>

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsVertexCreators`.

Исходники: EdgesSurfaceObjsCreator.cs.

```
Create(ref int stride, bool[] vertexAtributes, IEnumerable<T> surfaces) : void
```

NormalsLineObjsCreator<T>

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsVertexCreators`.

Исходники: NormalsLineObjsCreator.cs.

Базовые типы: `ILineObjsVertexCreator<T>`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

NormalsPointObjsCreator

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsVertexCreators`.

Исходники: NormalsPointObjsCreator.cs.

Базовые типы: `IPointObjsVertexCreator`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

NormalsSurfaceObjsCreator<T, U>

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsVertexCreators`.

Исходники: NormalsSurfaceObjsCreator.cs.

Базовые типы: `ISurfaceObjsVertexCreator<T, U>`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

OperationalController-18 — Создание объектов, операции и презентаторы

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

`BrigthnessController`

Пространство имён: `OperationalController.SceneBrightnessController`.

Исходники: BrigthnessController.cs.

```
ApplyBrightness(Color color, float dimFactor, float desaturateFactor) : Color
```

OperationalController-19 — Специализированные контракты контроллера

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

IGeometryCommands

Пространство имён: `OperationalController`.

Исходники: IGeometryCommands.cs.

```
CreateChamfer(IEnumerable<int> curveTags, double firstLength, double secondLengthOrAngle, bool isAngle, bool
reflected) : int[]
CreateLine(int startPointNumber, int endPointNumber) : int
CreatePoint(double x, double y, double z, double meshSize= 0) : int
CreatePointByVector(int basePointTag, int directionPointTag, double offset) : int
CreatePointProjectionOntoGeometry(int basePointTag, int dim, int geometryTag) : int
CreateSurface(int[] lineNumbers) : int
DeleteGeometryObject(int dim, int number, bool recursive= false) : void
InitializeGeometry(string gmshDllPath) : void
SetEmbeddedGeometryEntities(int hostDimension, int hostNumber, int embeddedDimension, IEnumerable<int>
embeddedNumbers) : void
UnloadGeometry() : void
```

IGeometryQueries

Пространство имён: `OperationalController`.

Исходники: IGeometryQueries.cs.

```
GetAdjacentGeometryObjects(IModelObject obj, int direction) : IEnumerable<IModelObject>
GetAdjacentGeometryObjects(int dim, int number) : Tuple<int[], int[]>
GetBoundary(int[] dimTags, bool combined= true, bool oriented= true, bool recursive= false) : int[]
GetCenterOfMass(int dim, int tag) : (double X, double Y, double Z)
GetCurveLength(int curveTag) : double
GetGeoObjPoints(int dim, int tag, double[] parametricCoord) : double[]
GetGeometryFilePath() : string
GetGeometryLastError() : string
GetModelCurve(int objNumber) : Curve
GetModelPoint(int objNumber) : GeometryPoint
GetModelSurface(int objNumber) : SurfaceFigure
GetModelVolumes() : IEnumerable<VolumeFigure>
GetParametrization(int dim, int tag, double[] coordinates) : double[]
GetParametrizationBounds(int dim, int tag) : Tuple<double[], double[]>
GetSurfaceNormal(int surfaceTag, double[] parametricCoordinates) : double[]
GetType(int dim, int tag) : string
SelectByScope(int startDim, IEnumerable<int> selectedNumbers, int targetDim) : HashSet<int>
```

IGroupCommands

Пространство имён: `OperationalController`.

Исходники: IGroupCommands.cs.

```
ClearGroupData() : void
ClearNotExistedGroupData() : void
CreateGroup(IEnumerable<IModelObject> objs) : void
CreateGroup(string grName, IEnumerable<IModelObject> objs) : void
CreateGroupByGeoObjs(int meshDim, int geoDim, int tag) : string
CreateGroupByMeshObjs(int dim, string setName) : string
CreateOpenSurfaceNodesGroup(string setName) : string
DeleteModelGroup(string grName) : void
```

IGroupQueries

Пространство имён: `OperationalController`.

Исходники: IGroupQueries.cs.

```
ContainsConditions(string grName) : bool
GetAllModelGroups() : IEnumerable<IGroup>
GetModelGroup(int grIndex) : IGroup
GetModelGroup(string grName) : IGroup
```

IInterfacedNodesFinder

Пространство имён: `OperationalController`.

Исходники: IInterfacedNodesFinder.cs.

```
Find(IEnumerable<IElement> elements) : Dictionary<int, List<int>>
Find(List<int> nodes, IEnumerable<IElement> elements) : Dictionary<int, List<int>>
```

OperationalController-20 — Специализированные контракты контроллера

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

IMeshCommands

Пространство имён: `OperationalController`.

Исходники: IMeshCommands.cs.

Части: OperationalController-20, OperationalController-21.

```
ChangeMeshSetName(int elemDim, string oldName, string newName) : void
ChangeMeshSetOrder(int elemDim, string name, int order) : void
ClearEmptySet() : void
ClearGeometryMesh() : void
ClearGeometryMesh(int dimension) : void
ClearModelCollection(ObjType objType) : void
ClearNotExistedModelData() : void
ConnectByBeams(string masterGroup, string slaveGroup, double radius, int maxBeams) : void
Create1DFrom2D() : bool
Create2DForm3D() : bool
CreateBoundaryEdges(List<string> linesNodes) : IEnumerable<ILineObject<Node>>
CreateElement(string setName, int dim, int[] nodeNumbers) : void
CreateElementsSet(string setName, int dim) : void
CreateNode(int number, float x, float y, float z) : void
CreateQuadMeshOnPoligon(List<Point2D> points, int numberOfElem) : void
DeleteModelObjects(ObjType objType, IEnumerable<int> objNumbs) : void
DeleteModelSet(ObjType objType, string name) : void
Extrude3DForm2D(Point3D vector, int layers, IEnumerable<IElement2D> elements) : void
ExtrudeElement1DAlongCurve(int[] curveTags, int startPointTag, int startNodeTag, double step) : string
ExtrudeElement3DAlongCurve(int surfaceTag, int[] curveTags, int startPointTag, double step, bool transfinite)
: string
ExtrudeElement3DAlongCurve(string setName, int[] curveTags, int startPointTag, double step, bool transfinite)
: string
ExtrudeElement3DRotate(int surfaceTag, float angle, Vector3 originPoint, Vector3 rotAxis, bool transfinite) :
string
GenerateMesh(int elemDim) : void
HideMesh(int elemDim) : void
```

OperationalController-21 — Специализированные контракты контроллера

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

IMeshCommands

Пространство имён: `OperationalController`.

Исходники: IMeshCommands.cs.

Части: OperationalController-20, OperationalController-21.

```
MergeElements(ObjType objType, string setMaster, string setSlave) : void
MergeNodes(List<List<int>> numbers) : void
MoveMesh(ObjType objType, Point3D vector) : void
RecombineMesh() : void
RefineMesh() : void
RenumberElements() : void
RotateMesh(ObjType objType, Point3D axis, float angle) : void
SetCurveMeshingSettings(int curveNumber, CurveMeshingSettings settings) : void
SetMeshAlgorithm2D(MeshAlgorithm2D algorithm) : void
SetMeshAlgorithm3D(MeshAlgorithm3D algorithm) : void
SetMeshMaximumSize(double size) : void
SetMeshMinimumSize(double size) : void
SetMeshPointSize(int pointNumber, double size) : void
SetMeshSizeFactor(double factor) : void
SetSurfaceMeshingSettings(int surfaceNumber, SurfaceMeshingSettings settings) : void
SetVolumeMeshingSettings(int volumeNumber, VolumeMeshingSettings settings) : void
```

IMeshQueries

Пространство имён: `OperationalController`.

Исходники: IMeshQueries.cs.

Части: OperationalController-21, OperationalController-22.

```
CalcElemsNormals(int elemDim) : IEnumerable<Curve>
CalcNodeNormals() : Dictionary<int, Point3D>
ContainsModelObject(ObjType objType, int objNumber) : bool
CreateModelObjectsPresentor(ISetInfo setInfo) : IObjsPresenter
FindBoundaryEdges() : List<string>
FindCoincidentObjects(ObjType objType, float distance) : List<List<int>>
FindFreeNodes() : IEnumerable<int>
FindInterfacedNodes(List<int> nodes, int elemDim) : Dictionary<int, List<int>>
FindInterfacedNodes(int elemDim) : Dictionary<int, List<int>>
GetAllModelElements() : IEnumerable<IElement>
GetAllModelNodes() : IEnumerable<Node>
GetAllModelObjects() : IEnumerable<IModelObject>
GetAllModelSetsInfo() : IEnumerable<ISetInfo>
GetCurveMeshingSettings(int curveNumber) : CurveMeshingSettings
GetGeometryMeshNodes(int dimension, IEnumerable<int> entityNumbers, bool includeBoundary= false) :
IEnumerable<Node>
```

```
GetMeshPointSize(int pointNumber) : double
GetMeshSettings() : MeshSettings
GetModelElements(int dim, string setName) : IEnumerable<IElement>
GetModelObject(ObjType objType, int objNumber) : IModelObject
GetModelObjects(ObjType objType) : IEnumerable<IModelObject>
GetModelSetInfo(ObjType objType, int objNumber) : ISetInfo
GetModelSetInfo(ObjType objType, string setName) : ISetInfo
GetModelSetsInfo(ObjType objType) : IEnumerable<ISetInfo>
GetModelSurfaceElements(int dim) : IEnumerable<ISurfaceElement>
```

OperationalController-22 — Специализированные контракты контроллера

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

IMeshQueries

Пространство имён: `OperationalController`.

Исходники: IMeshQueries.cs.

Части: OperationalController-21, OperationalController-22.

```
GetSectionSurfaces(Plane plane) : SurfaceFigure
GetSurfaceMeshingSettings(int surfaceNumber) : SurfaceMeshingSettings
GetVolumeMeshingSettings(int volumeNumber) : VolumeMeshingSettings
SelectE2DInPlane(float angleInGradus, int startNumber, Color color) : List<int>
SelectNodeInDirection(float angleInGradus, int numbFirst, int numbSecond, Color color) : List<int>
SelectNodeInPlane(Plane plane, Color color) : List<int>
```

IModelView

Пространство имён: `OperationalController`.

Исходники: IModelView.cs.

Части: OperationalController-22, OperationalController-23.

```
HideInsideSurfaces : bool [get, set]
Model : IModelData [get]
SelectedCount : int [get]
SelectedTypes : IEnumerable<ObjType> [get]
SelectionColor : Color [get, set]
Transparency : byte [get, set]
BeginUpdate() : IDisposable
ClearColor() : void
ClearColor(ObjType objType, IEnumerable<int> numbers) : void
ClearSelection() : void
ClearSelection(ObjType objType) : void
Deselect(ObjType objType, IEnumerable<int> numbers) : void
GetColor(ISetInfo setInfo) : Color
GetColor(ISetInfo setInfo, int number) : Color
GetColor(ObjType objType, int number) : Color
GetSelected(ObjType objType) : IEnumerable<int>
GetSelection() : IEnumerable<IModelObject>
GetViewMode(ISetInfo setInfo) : ViewMode
GetVisible(ObjType objType, int number) : bool
HideSelected() : void
IsSelected(ObjType objType, int number) : bool
Prune() : void
Reset() : void
Select(ObjType objType, IEnumerable<int> numbers) : void
```

OperationalController-23 — Специализированные контракты контроллера

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

IModelView

Пространство имён: `OperationalController`.

Исходники: IModelView.cs.

Части: OperationalController-22, OperationalController-23.

```
SetColor(ISetInfo setInfo, Color color) : void
SetColor(ObjType objType, IEnumerable<int> numbers, Color color) : void
SetSelection(ObjType objType, IEnumerable<int> numbers) : void
SetViewMode(ISetInfo setInfo, ViewMode viewMode) : void
SetVisible(ObjType objType, IEnumerable<int> numbers, bool visible) : void
ShowAll() : void
ToggleSelection(ObjType objType, IEnumerable<int> numbers) : void
TryGetSingleSelectedType(out ObjType objType) : bool
```

IObjectsMerger

Пространство имён: `OperationalController`.

Исходники: IObjectsMerger.cs.

```
Merge(List<List<int>> numbers, IObjectsSet<Node> modelObjects) : void
Merge(List<List<int>> numbers, IObjectsSet<Node> modelObjects, out IReadOnlyCollection<IElement>
changedElements, out bool nodesRemoved) : void
```

IObjectsRenumber

Пространство имён: `OperationalController`.

Исходники: IObjectsRenumber.cs.

```
Renumber(IObjectsData objectsData, ObjType objType) : void
```

IObjectsView

Пространство имён: `OperationalController`.

Исходники: IObjectsView.cs.

```
HiddenCount : int [get]
SelectedCount : int [get]
ClearObjectColor(IEnumerable<int> numbers) : bool
ClearObjectColors() : bool
ClearSelection() : bool
ClearSelectionAndHidden() : bool
Deselect(IEnumerable<int> numbers) : bool
GetSelected() : IEnumerable<int>
```

```
IsHidden(int number) : bool
IsSelected(int number) : bool
Prune(Predicate<int> exists) : bool
PruneSetStyles(IEnumerable<ISetInfo> existingSets) : bool
Select(IEnumerable<int> numbers) : bool
SetHidden(IEnumerable<int> numbers, bool hidden) : bool
SetObjectColor(IEnumerable<int> numbers, Color color) : bool
SetSelection(IEnumerable<int> numbers) : bool
SetSetColor(ISetInfo setInfo, Color color) : bool
SetViewMode(ISetInfo setInfo, ViewMode viewMode) : bool
ShowAll() : bool
ToggleSelection(IEnumerable<int> numbers) : bool
TryGetObjectColor(int number, out Color color) : bool
TryGetSetColor(ISetInfo setInfo, out Color color) : bool
TryGetViewMode(ISetInfo setInfo, out ViewMode viewMode) : bool
```

IObjsPresenter

Пространство имён: `OperationalController`.

Исходники: IObjsPresenter.cs.

Базовые типы: `IEnumerable<ICoordinated>`.

```
Name : string [get, set]
PresenterType : PresenterType [get]
ViewMode : ViewMode [get, set]
Count() : int
CreateEdgeFlags(int edgesLength) : bool[]
CreateIndexes() : Tuple<int, int, int, int>
CreatePointers(int indexLength) : int[]
CreateVertexes(int vertexLength, string vertexType) : float[]
```

OperationalController-24 — Специализированные контракты контроллера

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

`IPresentersCreator`

Пространство имён: `OperationalController`.

Исходники: IPresentersCreator.cs.

```
CreateLineObjectsPresenter<T>(IReadOnlyList<T> lineObjects, Color color) : IObjsPresenter where T :
ICoordinated
CreateLineObjectsPresenter<T>(IReadOnlyList<T> lineObjects, IReadOnlyList<Color> colors) : IObjsPresenter
where T : ICoordinated
CreatePointObjectsPresenter(IReadOnlyList<IModelObject> pointObjects, Color color) : IObjsPresenter
CreatePointObjectsPresenter(IReadOnlyList<IModelObject> pointObjects, IReadOnlyList<Color> colors) :
IObjsPresenter
CreatePointObjectsPresenter<T>(IReadOnlyList<T> pointObjects, Color color) : IObjsPresenter where T :
ICoordinated
CreateSurfaceObjectsPresenter<T, U>(IReadOnlyList<ISurfaceObject<T, U>> surfaceObjects, Color color, bool
drawInnerSurfaces= true) : ISurfaceObjsPresenter where T : ISurface<U> where U : IPoint
CreateSurfaceObjectsPresenter<T, U>(IReadOnlyList<ISurfaceObject<T, U>> surfaceObjects, IReadOnlyList<Color>
colors, bool drawInnerSurfaces= true) : ISurfaceObjsPresenter where T : ISurface<U> where U : IPoint
CreateSurfaceObjectsPresenter<T>(IReadOnlyList<T> surfaceObjects, Color color) : ISurfaceObjsPresenter where T
: ICoordinated
CreateSurfaceObjectsPresenter<T>(IReadOnlyList<T> surfaceObjects, IReadOnlyList<Color> colors) :
ISurfaceObjsPresenter where T : ICoordinated
```

OperationalController-25 — Специализированные контракты контроллера

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

`IProjectController`

Пространство имён: `OperationalController`.

Исходники: IProjectController.cs.

Базовые типы: `IProjectSession`, `IProjectEvents`, `IProjectView`, `IProjectStorage`, `IMeshQueries`, `IMeshCommands`, `IGeometryQueries`, `IGeometryCommands`, `IGroupQueries`, `IGroupCommands`, `IConditionQueries`, `IConditionCommands`.

Собственных членов сверх показанного интерфейсного контракта нет; связи показаны на диаграмме.

OperationalController-26 — Специализированные контракты контроллера

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

IProjectEvents

Пространство имён: `OperationalController`.

Исходники: IProjectEvents.cs.

```
Message : EventHandler<ProjectMessageEventArgs>
```

IProjectSession

Пространство имён: `OperationalController`.

Исходники: IProjectSession.cs.

```
FilePath : string [get]
FunctionsDB : FunctionDBData [get, set]
HasProject : bool [get]
IsGeometryInitialized : bool [get]
IsModified : bool [get]
MaterialsDB : MaterialDBData [get, set]
Name : string [get]
TaskKind : TaskKind [get]
TaskType : TaskType [get]
Append(string path, IProgress<int> progress= null) : void
ChangeProjectName(string name) : void
ChangeTaskKind(TaskKind taskKind) : void
ChangeTaskType(TaskType taskType) : void
ClearAllData() : void
Close() : void
Create(string projectFile) : void
CreateTask() : void
CreateTask(MaterialDBData materials, FunctionDBData functions) : void
ImportGeometry(string path) : void
Open(string path, IProgress<int> progress= null) : void
Save(string path) : void
```

IProjectStorage

Пространство имён: `OperationalController`.

Исходники: IProjectStorage.cs.

```
CopyFile(string fileName, string oldFolder, string newFolder) : bool
ExportGeometry(string filePath) : void
ExportMesh(string filePath) : void
LoadSMF(string path) : void
SaveSMF(string path) : void
```

OperationalController-27 — Специализированные контракты контроллера

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

IProjectView

Пространство имён: `OperationalController`.

Исходники: IProjectView.cs.

Части: OperationalController-27, OperationalController-28.

```
HideInsideSurfaces : bool [get]
SelectedCount : int [get]
SelectedTypes : IEnumerable<ObjType> [get]
SelectionColor : Color [get]
Transparency : byte [get]
BeginViewUpdate() : IDisposable
ClearColor() : void
ClearColor(ObjType objType, IEnumerable<int> numbers) : void
ClearSelection() : void
ClearSelection(ObjType objType) : void
Deselect(ObjType objType, IEnumerable<int> numbers) : void
GetColor(ISetInfo setInfo) : Color
GetColor(ISetInfo setInfo, int number) : Color
GetColor(ObjType objType, int number) : Color
GetSelected(ObjType objType) : IEnumerable<int>
GetSelection() : IEnumerable<IModelObject>
GetViewMode(ISetInfo setInfo) : ViewMode
GetVisible(ObjType objType, int number) : bool
HideSelected() : void
IsSelected(ObjType objType, int number) : bool
Select(ObjType objType, IEnumerable<int> numbers) : void
SetColor(ISetInfo setInfo, Color color) : void
SetColor(ObjType objType, IEnumerable<int> numbers, Color color) : void
SetHideInsideSurfaces(bool hide) : void
```

OperationalController-28 — Специализированные контракты контроллера

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

IProjectView

Пространство имён: `OperationalController`.

Исходники: IProjectView.cs.

Части: OperationalController-27, OperationalController-28.

```
SetSelection(ObjType objType, IEnumerable<int> numbers) : void
SetSelectionColor(Color color) : void
SetTransparency(byte transparency) : void
SetViewMode(ISetInfo setInfo, ViewMode viewMode) : void
SetVisible(ObjType objType, IEnumerable<int> numbers, bool visible) : void
ShowAll() : void
ToggleSelection(ObjType objType, IEnumerable<int> numbers) : void
TryGetSingleSelectedType(out ObjType objType) : bool
```

ISelectionHelper

Пространство имён: `OperationalController`.

Исходники: ISelectionHelper.cs.

```
SelectE2DInPlane(IObjectsData objectsData, float angleInGradus, int startNumber) : List<int>
SelectNodeInDirection(IObjectsData objectsData, float angleInGradus, int numbFirst, int numbSecond) : List<int>
SelectNodeInPlane(IObjectsData objectsData, Plane plane) : List<int>
```

ISurfaceObjsPresenter

Пространство имён: `OperationalController`.

Исходники: ISurfaceObjsPresenter.cs.

Базовые типы: `IObjsPresenter`.

```
CreateSeparators() : int[]
```

IVolumeMeshCreator

Пространство имён: `OperationalController`.

Исходники: IVolumeMeshCreator.cs.

```
Sew3Dfrom2D(List<IElement2D> firstSet, List<IElement2D> secSet) : List<IElement3D>
```

IGmshController

Пространство имён: `OperationalController.GmshController`.

Исходники: IGmshController.cs.

Части: OperationalController-28, OperationalController-29.

```
Gmsh : Gmsh [get]
IsInitialized : bool [get]
CalcCurveLength(int curveTag) : double
CalcVolume(int volTag) : double
CreateControlPoints() : List<GeometryPoint>
CreateGeometry(double surfAngle= Math.PI / 2, double curveAngle= Math.PI) : void
CreateLines() : List<Curve>
CreateSurfaceData() : Tuple<IObjectsSet<SurfaceFigure>, VolumeData>
CreateSurfaceFigures() : List<SurfaceFigure>
CreateVolumeFigures() : List<VolumeFigure>
DeleteMeshElements(IntPtr[] elementTags) : void
GetMeshEntities(int dim, int tag, List<Node> nodes) : List<IElement>
GetMeshObjects() : (List<Node>, List<ILineObject<Node>>, List<Element2D>, List<Element3D>)
GetMeshSets<T>(List<Node> nodes) : IEnumerable<ObjectsSet<T>> where T : class, IElement
GetNodes(int dim= -1, int tag= -1, bool includeBoundary= true) : List<Node>
GetRecombineSurface(int tag) : bool
GetSize(int tag) : double
GetSurfaceNodes(int tag) : int[]
GetTransfiniteCurve(int tag) : string[]
GetTransfiniteSurface(int tag) : string[]
GetTransfiniteVolume(int tag) : string[]
Load(string dllToLoad) : void
MergeGeometry() : void
SetGradientVolume(int tag, double power, double distMax, double surfSize, double coreSize) : void
```

OperationalController-29 — Специализированные контракты контроллера

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

IGmshController

Пространство имён: `OperationalController.GmshController`.

Исходники: IGmshController.cs.

Части: OperationalController-28, OperationalController-29.

```
SetRecombineSurface(int tag) : void
SetSize(int tag, double meshDize) : void
SetTransfiniteCurve(int tag, int numNodes, MeshType meshType= MeshType.Progression, double coef= 1.0) : void
SetTransfiniteSurface(int tag, Arrangement arrangement= Arrangement.Left, int[] cornerTags= null) : void
SetTransfiniteVolume(int tag, int[] cornerTags= null) : void
Unload() : void
```

IExtrusionStrategy<TResult>

Пространство имён: `OperationalController.MeshObjsCreator.Extrude`.

Исходники: IExtrusionStrategy.cs.

```
AddElement(CurveExtruder extruder, Matrix4x4 correctMatrix, int outIndex, int inIndex) : void
GetResult() : TResult
Initialize(CurveExtruder extruder, object extrudeElements) : void
PostProcess(CurveExtruder extruder, bool rotateStatus, int curveIndex, int lastCurveIndex, Matrix4x4
lastCorrectMatrix) : void
PreProcess(CurveExtruder extruder, Matrix4x4 firstMatrix) : void
```

INormalCalculator

Пространство имён: `OperationalController.MeshObjsUtility`.

Исходники: INormalCalculator.cs.

```
CalcElemsNormals(IEnumerable<ISurfaceElement> elements) : IEnumerable<Curve>
CalcNodeNormals(IEnumerable<IElement3D> elements) : Dictionary<int, Point3D>
```

IObjsColors

Пространство имён: `OperationalController.ModelScenePresentator.GlObjsPresenters`.

Исходники: IObjsColors.cs.

```
Type : ObjsColorsType [get]
CreateColors(int objectsCount) : IEnumerable<Color>
```

ILineObjsVertexCreator<T>

Пространство имён: OperationalController.ModelScenePresentator.Interfaces.

Исходники: ILineObjsVertexCreator.cs.

```
Create(ref int stride, float[] vertexAtributes, IVertexObject<T> nodeObject, System.Drawing.Color color) : void
```

IPointObjsVertexCreator

Пространство имён: OperationalController.ModelScenePresentator.Interfaces.

Исходники: IPointObjsVertexCreator.cs.

```
Create(ref int stride, float[] vertexAtributes, IModelObject pointObject, System.Drawing.Color color) : void
```

OperationalController-30 — Специализированные контракты контроллера

Диаграмма. Обзор проекта.

Это целевой состав API с сохранением существующих операций. События вне `IProjectEvents.Message` исключены из целевого контракта; код ещё может их содержать. Расхождения.

Для классов члены реализуемых интерфейсов не повторяются. Внешние стандартные интерфейсы .NET не раскрываются. Полный состав типа при разбиении приведён на перечисленных ниже диаграммах.

ISurfaceObjsVertexCreator<T,U>

Пространство имён: `OperationalController.ModelScenePresentator.Interfaces`.

Исходники: ISurfaceObjsVertexCreator.cs.

```
Create(ref int stride, float[] vertexAtributes, IEnumerable<T> surfaces, System.Drawing.Color color) : void
```

IExtractor1DFrom2D

Пространство имён: `OperationalController.ObjsCreator`.

Исходники: IExtractor1DFrom2D.cs.

```
Create(int startNumber, IElement2D[] elements) : IEnumerable<Beam>
```

IExtractor2DFrom3D

Пространство имён: `OperationalController.ObjsCreator`.

Исходники: IExtractor2DFrom3D.cs.

```
Create(int StartNumber, IEnumerable<IElement3D> elements) : IEnumerable<Element2D>
FindOpenSurfaces(IEnumerable<IElement3D> elements) : IEnumerable<ElementSurface>
```

ISurfaceMeshCreator

Пространство имён: `OperationalController.SurfaceMeshCreator`.

Исходники: ISurfaceMeshCreator.cs.

```
DeloneTriangulation(IEnumerable<Node> nodes) : IEnumerable<IElement2D>
```

IMesh2DRefiner

Пространство имён: `OperationalController.Utilities`.

Исходники: IMesh2DRefiner.cs.

```
Refine(IEnumerable<IElement2D> elements, float elementSize) : void
```

ModelView: целевое представление без событий

`ModelView` и `IModelView` — внутренние типы сборки `OperationalController`. Клиент вызывает команды и запросы `IProjectView` через `IProjectController`. Событие для клиента объявлено в `IProjectEvents`, реализовано и вызывается только `ProjectController`. Нижние библиотеки и `ModelView` событий в целевой архитектуре не имеют.

Обзорная диаграмма, полный указатель типов и контрактов, правила контроллера, реестр расхождений.

Целевой протокол изменений

1. Контроллер выполняет команду представления. 2. Вид меняет своё состояние и формирует сведения о результате: затронутые наборы и флаги изменений. 3. Контроллер получает эти сведения без постоянной подписки на событие вида. 4. Контроллер создаёт `ViewChange` и сам вызывает `Message`.

Для `BeginViewUpdate` изменения объединяются до завершения внешнего пакета. Каждый набор передаётся с точными накопленными флагами; отсутствие изменения не создаёт сообщения. Этот пакет относится к представлению и не вводит общего накопителя событий геометрии, наборов, групп и условий: они сообщаются после выполненных шагов операции.

Способ передачи результата внутренней команды ещё требует проектирования API. Допустим результат операции или временный обратный вызов, переданный для конкретной команды/пакета. Событие `Changed` и постоянные подписки на него не сохраняются под другим названием. Существующие сигнатуры внутренних операций показаны на поддиаграммах; кроме исключения события, они не выданы за уже реализованный новый протокол.

Что уже реализовано и что требует миграции

В текущем коде `ChangedSets` накапливает флаги, `ModelView.Flush` создаёт `ModelViewChangedEventArgs` и вызывает `Changed`, а `ProjectController.OnViewChanged` преобразует данные в `ViewChange`. `Replace` переподписывает контроллер на новый вид. Это текущее устройство, которое должно быть изменено по EVT-05.

Сохранить при миграции нужно сведения по каждому набору, работу вложенных пакетов, поведение повторной команды без изменения и отсутствие ложного `ViewChange` при переносе настроек в новый вид. `ModelViewChangedEventArgs` — существующий DTO; имя с суффиксом `EventArgs` само по себе не требует наличия события. Его переименование не выполнено.

Данные модели и жизненный цикл

`Model` хранит сущности и группы; выделение, скрытость, цвета и настройки представления обрабатывает вид. Клиент не получает ссылку на текущий `ModelView`.

При замене проекта контроллер создаёт вид новой модели и переносит только `SelectionColor`, `Transparency` и `HideInsideSurfaces`. Выделение и индивидуальные стили не переносятся. После замены клиент получает `ProjectChange`; перенос настроек не требует отдельного `ViewChange`. `Append` сохраняет модель и её вид. Открытый пакет завершается до замены проекта.

Источники: ModelView.cs, ChangedSets.cs, IModelView.cs, ProjectSession.cs, ProjectView.cs.

История проектирования

Ниже сохранены прежние заметки, сценарии и открытые вопросы. Упоминания Changed, подписок, прежних API и клиентов описывают прежнее предложение либо состояние до миграции; они не определяют целевое размещение событий. Соответствие всех исторических замечаний текущему коду и клиентским приложениям в этой ревизии не подтверждается.

<details> <summary>Архив прежних решений и обсуждений</summary>

Задача

Скрытость и выделение объектов жили в самих объектах модели (Color, ViewState в IModelObject) и вычислялись через цвет: «выделено» означало «цвет равен SelectObjectColor» — 19 мест в GUI. Отсюда потеря выделения при смене прозрачности (MainMenuEvents.cs:107-109) и пять разбросанных вызовов SetBackColorToAllObjects() вместо одного уведомления.

Color и ViewState из IModelObject убраны. Наборы модели также не хранят цвет и режим отображения: всё состояние представления находится в ModelView, см. раздел «Цвет хранится в ModelView».

Состояние вида выносится из модели во внутренний ModelView. Модель не знает о представлении, представление знает о модели. Публичные операции вида находятся в IProjectView.

Единица хранения — тип, а не объект

ModelView хранит один словарь ObjType -> IObjectsView. Запись для каждого значения ObjType создаётся в конструкторе ModelView и существует всё время жизни вида. Методы чтения только возвращают это состояние и ничего не создают.

IObjectsView — состояние вида объектов одного типа. Внутри находятся два разреженных множества номеров (hidden, selected), словарь индивидуальных цветов объектов (objectColors) и словарь явных переопределений стилей наборов (setStyles).

Рассматривался вариант ObjType -> List<ViewObject>, где ViewObject держит признаки одного объекта. Отклонён:

1. Поиск. GetVisible и GetColor вызываются на каждый объект при сборке вершинных буферов. Множество номеров отвечает за O(1), список — за O(n) на каждый объект. 2. Память. Хранятся только исключения — скрытые, выделенные и индивидуально окрашенные объекты. Объект на запись это заголовок плюс поля вместо номера и, при необходимости, цвета. 3. Состояние по умолчанию. У разреженного множества «записи нет» означает «видим и не выделен» — структурная гарантия. С ViewObject запись надо создавать при первом признаке и удалять, когда все признаки вернулись к умолчанию, иначе коллекция растёт монотонно. Инвариант вместо гарантии. 4. Параллельный граф. ViewObject со ссылкой на IModelObject дублирует модель и расходится с ней при удалении и перенумерации — возврат к ViewState внутри объекта, от которого и уходим.

Ключ — (ObjType, Number)

Номера не уникальны между типами: узел N5 и элемент N5 — разные объекты. Поэтому ObjType выбирает IObjectsView, а Number ищется внутри него.

Разрешение цвета в GetColor

1. объект выделен → SelectionColor 2. иначе объект имеет индивидуальный цвет → цвет из objectColors 3. иначе набор имеет явно заданный цвет → цвет из SetStyle 4. иначе → стандартный цвет по ISetInfo.Number 5. к результату применяется Transparency (альфа)

Пункт 1 убирает нынешнее «выделено == цвет равен `SelectObjectColor`» и чинит потерю выделения при смене прозрачности.

Кто знает, в каком объект наборе

Вид не знает и знать не должен. Принадлежность объекта набору — это данные задачи (наборы элементов это части конструкции, материалы), и живёт она в модели: `IObjectsData.GetSetInfo(ObjType, int)`. `ObjectsView` хранит явные стили с ключом `ISetInfo`, сравниваемым по ссылке.

Но спрашивать это на каждый объект нельзя. Реализация для элементов такая:

```
E1DCollection.Select(x => x.Value).FirstOrDefault(x => x.ContainsKey(objNumber));
```

Линейный перебор наборов, в каждом `ContainsKey`, плюс аллокации LINQ — и всё это на каждый вызов. Если сборка буферов будет звать `GetColor(ObjType, int)` поштучно, получится O(объекты × наборы) и куча мусора на кадр.

Для точек, кривых, поверхностей и узлов набор ровно один на тип (`PointsSet`, `CurveSet`, `SurfaceSet`, `NodesSet`), так что проблема касается только элементов — но именно их в модели больше всего.

Сборка буферов идёт по наборам

Разворачиваем цикл наизнанку: снаружи наборы, внутри их объекты.

```
foreach (var setInfo in objects.GetSetsInfo(type))
{
    var mode = view.GetViewMode(setInfo);
    var visible = new List<IModelObject>();
    var colors = new List<Color>();
    foreach (var number in setInfo.GetNumbers())
    {
        if (!view.GetVisible(type, number))
            continue;
        visible.Add(objects.Find(type, number));
        colors.Add(view.GetColor(setInfo, number));
    }
    ...
}
```

Набор разрешается один раз на набор, а не на объект: `GetColor(ISetInfo, int)` уже знает набор, поэтому на объект остаются O(1)-проверки выделения, индивидуального цвета и явного стиля набора. Медленного `GetSetInfo(objType, number)` с перебором наборов в горячем цикле нет. Что делают с этими двумя списками дальше — в разделе «Что получает презентер».

Это и естественный порядок для отрисовки: буфер всё равно нарезается по наборам и режимам (`ViewMode`), так что снаружи цикла оказывается ровно то, что задаёт состояние отрисовки.

`GetColor(ObjType, int)` при этом остаётся, но его место — точечные запросы: панель свойств, подсказка под курсором, отладка. В контракте это надо написать прямо, иначе рано или поздно его позовут в цикле.

`GetColor(ISetInfo)` возвращает исходный цвет набора без применения общей прозрачности. Прозрачность применяется только при получении итогового цвета конкретного объекта через одну из двухобъектных перегрузок `GetColor`.

Ключ стиля — сам набор, а не его имя

Переименование наборов в Gerbera есть, поэтому ключевать стили строкой нельзя: после переименования цвет потерялся бы, а набор получил бы новый случайный.

Имя как ключ требовало бы `RenameSet(ObjType, oldName, newName)` на виде и правила «операция переименования обязана его позвать» — забыл вызвать, цвет молча сбросился. Ещё одно правило того же сорта, что `ClearSelection` перед заменой, и такое же молчаливое при нарушении.

Правило не нужно: `ObjectsSet.Name` объявлен как `{ get; internal set; }`, то есть переименование происходит внутри сборки `Model` и правит тот же экземпляр. Значит ключом надо брать сам `ISetInfo` по ссылке — стиль переживает переименование сам собой, без единой строчки кода и без правила, которое можно забыть.

`ISetInfo.Number` не используется как ключ `setStyles`: стиль по-прежнему привязан к экземпляру набора. Номер служит только для O(1)-выбора стандартного цвета из палитры.

Стабильный номер набора и стандартный цвет

Для наборов одного `ObjType` значение `Number` назначает `ObjectsCollection<T>`:

- `Number` назначается при добавлении набора в `ObjectsCollection<T>`; - счётчик `nextSetNumber` увеличивается после каждого успешного добавления; - удаление набора не изменяет номера оставшихся; - `Clear()` очищает коллекцию и сбрасывает `nextSetNumber` в ноль; - при загрузке наборы получают новые номера в порядке чтения; - после повторного открытия стандартные цвета могут отличаться; - в пределах жизни одной `ModelData` цвета остаются стабильными; - `ModelView` только читает `setInfo.Number` и ничего не создаёт.

Одиночные наборы узлов, точек, кривых и поверхностей получают `Number = 0` в конструкторе `ObjectsData`. Новый `ObjectsSet<T>` до включения в коллекцию имеет `Number = -1`; запрос стандартного цвета такого набора считается ошибкой.

Стандартный цвет вычисляется без обхода наборов:

```
var colorIndex = setInfo.Number % setColors.Length;
return setColors[colorIndex];
```

Для узлов стандартным остаётся `Color.Blue`, для геометрических точек — `Color.FromArgb(255, 207, 64)`. Для остальных типов используется циклическая палитра `SteelBlue`, `Orange`, `MediumSeaGreen`, `MediumOrchid`, `Goldenrod`, `IndianRed`.

Номер пока не сохраняется в файле модели. Это сознательное ограничение: стабильность гарантируется в пределах текущего экземпляра `ModelData`, а не между сеансами работы.

В текущем контракте `ISetInfo.Number` имеет публичный setter. Поэтому уникальность обеспечивает штатный путь через `ObjectsCollection<T>`; ручное изменение номера внешним кодом остаётся нарушением этого правила.

Практические детали:

- `ObjectsSet` не переопределяет `Equals` и `GetHashCode`, так что ссылочное равенство работает по умолчанию. Словарь всё же стоит создать с явным `ReferenceEqualityComparer.Instance` — это фиксирует намерение и защищает от того, что когда-нибудь кто-то добавит `Equals` по имени. - Словарь держит ссылки на наборы, поэтому удалённый набор не освободится сам. `Prune` должен выбрасывать стили наборов, которых больше нет в `GetSetsInfo(type)`. С ключом по имени это понадобилось бы точно так же. - Сигнатуры `IObjectsView` принимают `ISetInfo`, а не строку — совпадают с `IModelView` и не заставляют вызывающего доставать имя.

`Changed` и `BeginUpdate` объединяют изменения внутри вида. На `Changed` подписан контроллер; он пересылает изменение клиентам как `Message(ViewChange)`. Клиенты вызывают `BeginViewUpdate()` на `IProjectView`.

Команды как операции над множеством

Команда	Операция	Номера вне аргумента
`Select`	объединение	не трогает
`Deselect`	вычитание	не трогает
`ToggleSelection`	симметрическая разность	не трогает
`SetSelection`	замена	снимает выделение
`ClearSelection`	опустошение	—

`SetSelection` держим отдельным методом, а не парой `ClearSelection` + `Select`: одно уведомление `Changed` вместо двух и невозможно забыть первый шаг.

Каждая команда `IObjectsView` возвращает `bool` «состояние изменилось». По нему `ModelView` решает, поднимать ли `Changed`. Записи из словаря не удаляются: состояния всех типов создаются в конструкторе и существуют всегда. Благодаря этому чтение не имеет побочных эффектов, а повторная команда с тем же состоянием не перерисовывает сцену без причины.

Сценарии Gerbera

Действие	Модификатор	Команда
клик	нет	`SetSelection` — выделен только кликнутый
клик	Shift	`ToggleSelection` — невыделенный добавляется, выделенный выбывает
контур	нет	`SetSelection` — выделены только попавшие
контур	Shift	`Select` — попавшие добавляются к текущим
контур	Ctrl	`Deselect` — попавшие вычитаются из текущих
скрыть выделенные	—	`HideSelected()`
удалить выделенные	—	операция контроллера, см. ниже

Модификаторы остаются понятием GUI. Вид даёт команду на каждую операцию над множеством, чтобы вызывающий выбирал одну, а не собирал её из нескольких.

Отбор попаданий обязан фильтровать по `GetVisible`, иначе контур захватит скрытое.

Shift на клике и на контуре означает разное

Клик с Shift снимает выделение с уже выделенного объекта (симметрическая разность), контур с Shift — не снимает ни с кого (объединение). Это не опечатка в требованиях, а сознательное разделение: одиночный клик — это переключатель, контур — накопитель. Зафиксировано здесь, потому что при рефакторинге возникает соблазн свести обе ветки к одной команде «Shift = добавить», и тогда клик по выделенному объекту перестанет его снимать.

Ctrl задействован только на контуре, Alt не задействован вовсе.

Режим «любой тип»

Когда на GUI выбран тип «Объект», попасть может объект любого типа, и выделение становится многотиповым.

«Объект» — понятие Gerbera, а не элемент `ObjType`: в перечислении только `Точка`, `Кривая`, `Поверхность`, `Объем`, `Узел`, `Элемент1D`, `Элемент2D`, `Элемент3D`. Благодаря этому граница проходит сама собой — в `IModelView` «Объект» попасть не может даже по ошибке, и ключ словаря `views` всегда конкретный тип. Защитная проверка на входе не нужна.

Роль режима — сказать, какие типы рассматривать при отборе. Результатом в любом режиме служат пары (фактический тип, номер). Разница только в том, сколько разных типов окажется в результате.

Отсюда же польза `SelectedTypes` и `TryGetSingleSelectedType`: команды, требующие однородного выделения (создание группы, условия), в этом режиме обязаны проверить, что тип один.

PickHandler — роль, а не существующий класс

В BazisCore такого типа нет, и появиться он там не должен: попадание в объект считается там, где есть камера, матрицы проекции и буферы, то есть в Gerbera. На диаграмме он обозначает границу — кто именно вызывает команды выделения.

Его работа — превратить событие ввода в вызовы вида:

1. развернуть точку или контур в попадания, разрешив неоднозначность (клик рядом с узлом, лежащим на элементе) по своим правилам приоритета; 2. отбросить скрытое через `GetVisible`; 3. перевести модификатор в команду по таблице выше; 4. вызвать её — для одного типа один раз, для нескольких циклом.

Чего он не делает: не хранит выделение — оно живёт в `ModelView`; не знает о подсветке — это `GetColor`; не решает, что с выделенным будет дальше.

Набор номеров даёт не только он. В `OperationalController` есть `ISelectionHelper` с выборками по геометрии: `SelectE2DInPlane`, `SelectNodeInPlane`, `SelectNodeInDirection` — каждая возвращает номера одного типа. Это второй поставщик, и он тоже должен заканчиваться вызовом команды `IModelView`, а не собственной раскраской. Команды вида не должны знать, откуда пришёл набор: от клика, от контура, от выборки по плоскости или из группы.

На диаграмме `ISelectionHelper` не показан — он уводил картинку за лимит ширины, а к разделению модели и вида отношения не имеет.

Многотиповые операции — цикл, а не носитель набора

Все изменяющие команды принимают `ObjType` и `IEnumerable<int>`. Если типов несколько, вызывающий делает цикл внутри `BeginUpdate`.

Отдельный носитель набора здесь был и убран. Сначала завели `ISelectedObjects` и `SelectedObjects` — словарь `ObjType -> HashSet<int>` с методом `Add`, чтобы многотиповая замена была одной атомарной операцией. Доводы против оказались сильнее:

- атомарность уже даёт `BeginUpdate` — одно `Changed` на весь пакет, поэтому главный довод за носителя не работает; - тип нигде не хранится, он транзитный: возникает на событии мыши и живёт до конца вызова. Это словарь, у которого чтение целиком покрывается BCL; - пять перегрузок в `IModelView` плюс интерфейс плюс класс — заметный вес ради того, что делает цикл в три строки на стороне вызывающего.

Цена решения — два правила для вызывающего, оба ниже. Если когда-нибудь набору понадобится нести что-то ещё (порядок выбора, режим), носитель вернётся.

Правило 1: замена начинается с ClearSelection

`SetSelection(type, ids)` означает «выделено ровно это», включая другие типы. В цикле по типам каждый вызов стирал бы предыдущий, поэтому многотиповая замена пишется так:

```
using (view.BeginUpdate())
{
    view.ClearSelection();
    foreach (var type in hitTypes)
        view.Select(type, GetHits(type));
}
```

Забытый `ClearSelection` даёт накопление выделения от контура к контуру — самая вероятная ошибка в этой схеме, потому что молчаливая.

Правило 2: перед изменением — материализовать

`GetSelection()`, `GetSelected(ObjType)` и `SelectedTypes` ленивы. Удаление или любая другая мутация во время обхода рвёт перечисление. Снимок делает вызывающий, одним `ToList`, и делает его осознанно — там, где собирается менять модель.

`GetSelection()` отдаёт `IEnumerable<IModelObject>` через `yield return`: у объекта есть и `ObjType`, и `Number`, так что вызывающему хватает его одного, а `ModelView` достаёт объект через `ObjectsData.Find`. Для одиночных наборов это словарный поиск; для элементов поиск дополнительно проходит по наборам типа.

Последовательность вызовов по сценариям

`view` — это `IModelView`.

Клик без модификатора, тип выбран

1. хит-тест, для каждого кандидата `view.GetVisible(type, n)` 2. `view.SetSelection(type, new[] { 42 })` 3. внутри `ModelView`, под `BeginUpdate`: `v.ClearSelection()` у всех записей `views`, кроме `type`; `GetView(type).SetSelection(...)`; одно `Changed` на выходе

Клик с Shift, тип выбран

1. хит-тест, `GetVisible` 2. `view.ToggleSelection(type, new[] { 42 })` → `GetView(type).ToggleSelection(...)`, другие типы не трогаются

Контур без модификатора, режим «Объект»

1. отбор попаданий с `GetVisible`, группировка по фактическому типу 2. `using (view.BeginUpdate())`, затем `view.ClearSelection()` и цикл `view.Select(type, numbers)` по типам 3. одно `Changed` на выходе из пакета

Контур с Shift — то же без `ClearSelection`: цикл `view.Select(type, numbers)`.

Контур с Ctrl — цикл `view.Deselect(type, numbers)`. Внутри `ModelView` `GetView` возвращает заранее созданное состояние указанного типа и ничего не создаёт.

Скрыть выделенные

1. `view.HideSelected()` 2. под `BeginUpdate` по каждой записи `views`: `v.SetHidden(v.GetSelected(), true)`, затем `v.ClearSelection()`

Снимок не нужен: всё происходит внутри вида, `SetHidden` пишет в `hidden`, читая `selected` — это разные множества.

Удалить выделенные

```
var doomed = view.GetSelection().ToList();      // правило 2
foreach (var obj in doomed)
    objects.Remove(obj.ObjType, obj.Number);
view.Prune();
```

Синхронизация с моделью

Номера объектов меняет только явная перенумерация: `ObjectsData.Remove` удаляет запись из набора, но чужих номеров не трогает. Значит источников рассинхрона два — удаление (лечится `Prune`) и перенумерация (`Reset`).

Prune

Скрытость и выделение хранятся по номерам, поэтому вид не узнаёт об удалении сам: номер удалённого остаётся в множестве. Множества растут от каждого цикла «создал-удалил», а номера переиспользуются, так что новый объект оказывается «уже выделенным» или «уже скрытым».

`Prune` спрашивает, а не получает список: вид отдаёт свои номера на проверку предикату `Predicate<int>` `exists`. Проверяются `selected`, `hidden` и ключи `objectColors`. Чистить нужно единицы записей, а список существующих — это проход по всей модели и лишняя коллекция. Сам `ObjectsView` о модели не знает, предикат замыкает `ModelView`:

```
IObjectsData objects = model.ObjectData;
bool changed = view.Prune(number => objects.Find(type, number) != null);
```

Для одиночных наборов `ObjectsData.Find` обращается к словарю за O(1). Для элементных типов `ObjectsCollection.Find` последовательно проверяет наборы, поэтому текущая стоимость `Prune` зависит и от числа записей вида, и от числа наборов соответствующего типа.

Кроме номеров объектов, `Prune` удаляет явные стили наборов, которых больше нет в `GetSetsInfo(type)`.

`IModelView.Prune()` обязан проходить по всем типам, а не только по тем, что удаляли. `ObjectsData.Remove` каскадный: удаление точки дёргает `CurveSet.ClearNotExisted()` и `SurfaceSet.ClearNotExisted()`, то есть уносит с собой кривые и поверхности. Попытка «оптимизировать» чистку до затронутых типов оставит в виде номера объектов, исчезнувших по цепочке.

Ради этого предиката сначала были заведены интерфейс `IObjectExistence` и класс-адаптер `ObjectsExistence`. Два типа делали работу одного параметра — убраны.

Название `Prune` — из садоводства, «подрезать», срезать засохшие ветки. Если метафора читается плохо, ясная замена — `RemoveMissing` или `DropMissing`.

Перенумерация: пока сбрасываем вид целиком

`ObjectsRenumber` карту старый → новый не отдаёт. Принятое решение: после перенумерации `ProjectController` зовёт `IModelView.Reset()` — выделение, скрытость и индивидуальные цвета теряются, но явно, а не подменяются чужими объектами. `Reset()` обходит виды, очищает эти три состояния и поднимает одно `Changed`; явные стили наборов он не трогает (см. раздел про цвет).

Следствие: `Remap` в интерфейсах нет. Метод без вызывающего вводить незачем. Когда `Renumber` научится отдавать карту, `Remap` вернётся, а `Reset` уйдёт.

Второе следствие: `ObjectsRenumber` ушёл с диаграммы. Он попал туда только из-за стрелки `Remap`; без неё он никак не связан с представлением и остаётся обычной операцией над моделью.

ObjectsRenumber: что он делает

Живёт в `OperationalController.MeshObjsUtility`. Делает сплошную перенумерацию объектов одного типа: обходит набор и присваивает номера подряд с нуля через `SetNumber`. Реализован только `ObjType.Узел`; для `Точка` и `Поверхность` бросает исключение, для остальных — `NotImplementedException`. Ветка для элементов закомментирована, вместо неё тот же алгоритм живёт вторым экземпляром в `ProjectController.RenumberElements`, который нумерует `E1D`, `E2D` и `E3D` сквозным счётчиком.

Старый номер теряется в момент `SetNumber` — отсюда и отсутствие карты. Перенумерация ничего не удаляет, поэтому `Prune` промолчал бы, а номера в виде остались бы валидными, указывая на другие объекты: пользователь увидел бы, что подсвечено не то, что он выделял, без единой ошибки в логе.

Лечится дёшево: `Renumber` возвращает `IReadOnlyDictionary<int,int>` — обход объектов всё равно идёт, достаточно запомнить старый номер перед `SetNumber`. Заодно стоит свести `RenumberElements` в ту же реализацию.

Удаление выделенных

Удаление меняет модель, поэтому его нет в `IModelView`: вид хранит состояние, а не командует моделью. Место операции — `ProjectController`, у которого есть и модель, и вид. Порядок обязателен:

1. снимок выделения (`ToList`) — до изменений 2. `objects.Remove(type, number)` по каждому объекту снимка 3. `view.Prune()`

Без шага 1 удаление рвёт перебор выделения. Без шага 3 номера удалённых остаются в множествах.

Скрытие снимает выделение

Иначе следующая команда — удаление, фаска, создание условия — выполнится над объектами, которых пользователь не видит. `HideSelected()` делает скрытие и `ClearSelection()` одним пакетом.

Что сценариями не востребовано

Не мешает, но и не оправдано, пока не назван сценарий:

- `ClearSelection(ObjType)` — снять выделение одного типа. Ни клик, ни контур этого не делают. - `SetVisible(ObjType, IEnumerable<int>, bool)` — скрытие идёт только от выделения, через `HideSelected()`. Форма с номерами нужна разве что команде «скрыть всё, кроме выбранного». - `GetColor(ISetInfo, int)` — сценариями выделения не востребован, но нужен сборке буферов по наборам, см. «Что получает презентер».

Цвет хранится в ModelView

`IModelObject` и `ISetInfo` не хранят цвет и режим отображения. `ISetInfo` остаётся информационным интерфейсом: имя, номер, тип, количество, номера, координаты и центр. Форматы модели не сохраняют состояние представления.

`ObjectsView` хранит два независимых слоя цвета:

- `objectColors` — разреженный словарь индивидуально окрашенных объектов; - `setStyles` — словарь явных переопределений стиля набора по ссылке на `ISetInfo`.

`SetStyle.Color` и `SetStyle.ViewMode` допускают `null`. Поэтому установка только цвета не создаёт неявного переопределения режима и наоборот. Методы `TryGetSetColor` и `TryGetViewMode` только читают словарь; `SetSetColor` и `SetViewMode` создают `SetStyle`, когда действительно выполняется команда изменения.

Внутренний `IModelView` реализует, а публичный `IProjectView` делегирует:

- `GetColor(ISetInfo)` и `SetColor(ISetInfo, Color)` для цвета набора; - `SetColor(ObjType, IEnumerable<int>, Color)` и `ClearColor(...)` для индивидуальной окраски объектов; - `GetViewMode(ISetInfo)` и `SetViewMode(...)` для режима отображения набора.

Стандартные значения не хранятся в `ObjectsView`: `ModelView` вычисляет цвет по `ISetInfo.Number`, а режим — по `ObjType`. Поэтому чтение стандартного цвета или режима не создаёт `SetStyle` и вообще не изменяет состояние.

`Reset()` очищает выделение, скрытость и индивидуальные цвета, но сохраняет явные стили наборов. `Prune()` удаляет индивидуальные цвета отсутствующих объектов и стили удалённых наборов.

Изменения публикуются через `Changed` с флагами `SetColor`, `ObjectColor`, `ViewMode`, `SelectionColor` и `Transparency`, поэтому клиент получает точную причину необходимой перерисовки.

ICoordinated переезжает в Geometry

`ICoordinated` объявлен в `Geometry` (`PrGeometry\Interfaces\ICoordinated.cs`), а не в `Model` — этот шаг в коде уже сделан. Структурно он законен: `Model` уже ссылается на `Geometry`, там же живёт `Point3D`, который интерфейс возвращает, — цикла не возникает. По смыслу тоже верно: «сколько у объекта точек и какие» — чистая геометрия, к расчётной модели отношения не имеет.

Но зависимость PostProc от Model это не снимает. Ожидание было именно такое, и оно не подтвердилось. По коду PostProc ссылается на `Model.csproj` и использует оттуда:

Тип	Упоминаний	Где
`IModelObject`	17	`ResultFigure`, `ResultSurface`, `ResultPoint` — они его реализуют
`ObjType`	8	там же
`IPoint`	5	`ResultsFieldsCreator`, `IsoLevelFieldsCreator`

Тип	Упоминаний	Где
`ISurfaceElement`	4	`ResultsFieldsCreator`
`ILineObject`	4	`LineGradientsCreator`
`ISurfaceObject`	1	`ResultFigure` реализует его
`Node`	1	`ResultsFieldsCreator`

Все восемь затронутых файлов лежат в `ScenePresenter`; остальной PostProc — экспорт, слияние результатов, контроллер — про `Model` не знает вовсе.

Дальше зависимость распадается на две части.

Уйдёт само. `IModelObject` и `ObjType` — это 25 из 40 упоминаний, и почти все они в `ResultFigure`/`ResultSurface`/`ResultPoint`, которые сегодня реализуют `Number`, `ObjType`, `ViewState`, `Color`, `SlaveColor`, `InitialColor`, `Dim`, `CompareTo`, `Equals`, `GetHashCode`, `SetNumber` — при том что `Number` у них всегда 0, а `ObjType` захардкожен «Поверхность». Это ровно та уборка, что уже записана ниже: объекты сцены перестают быть объектами модели.

Останется по-настоящему. `ResultsFieldsCreator`, `IsoLevelFieldsCreator` и `LineGradientsCreator` читают сетку — `Node`, `ISurfaceElement`, `ILineObject`, `IPoint`. Поля результатов строятся по узлам и элементам, это суть работы, а не случайная связка. Снять её можно двумя способами: перенести и эти интерфейсы в `Geometry` (они тоже описывают форму, а не расчёт), либо завести в PostProc собственный контракт на «источник сетки» и реализовать его на стороне Model.

Итого перенос `ICoordinated` был правильным и дешёвым, но это первый шаг из трёх, а не решение задачи целиком.

Заодно к уборке

- `ISurface<T>` переобъявляет `NumberOfPoints` своей копией — наследовать `ICoordinated`. - В `IModelObject` соседствуют `Dim` и `ObjType`, хотя `ObjType` однозначно задаёт размерность, а `Dim` хранится полем и может с ним разойтись. - `ResultFigure`/`ResultSurface`/`ResultPoint` больше не `IModelObject`: это объекты сцены (`Number` всегда 0, `ObjType` захардкожен, цвет задаётся шкалой `ItemRange`). Отсюда же уходят `CompareTo`, `Equals`, `GetHashCode`, `SetNumber`, `SlaveColor`, `InitialColor`, `Dim`.

Объём изменений в Gerbera

Оценка по коду `C:\Gerbera\GUI` на 19.09.2026. Всего там 323 файла `.cs`, из них 284 не `.Designer.cs`. Просмотрено 137 файлов, отобранных по релевантности (сцена, выборка, цвет, наборы, навигатор, панель свойств, VBO) — в остальных затрагиваемого API по именам и структуре каталогов не ожидается, но полной гарантии это не даёт.

Каркас уже стоит

`Methods\ModelViewOperations.cs` в Gerbera существовал для прежнего контракта: подписывался на `IModelView.Changed`, пересобирает буферы обходом `foreach ObjType → foreach setInfo`, читает `IsSelected` и `GetVisible`, прокидывает `SelectionColor` из настроек. В CORE `IModelView` и `IObjectsView` реализованы внутри CORE; цвет наборов и индивидуальные цвета объектов также находятся в виде. При миграции Gerbera подписку нужно перевести на `IProjectController.Message`.

Поэтому вопрос не «сколько переписывать», а «сколько осталось».

Что придётся тронуть

Что	Вхождений	Файлов
`SelectObjectColor`	8	3
`.ViewState`	10	3
`SetBackColor`	6	4
`.SetColor(`	1	1
`ViewMode`	49	11
`ISetInfo`	17	10

Работа распадается на три сорта.

Механика, без раздумий. `obj.ViewState` → `GetVisible`/`SetVisible` (`ShowAdjacencies`, `UtilityToolStrip`, `VBOController`); `SetBackColorToAllObjects` исчезает, его заменяет одно `Changed` (`ShowHideEvents`, `Buttons\SelectObjects`, `SceneEvents`, `SelectByPoint`). Порядка полутора десятков мест.

Требует подумать — три файла. `Scene\SelectObjects\SelectByPoint.cs` (9.7 КБ), `Scene\SelectObjects\SelectByRect.cs` (5.9 КБ) и `Scene\Buttons\SelectObjects.cs`: сюда ложится таблица модификаторов и правило «замена начинается с `ClearSelection`». Плюс `Scene\VBObjects\ColorObjects.cs` — сейчас он перекрашивает буферы по типам объектов, после переноса цвета его работа сводится к `SetColor(ISetInfo, Color)` и одному `Changed`, и файл, скорее всего, исчезает целиком.

Самая объёмная часть — `ViewMode`, 49 вхождений. Но она почти вся в слое VBO (`VBOController`, `VBObject`, `SurfaceObjects`, `PointObjects`, `IVBObject`, `ISceneControl`), где режим просто читается у набора; замена — читать у вида. `DisplayToolStrip` (12 вхождений) — это переключатель режима в UI, его вызов уходит в `SetViewMode` на виде. Работа монотонная, но однотипная.

Чего не коснётся

Диалоги, базы материалов и функций, консоль, анимация, гант, навигатор задач, локализация, настройки решателя, мастера импорта — примерно 85% файлов Gerbera. Изменения сосредоточены в каталогах `Scene`, `Methods\Scene`, `Methods\Navigator` и `PropertiesPanel`.

Где вероятнее всего сломается

Забытый `ClearSelection` перед циклом в `SelectByRect`: рамки начнут копить выделение вместо замены. Отказ молчаливый, в логе ничего. Это первый кандидат в ручной тест после переделки.

NullModelView удалён

Проверено по коду 21.09.2026, до удаления.

В тестах он не использовался ни разу. `TestProject\ControllerTests\ModelViewTests.cs` одиннадцать раз строит настоящий `new ModelView(...)`. Версия «держим ради тестов» не подтвердилась.

Создавался он в двух местах:

1. `ProjectController.ModelView`, когда `ModelData == null` — «проект не открыт»; 2. перегрузки `PresentersCreator` без вида и безпараметрические конструкторы презентеров. Этих перегрузок нет в `IPresentersCreator`, то есть код, работающий через интерфейс, до них не дотягивался.

Принятое решение

`NullModelView` удалён целиком. В текущем контроллере всегда есть внутренний `ModelView` для пустой или открытой модели; публичного `ProjectController.ModelView` нет. Перегрузки без вида и безпараметрические конструкторы презентеров были удалены вместе с `NullModelView`.

Что этим выигрывается: правила цвета существуют ровно в одной реализации. До удаления `NullModelView` дублировал `GetDisplayColor`, разбор цвета из `ModelView` и `GetDefaultViewMode` из `ObjectsView`, причём цвет по умолчанию уже разошёлся — `ObjectsView` раздавал палитру (`SteelBlue`, `Orange`, `MediumSeaGreen`, ...), а `NullModelView` возвращал `DarkGray` для всего, кроме узлов. Одна модель выглядела по-разному в зависимости от того, какой вид подставлен. Второго источника умолчаний больше нет.

Заодно уходит дефект: геттер создавал `new NullModelView(null)` при каждом обращении, из-за чего `ReferenceEquals` в `SubscribeToModelView` всегда был ложен, и Gerbera на каждом вызове переподписывалась на одноразовые объекты с пустыми `add`/`remove`. Теперь клиент подписывается один раз на `ProjectController.Message`.

Чем платим

В Gerbera остаются обращения к прежнему `project.ModelView` (при прежней оценке — 71 обращение в 26 файлах). Их нужно перевести на операции `IProjectView` через контроллер. Обработчики вида должны подписываться на `Message(ViewChange)` и при необходимости переносить обновление в поток UI. Сам CORE предоставляет пустую модель и внутренний вид даже до открытия проекта.

Что остаётся непреложным

Презентеры `null` не принимают: конструктор бросает `ArgumentNullException`. Это не обход, а контракт — нет вида, нечего и собирать, а в пустой модели нет и наборов, так что цикл сборки просто не выполнится.

Что получает презентер

Презентеру нужны видимость объекта и его цвет. Перебрали пять вариантов, прежде чем нашёлся правильный, и все промежуточные разбились об один и тот же камень: презентер получал *способ спросить* про вид вместо *ответа*.

`IObjectsView` вместо `IModelView` полем. Отклонено: в нём нет `SelectionColor` и `Transparency` — они общемодельные. И он изменяющий: `Select`, `SetHidden`, `Prune`.

`ISetDisplay` как поле презентера. Заводился и отменён: интерфейс плюс класс ради кэша из двух значений, да ещё с семантикой снимка, за которой надо следить.

Создание презентеров внутри `ModelView`. Отклонено: вид получил бы зависимость от слоя отрисовки и вторую ответственность — фабрику графических объектов. Плюс результаты PostProc собираются презентером, которому вид не нужен вовсе.

`IModelView` плюс `ISetInfo` полями, как в коде сегодня. Работает, но в презентер уезжает тридцать членов интерфейса вместо двух вопросов, и он теоретически может позвать `Reset()` посреди сборки.

`IDrawState` параметром каждого метода сборки. Снимал вопрос устаревания — состояние живёт один вызов, — но ломал сам объект: презентер без переданного состояния ничего не мог, то есть самостоятельным объектом не был, а четыре сигнатуры обрастали параметром. Попытка накрыть тем же интерфейсом градиентные фигуры PostProc довела до `ScaleDrawState` в `OperationalController` — типа пост-процессора в чужой сборке. Это и был признак, что абстракция подгоняет под себя раскладку проектов, а не наоборот.

Принято: два списка — объекты и цвета

Презентер получает результат, а не источник:

```
public SurfaceObjsPresenter(IReadOnlyList<ISurfaceObject<T, U>> objects, IReadOnlyList<Color> colors)
{
    if (objects == null) throw new ArgumentNullException(nameof(objects));
    if (colors == null) throw new ArgumentNullException(nameof(colors));
    if (objects.Count != colors.Count)
        throw new ArgumentException("Число цветов не совпадает с числом объектов");

    this.objects = objects;
    this.colors = colors;
}

public SurfaceObjsPresenter(IReadOnlyList<ISurfaceObject<T, U>> objects, Color color)
```

Списки параллельны: `colors[i]` — цвет `objects[i]`. Длины сверяются в конструкторе, дальше индекс общий и проверять нечего.

`IObjsPresenter` возвращается к сигнатурам без параметра состояния:

```
public interface IObjsPresenter
{
    string Name { get; }
    ViewMode ViewMode { get; set; }
    PresenterType PresenterType { get; }

    Tuple<int,int,int,int> CreateIndexes();
    int[] CreatePointers(int indexLength);
    float[] CreateVertexes(int vertexLength, string vertexType);
    bool[] CreateEdgeFlags(int edgesLength);
    int Count();
}
```

Только видимые. Список объектов уже отфильтрован, поэтому проверок видимости внутри презентера нет вовсе — ни `IsVisible`, ни `GetVisible`. Скрыть объект значит не положить его в список.

Вторая перегрузка — один `Color` на всех. Нормали и предпросмотр не выделяют массив из N одинаковых значений. Четыре случая закрываются двумя конструкторами, и ни одного нового типа не появляется.

Кто собирает цвета

Случай	Кто собирает	Чем
набор модели	`ProjectController`	`GetVisible` отсеивает, `GetColor(ISetInfo, int)` красит
результаты PostProc	Gerbera	шкала `ItemRange` по значению в объекте
нормали, предпросмотр	место вызова	один `Color`, вторая перегрузка
объекты вне `ModelData`	место вызова	один `Color` либо собственный массив

Ответ на вопрос «как рисовать» даётся стандартными типами — `IReadOnlyList<T>` и `Color`. Поэтому в сигнатурах создателя вида нет, `ScaleDrawState` не нужен, а из `PresentersCreator` исчезают парные перегрузки «с видом» и «без вида» вместе с `CoordinatedSurfaceObjsPresenter`.

Чем это лучше предыдущих попыток

- Презентер — законченный объект: после конструктора он умеет всё, что обещает интерфейс. Это и было главное возражение против варианта с параметром. - Устаревать нечему: цвета посчитаны на момент создания, а презентер набора пересоздаётся на `Changed` — как и сегодня. - Ни одного нового типа: ни `ISetDisplay`, ни `IDrawState`, ни `Colored<T>`. `Colored<T>` в BCL нет — `Tuple/ValueTuple` пришлось бы либо заворачивать в свой тип (снова новый тип, да ещё в спорной сборке), либо тащить кортежи в публичные сигнатуры, чего мы избегаем. - Правило цвета остаётся в одном месте — в `ModelView.GetColor`. Три вершинных сборщика, которые сегодня дублируют «выделен → `SelectionColor`», получают готовый цвет и правила не знают.

Чем платим

Два списка на набор — это две аллокации на пересборку. Размер их равен числу видимых объектов набора, то есть той же величины, что и буфер вершин, который строится тут же рядом; на фоне VBO это незаметно.

Второе — дисциплина вызывающего: положить в списки одно и то же количество и в одном порядке. Проверка длины в конструкторе ловит первую половину ошибки, порядок не ловит никто. Это цена за то, что презентер не знает о виде.

Следствия для `IModelView`

`GetColor(ISetInfo, int)` остаётся и получает главного вызывающего — сборку буферов по наборам, см. «Где живёт цикл отбора». `GetColor(ObjType, int)` остаётся для точечных запросов: панель свойств, подсказка под курсором, отладка.

`GetDisplayColor(Color)` уходит внутрь `ModelView`: по коду его звали два цветовых сборщика (теперь они получают готовый цвет), сам `ModelView` и удалённый `NullModelView`; в Gerbera — ни разу.

Режим отображения

`ViewMode` является частью стиля набора. Явное переопределение хранится в `SetStyle`, а стандартный режим `ModelView` вычисляет по `ObjType`. При создании презентера `ProjectController` получает режим через `ModelView.GetViewMode(setInfo)` и присваивает его презентеру.

Где живёт цикл отбора

Списки кто-то должен наполнить. Рассматривались два места.

Метод вида. `IModelView` получил бы пару обобщённых методов:

```
IEnumerable<TObj> GetVisible<TObj>(ISetInfo setInfo, IEnumerable<TObj> objects) where TObj : IModelObject;
IReadOnlyList<Color> GetColors<TObj>(ISetInfo setInfo, IReadOnlyList<TObj> objects) where TObj : IModelObject;
```

Отклонено. Параметр типа здесь понадобился не по делу: внутри вида тип объекта стирается до `IModelObject`, а презентеру нужен `ISurfaceObject<T, U>`, поэтому тип пришлось бы протаскивать обратно только затем, чтобы не появилось приведение вниз. Дженерики решали проблему, которую сами же и создавали. Плюс два новых члена в интерфейсе, где их и так три десятка.

Метод контроллера — принято. У `ProjectController` типизированная коллекция уже на руках, стирать нечего, обобщения не нужны:

```
public IObjsPresenter CreateModelObjectsPresentor(ISetInfo setInfo)
{
    var visible = new List<ISurfaceObject<T, U>>();
    var colors = new List<Color>();

    foreach (var obj in surfaceObjs)
    {
        if (!modelView.GetVisible(obj.ObjType, obj.Number))
            continue;

        var color = modelView.GetColor(setInfo, obj.Number);
        visible.Add(obj);
        colors.Add(color);
    }

    return creator.CreateSurfaceObjectsPresenter(visible, colors);
}
```

Переезжает цикл, а не правило. Правило «выделен → `SelectionColor`, иначе индивидуальный цвет, иначе цвет набора, затем `Transparency`» остаётся в `ModelView`: контроллер цвет спрашивает, а не вычисляет. Обход знанием не является, поэтому его вынос ничего не размазывает.

Условие: цикл существует ровно в одном методе

Если он окажется в двух местах, вернётся худшая ошибка этой схемы — забытый `GetVisible` и молча отрисованные скрытые объекты. Раньше от неё защищал сам презентер, теперь защищать некому: он про видимость не знает вовсе.

Поэтому сборка набора — метод контроллера, а не код на стороне Gerbera:

```
var sets = modelData.ObjectData.GetSetsInfo(objType);
foreach (var setInfo in sets)
{
    var presenter = controller.CreateModelObjectsPresentor(setInfo);
    // Передать presenter в VBO сцены.
}
```

Цикл по наборам может принадлежать клиенту, но диспетчеризация по `ObjType` на точки, линии и поверхности и цикл фильтрации объектов конкретного набора живут в `CreateModelObjectsPresentor`.

Побочный, но важный эффект: `RefreshModelSetBuffer` в Gerbera становится короче, а не длиннее — он получает готовые презентеры и раздаёт их в VBO. Это единственный способ удержать оценку «85% Gerbera не трогаем»: если писать цикл с `GetVisible` и `GetColor` на стороне GUI, изменённых файлов станет заметно больше.

Судьба `GetColor(ISetInfo, int)`

Этот член в ходе проработки убирался и возвращался трижды, поэтому решение стоит зафиксировать вместе с правилом, по которому оно принято: член живёт в интерфейсе, только если назван вызывающий вне `ModelView`.

Здесь вызывающий назван — `CreateModelObjectsPresentor`. Член остаётся.

Внутренние грани объёмных элементов

Раздел написан по следам регрессии: при переходе на `ModelView` (коммит `18f74ce`, 18.09.2026) пропала возможность не рисовать грани 3D-элементов, закрытые соседними элементами. Одновременно исчезли `ChangeInsideSurfacesState` из `IMeshCommands` и `ProjectController`, алгоритм `MeshObjsUtility/ChangeInsideSurface` и учёт `ViewState` отдельных граней в презентерах.

Это не украшение, а обязательный шаг

Для сетки из гексаэдров N×N×N всего граней $6N^3$, на поверхности $6N^2$ — отношение ровно N. Для 50×50×50 (125 тыс. элементов) это 750 тыс. граней против 15 тыс.

По текущему коду `CreateIndexes` на треугольник отводит 9 float координат и 12 float цвета, то есть 42 float на четырёхугольную грань — 168 байт.

	граней	координаты и цвет
все грани	750 000	~126 МБ
только внешние	15 000	~2,5 МБ

Это видеопамять, и она пересобирается на каждый `Changed`. Без отсечения объёмная сетка рабочего размера неотображаема.

В других пакетах это называется free faces (Femap, HyperMesh), skin (Abaqus), surface extraction (VTK — `vtkDataSetSurfaceFilter`, через него ParaView строит представление Surface). Алгоритм везде один: грань с двумя владельцами внутренняя. То есть удалённый код реализовывал канонический подход.

Важно, что другие уровни отсечения его не заменяют. Z-буфер даёт верную картинку только в непрозрачном режиме и платит за всю геометрию. Backface culling убирает половину оставшегося, но внутренние грани не трогает. Occlusion culling решает другую задачу — деталь за деталью в сборке — и платит за это каждый кадр, тогда как внутренняя грань скрыта всегда и убирается один раз.

Отсюда же следствие для текущего состояния: в полупрозрачном режиме внутренние грани сейчас проступают сеткой внутри детали, а совпадающие грани соседних элементов дают z-fighting при вращении.

Почему нельзя вернуть старый метод

`ChangeInsideSurfacesState` совмещал три вещи: вычисление совпадающих граней, хранение результата в `surface.ViewState` и чтение этого поля презентером. Второе мы убрали намеренно — и правильно: грань не имеет собственного номера, ключ `(ObjType, Number)` её не адресует, а хранимая производная величина немедленно потребовала бы синхронизации с `Prune`, `Reset` и перенумерацией.

Первое и третье восстанавливаются по отдельности.

Принято: грань знает своих хозяев

Признак не хранится вовсе — хранится связь, а `IsInner` считается на ходу.

Устройство взято у `Node`, где это уже сделано: узел держит `List<IElement>` с `AddElement`, `RemoveElement` и `GetElements()`, и его `ExistState` каскадно гасит владеющие элементы. То есть владение в этой модели — не справка, а поддерживаемая связь.

Грань отличается тем, что одного владельца знает с рождения: её строит собственный элемент. Поэтому нового поля нужно ровно одно — сосед.

```
public class ElementSurface : ISurface<Node>
{
    Node[] nodes;
    readonly IElement owner;
    IElement neighbour;

    public ElementSurface(IElement owner, int surfaceNumber, Node[] nodes)
    {
        if (nodes.Length < 3)
            throw new Exception("Для создания поверхности требуется не меньше 3-х узлов!");

        this.owner = owner ?? throw new ArgumentNullException(nameof(owner));
        this.nodes = nodes;
        Number = surfaceNumber;
    }

    public int Number { get; }

    public int ElementNumber => owner.Number;

    public bool IsInner => neighbour != null && neighbour.ExistState;

    public IEnumerable<IElement> GetElements()
    {
        yield return owner;

        if (neighbour != null)
            yield return neighbour;
    }

    public int[] CreateNodesKey()
    {
        var key = new int[nodes.Length];
        for (var index = 0; index < nodes.Length; index++)
            key[index] = nodes[index].Number;

        Array.Sort(key);
        return key;
    }

    internal void SetNeighbour(IElement element)
    {
        neighbour = element;
    }
}
```

Конструктор принимает элемент, а не его номер: номер тогда не дублируется, а ссылка переживает перенумерацию — `ObjectsRenumber` уже был источником тихих ошибок, и повторять это на новом поле не стоит.

Список хозяев по образцу `Node` был бы буквальнее, но дороже: 750 тыс. граней — это 750 тыс. объектов `List` с их массивами, порядка 50 МБ и полтора миллиона объектов для GC. Снаружи `GetElements()` выглядит как у узла, внутри это две ссылки без единой аллокации.

Бонус от `ExistState` в `IsInner`. Удаление элементов само обнажает грани соседей — без разлинковки и без единого вызова. А удаление в GUI и есть самая частая правка сетки. Поэтому `Mark` нужен только после добавления элементов, а не после любой правки. Хранимый `bool` здесь соврал бы и показал полость сплошной.

Признак объявляется на `ISurface<T>`

Не на `ElementSurface` и не на `IElementSurface`, хотя по смыслу место там. Причина в типах: `ISurfaceElement : ISurfaceObject<ElementSurface, Node>` подставляет конкретный класс, а презентер обобщён как `SurfaceObjsPresenter<T, U> where T : ISurface<U>`. Член, объявленный ниже `ISurface<T>`, презентеру не виден без приведения вниз.

У `GeometrySurface`, `SurfaceFigure` и `ResultSurface` ответ всегда отрицательный — три строки.

Связывание

```
public class MarkSurfaces
{
    public void Mark(IEnumerable<ISurfaceElement> elements)
    {
        if (elements == null) throw new ArgumentNullException(nameof(elements));

        var found = new Dictionary<int[], ElementSurface>(new SurfaceNodesComparer());
        var defects = new List<ElementSurface>();

        foreach (var element in elements)
        {
            foreach (var surface in element.GetSurfaces())
            {
                var key = surface.CreateNodesKey();
                var opposite = default(ElementSurface);

                if (!found.TryGetValue(key, out opposite))
                {
                    surface.SetNeighbour(null);
                    found[key] = surface;
                    continue;
                }

                if (opposite.IsInner)
                {
                    defects.Add(surface);
                    continue;
                }

                var oppositeOwner = GetOwner(opposite);
                var ownerOfSurface = GetOwner(surface);
                opposite.SetNeighbour(ownerOfSurface);
                surface.SetNeighbour(oppositeOwner);
            }
        }

        ReportDefects(defects);
    }
}
```

Один проход, а не два: со ссылкой соседа можно проставить сразу при встрече пары, счётчик не нужен.

`SetNeighbour(null)` на первой встрече обязателен — повторный `Mark` после добавления элементов должен рвать старые связи, иначе грань, ставшая внешней, останется помеченной внутренней.

Третья одинаковая грань — дефект сетки: пара уже связана, у неё `IsInner` истинен, и молча затереть соседа нельзя. Такие грани попадают в отчёт, а не теряются.

Ключ строит сама грань, но не запоминает. `CreateNodesKey` живёт в `ElementSurface`: ключ — функция её узлов, и знать о них должна она. Внутри класса есть прямой доступ к `nodes`, поэтому одна аллокация вместо четырёх — ни `List`, ни `ToArray`, ни итератора `GetVertexes()`.

Сохранять результат мы отказались. `IModelObject.SetNumber` делает номера узлов изменяемыми, и после перенумерации сохранённый ключ молча неверен — не только по значениям, но и по порядку сортировки. Это та же ловушка, из-за которой соседом хранится ссылка, а не номер. Публичный сеттер `this[int ind]` добавляет второй путь к устареванию. При этом ключ нужен только внутри `Mark`, а массивы живут и умирают в нулевом поколении — самый дешёвый сценарий; кэш перевёл бы 750 тыс. массивов во второе поколение навсегда, около 30 МБ. Инвалидация обошлась бы в полный обход граней, то есть в тот самый проход, ради которого кэш заводился.

Компаратор нужен свой. Существующий `ElementSurfaceEqualityComparer` делегирует в `s.GetHashCode()`, а `ElementSurface` его не переопределяет — возвращается хеш по ссылке, и в `Dictionary` равные грани получают разные хеши.

```
public class SurfaceNodesComparer : IEqualityComparer<int[]>
{
    public bool Equals(int[] first, int[] second)
    {
        if (first.Length != second.Length)
            return false;

        for (var index = 0; index < first.Length; index++)
        {
            if (first[index] != second[index])
                return false;
        }

        return true;
    }

    public int GetHashCode(int[] nodeNumbers)
    {
        var hash = 17;
        foreach (var number in nodeNumbers)
            hash = hash * 31 + number;

        return hash;
    }
}
```

Ключ переменной длины, поэтому квадратичные элементы (`Tetra_c`, `Triangle_c`) проходят без оговорок.

Что получает презентер

Один `bool`. Не маска, не список, не делегат.

```
public SurfaceObjsPresenter(IReadOnlyList<ISurfaceObject<T, U>> surfaceObjects, IReadOnlyList<Color> colors,
bool drawInnerSurfaces)

private IEnumerable<T> GetDrawnSurfaces(int index)
```

```
{
    foreach (var surface in surfaceObjects[index].GetSurfaces())
    {
        if (drawInnerSurfaces || !surface.IsInner)
            yield return surface;
    }
}
```

Через этот метод идут все шесть обходов граней: `CreateIndexes`, `CreateSeparators`, `CreateEdgeFlags` и три вершинных сборщика. У сборщиков меняется сигнатура — вместо объекта они принимают отобранные грани:

```
void Create(ref int stride, float[] vertexAtributes, IEnumerable<T> surfaces, Color color);
```

Это улучшение и само по себе: сборщик вершин перестаёт знать про объект и работает с тем, что рисует.

Флаг приходит из `IModelView.HideInsideSurfaces` через `CreateModelObjectsPresentor`. Контракт «презентер о виде не знает» не нарушается: `bool` — такой же стандартный тип, как `Color`.

`HideInsideSurfaces` общемодельный, а не пер-сетовый: грань может быть общей для элементов из разных наборов, и пер-сетовый флаг дал бы для одного куска геометрии два противоречивых ответа.

Это не повторение ошибки с ViewState

`ViewState` был решением пользователя — что показать. Владение гранью — топология сетки, того же рода, что связность или `ExistState`. Модель имеет право знать собственную топологию; вид решает только, рисовать ли внутренние грани.

Побочная выгода: та же связь нужна `NormalsCalculator`, `FindOpenSurfaceNodes` и `FindBoundaryEdges` — все они по сути спрашивают про внешнюю оболочку.

Отклонённые варианты

Вернуть `ViewState` на грань. Хранимая производная величина со всеми проблемами синхронизации, ради которых затевалось разделение.

Маска граней третьим параллельным списком. `IReadOnlyList<bool[]>` рядом с цветами. Работало бы, но маски пришлось бы пересобирать при каждой пересборке буфера, а признак на самой грани не требует ничего.

Расплющить список до граней. Единицей презентера стала бы грань. Чинит отрисовку и ломает выбор: `CreateSeparators()` и `Count()` дают одну запись на объект, и по ним Gerbera отображает диапазоны VBO обратно на элементы модели.

Декоратор элемента, чей `GetSurfaces()` отдаёт только внешние грани. Презентер не меняется вовсе, но `IModelObject` тянет `IEquatable`, `IComparable` и `IEqualityComparer`, а `GetEnumerator()` презентера отдаёт объекты наружу: обёртка уехала бы в Gerbera и перестала быть равной исходному элементу. Ошибка тихая и всплыла бы в выборе, а не в картинке.

Отдельный объект-результат с кэшем в `ProjectController`. Нужен был, пока признак не жил на грани. С хозяевами на грани кэшировать нечего.

Дефекты, найденные попутно

- `ElementSurface` не реализует `IElementSurface`, хотя интерфейс существует и содержит ровно её `ElementNumber`. Интерфейс лежит неиспользуемым. - `ElementSurfaceEqualityComparer` непригоден как компаратор словаря, см. выше. - `ElementSurface.Equals(ElementSurface)` не переопределяет `object.Equals`, поэтому `Contains`/`Distinct` по `ISurface<Node>` его не позовут. Сравнивает координаты с допуском и только по трём узлам. - `SurfacesCreator.CreateSurfacesFromSetOfNodes` статический, перебирает тройки узлов за O(n³) и вызывается из закомментированной строки в `Element3D` — конкретные элементы строят грани сами. Похоже на мёртвый код.

Тесты

Регрессия прошла молча, потому что теста не было. Минимум:

- два гексаэдра с общей гранью: 12 граней, 10 внешних, у каждого элемента ровно одна грань с `IsInner`; - одиночный гексаэдр: 6 внешних, ни одной внутренней; - те же два гексаэдра, затем `ExistState = false` у одного: у второго все грани снова внешние, без повторного `Mark`; - повторный `Mark` после добавления третьего элемента не оставляет старых связей.

Не проверено

- Объявлен ли `GetSurfaces()` в `ISurfaceElement` и стабилен ли порядок граней. Для признака на грани порядок некритичен, в отличие от варианта с маской. - Передача `this` из конструктора `Hexa` в `ElementSurface` даёт ссылку на ещё не достроенный объект. Сейчас безопасно — грань обращается к `owner` только после конструктора, — но это ограничение стоит помнить. - Есть ли в Gerbera режим сечения. Если да, при отсечении плоскостью внутренние грани обязаны проступить, иначе деталь выглядит полой; тот же вопрос к разнесённому виду.

Открытый вопрос

В режиме «Объект» при клике по точке, где узел лежит на элементе, приоритет типа выбирает обработчик по своим правилам или пользователь через настройку? От ответа зависит, нужен ли виду фильтр допустимых для выборки типов, или это целиком забота Gerbera.

</details>